Empirical Software Engineering (2025) 30:65
https://doi.org/10.1007/s10664-025-10614-4

Bugs in large language models generated code: an empirical study

Florian Tambon[1] · Arghavan Moradi-Dakhel[1] · Amin Nikanjam[1] · Foutse Khomh[1] · Michel C. Desmarais[1] · Giuliano Antoniol[1]

Accepted: 10 January 2025 / Published online: 13 February 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract
Large Language Models (LLMs) for code have gained significant attention recently. They can generate code in different programming languages based on provided prompts, fulfilling a long-lasting dream in Software Engineering (SE), i.e., automatic code generation. Similar to human-written code, LLM-generated code is prone to bugs, and these bugs have not yet been thoroughly examined by the community. Given the increasing adoption of LLM-based code generation tools (e.g., GitHub Copilot) in SE activities, it is critical to understand the characteristics of bugs contained in code generated by LLMs. This paper examines samples of 333 bugs collected from code generated using three leading LLMs (i.e., CodeGen, PanGu-Coder, and Codex) and identifies the following 10 distinctive bug patterns: *Misinterpretations, Syntax Error, Silly Mistake, Prompt-biased code, Missing Corner Case, Wrong Input Type, Hallucinated Object, Wrong Attribute, Incomplete Generation, and Non-Prompted Consideration*. The bug patterns are presented in the form of a taxonomy. The identified bug patterns are validated using online surveys with over 50 LLM practitioners and researchers. The surveyed participants generally asserted the significance and prevalence of the bug patterns. Researchers and practitioners can leverage these findings to develop effective quality assurance techniques for LLM-generated code. This study sheds light on the distinctive characteristics of LLM-generated code.

Keywords Large language models · Bugs · Software testing · Empirical study

1 Introduction

The recent surge in the development of Large Language Models (LLMs) tailored for code generation has garnered significant attention (Wong et al. 2022; Imai 2022; Sakib et al. 2023).

Communicated by: Andy Zaidman

Florian Tambon and Arghavan Moradi-Dakhel contributed equally to this work

This work was supported by: Fonds de Recherche du Québec (FRQ), the Canadian Institute for Advanced Research (CIFAR) as well as the DEEL project CRDPJ 537462-18 funded by the Natural Sciences and Engineering Research Council of Canada (NSERC) and the Consortium for Research and Innovation in Aerospace in Québec (CRIAQ), together with its industrial partners Thales Canada inc, Bell Textron Canada Limited, CAE inc and Bombardier inc.

Extended author information available on the last page of the article

经验软件工程 (2025) 30:65
https://doi.org/10.1007/s10664-025-10614-4

大语言模型生成代码中的错误：一项实证研究

Florian Tambon[1] ·Arghavan Moradi-Dakhel[1] ·Amin Nikanjam[1] · Foutse Khomh[1] ·Michel C. Desmarais[1] ·Giuliano Antoniol[1]

接受日期：2025年1月10日 / 在线发表日期：2025年2月13日 © 作者，专属授权给斯普林格科学商务媒体有限公司，斯普林格自然集团的一部分，2025年

摘要
用于代码的大型语言模型（LLMs）最近引起了广泛关注。它们可以根据提供的提示生成不同编程语言的代码，实现了软件工程（SE）中长期以来的梦想，即自动代码生成。与人类编写的代码类似，LLM生成的代码也容易出现漏洞，而这些漏洞尚未被社区彻底研究。鉴于基于LLM的代码生成工具（例如GitHub Copilot）在SE活动中越来越多地被采用，了解LLM生成代码中漏洞的特征至关重要。本文检查了使用三种领先LLM（即CodeGen、盘古-Coder和Codex）生成的代码中收集到的333个漏洞样本，并识别出了以下10种独特的漏洞模式：误解、语法错误、愚蠢错误、提示偏向代码、缺失边界情况、错误输入类型、虚构对象、错误属性、不完整生成和非提示考虑。

关键词 大型语言模型 · 错误 · 软件测试 · 实证研究

1 引言

最近，针对代码生成的大型语言模型（LLM）的开发激增，引起了广泛关注（Wong 等，2022；Imai，2022；Sakib 等，2023）。

通讯作者：Andy Zaidman Florian Tambon 和 Arghavan Moradi-Dakhel 对此工作贡献相同

本研究得到了以下单位的支持：魁北克研究基金会（FRQ）、加拿大高级研究院（CIFAR），以及由加拿大自然科学与工程研究理事会（NSERC）资助的 DEEL 项目 CRDPJ 537462-18 及魁北克航空航天研究与创新联盟（CRIAQ），以及其工业合作伙伴 Thales Canada inc、Bell Textron Canada Limited、CAE inc 和 Bombardier inc。

扩展作者信息可在文章的最后一页查看

Transformer-based models such as Codex (Chen et al. 2021) and Llama-2 (Touvron et al. 2023), trained on large amounts of open-source code repositories, have demonstrated success in producing code across various programming languages, such as Python, Java, and C (Yu et al. 2023; Jin et al. 2023; Liu et al. 2023). These models approach code generation as a transformation process, converting natural language descriptions (prompts) into executable programming language statements.

Positioned as potential AI pair programmers (Moradi Dakhel et al. 2023; Imai 2022; Bird et al. 2022) in software projects, LLM-based code generators are poised to play a crucial role in the quality of the overall software project. However, similar to human-written code, LLM-generated code is prone to errors (Moradi Dakhel et al. 2023). Nonetheless, Asare et al. (2023) who recently examined vulnerabilities contained in LLM-generated code (using Copilot) compared to those in human-written code, reported that LLM models do not generate the same vulnerabilities as humans. In parallel, several studies (Li et al. 2024; Kou et al. 2023) reported that LLMs and human developers may not focus on the same part of the prompt to solve a coding task. This raises an important question: Do LLMs generate faults similar to those produced by human developers? This question is crucial because the effectiveness of popular quality assurance techniques like mutation testing depends on a precise characterization of faults occurring in the code under test (Just et al. 2014).

To illustrate the potential differences between bug patterns in LLM-generated code and those in human-written code, let's consider the example from Listing 1 which presents the code generated by Codex (Chen et al. 2021), a well-known LLM to generate code, for the task of "returning the flags present in the Python ArgumentParser object" (shown in green as a docstring in the figure).

Listing 1 An example of a code generated by Codex Chen et al. (2021). The LLM is given the function signature and docstring (shown in green) as a prompt and should generate the code to address the task in the prompt. We labeled this buggy code as " Non-Prompted Consideration (NPC)" in our study.

```
1  def parser_flags(parser):
2      '''
3      Given an argparse.ArgumentParser instance, return its argument flags in a space−separated
4      string.
5      '''
6      flags = []
7      for action in parser._actions:
8          if isinstance(action, argparse._SubParsersAction):
9              for choice, subparser in action.choices.items():
10                 flags.append(choice)
11                 flags += parser_flags(subparser)
12         else:
13             flags.append(action.option_strings[0])
14     return " ".join(sorted(flags))
```

While the code seems reasonably correct at first glance, the attentive reader will notice in the last line (Line 14) that the LLM suddenly decided to sort the flags it extracted using the 'sorted' function. This step, not requested in the prompt, leads to the code being faulty for the requested use case. This kind of snippet is interesting as it highlights two key points: 1) LLMs might generate a seemingly correct code at first glance, which could deceive non-experienced users particularly if no (or not enough) testing is performed, and 2) LLMs may add a non-prompted feature to the code, leading to an error, something that a human developer would rarely do, which shows that LLMs bugs might not be exactly similar to human-made bugs.

基于 Transformer 的模型，如 Codex（Chen 等, 2021）和 Llama-2（Touvron 等, 2023），在大量开源代码库上进行训练，已经证明在生成各种编程语言代码方面取得了成功，如 Python、Java 和 C（Yu 等, 2023；Jin 等, 2023；Liu 等, 2023）。这些模型将代码生成视为一种转换过程，将自然语言描述（提示）转换为可执行的编程语言语句。

作为软件项目中潜在的人工智能配对程序员（Moradi Dakhel 等, 2023；Imai, 2022；Bird 等, 2022），基于大语言模型（LLM）的代码生成器有望在整体软件项目的质量中发挥关键作用。然而，与人类编写的代码类似，LLM生成的代码也容易出错（Moradi Dakhel 等, 2023）。尽管如此，Asare 等（2023）最近对LLM生成的代码（使用Copilot）中存在的漏洞与人类编写的代码中的漏洞进行了比较，报告称LLM模型不会生成与人类相同的漏洞。与此同时，一些研究（Li 等, 2024；Kou 等, 2023）报告称，LLM与人类开发者在解决编码任务时可能不会关注提示的相同部分。这提出了一个重要问题：LLM生成的故障是否类似于人类开发者产生的故障？这个问题至关重要，因为像变异测试这样的流行质量保证技术的有效性依赖于精确的特征描述。

为了说明大型语言模型（LLM）生成的代码中的错误模式与人类编写的代码中的错误模式可能存在的差异，我们来看示例清单1，该清单展示了Codex（Chen 等, 2021）生成的代码，这是一个著名的用于生成代码的LLM，用于"返回Python ArgumentParser对象中存在的标志"（如图中绿色文档字符串所示）的任务。

列表 1 由 Codex Chen 等人（2021）生成的代码示例。LLM 根据函数签名和文档字符串（显示为绿色）作为提示，并应生成代码以处理提示中的任务。在我们的研究中，我们将这段有错误的代码标记为"非提示考虑（NPC）"。

```
1  def 解析器_标志(解析器):
2      '''
3      给定一个 argparse.ArgumentParser 实例，返回其以空格分隔的参数标志
4      字符串。
5      '''
6      旗帜 = []
7      for action in parser._actions:
8          if isinstance(action, argparse._SubParsersAction):
9              对于 choice, subparser 在 action.choices.items() 中:
10                 flags.append(选择)
11                 flags += 解析器_flags(子解析器)
12         否则:
13             flags.append(action.option_strings[0])
14     return " ".join(sorted(flags))
```

虽然乍一看代码似乎相当正确，但细心的读者会注意到在最后一行（第14行），LLM 突然决定使用 'sorted' 函数对提取的标志进行排序。这个步骤在提示中并未请求，导致代码在所需的用例中出错。这种代码片段很有趣，因为它突出了两个关键点：1）LLM 可能乍一看生成似乎正确的代码，这可能会误导经验不足的用户，尤其是在没有（或测试不足）的情况下；2）LLM 可能会向代码中添加未提示的功能，从而导致错误，这是人类开发者很少做的事情，这表明 LLM 的错误可能与人类造成的错误并不完全相同。

While there exist studies that investigated bugs in LLM-generated code (Fan et al. 2023; Liu et al. 2023; Pan et al. 2023), to the best of our knowledge, none of them examined the bug patterns in code generated by LLMs, when applied to real programming tasks (i.e., involving the implementation of functionalities).

This paper fills this gap in the literature by examining the characteristics of bugs contained in code generated by three leading LLMs: CodeGen, PanGu-Coder, and Codex. The identified characteristics are then organized in a taxonomy and validated via a survey of practitioners and researchers. Our investigation is conducted using the following Research Questions (RQs):

RQ1 What are the characteristics of bugs occurring in code generated by LLMs for real-world project tasks?

RQ2 To what extent are the identified bug patterns in LLM-generated code relevant for software practitioners and researchers working with LLMs?

We conducted our study using CoderEval (Yu et al. 2023), which is a dataset of 230 Python functions selected from 43 real-world open-source projects hosted on GitHub. Each of the selected LLMs was used to generate code for the different tasks described in CoderEval. We identified the bugs contained in the obtained code and manually analyzed the characteristics of two random samples of 333 bugs using two types of prompt provided in CoderEval (either the original docstring or human annotated label). This led us to organize our findings in a taxonomy composed of 10 categories of bugs: *Misinterpretations, Syntax Error, Silly Mistake, Prompt-biased code, Missing Corner Case, Wrong Input Type, Hallucinated Object, Wrong Attribute, Incomplete Generation, and Non-Prompted Consideration.* To assess the validity of the obtained bug patterns, we performed two online surveys. The first one, with 20 software practitioners and researchers working with LLMs, aimed at discovering any new bug patterns we may have missed in our sampling. The second one, with 45 software practitioners and researchers working with LLMs, aimed to validate our bug patterns and analyze their characteristics through practitioners' lenses. The obtained results show that respondents encountered the different bug patterns we observed, in a proportion similar to that obtained from our sample set.

This paper makes the following contributions:

- We study bug patterns occurring in LLM-generated code and their prevalence.
- We propose a taxonomy from the characteristics of the bug patterns in LLM-generated code.
- We validate our findings using surveys and provide insights for researchers and practitioners.
- We make the dataset used in this study publicly available online (The replication package 2023) for other researchers and practitioners to replicate or build upon our work.

The rest of the paper is organized as follows We present the background of the study in Sect. 2. Next, we detail the methodology of the study in Sect. 3 for both the empirical study of bugs and the validation survey. We then report our empirical investigation of bugs in LLM-generated code in Sect. 4.1. The validation survey results are presented in Sect. 4.2. We provide a discussion on our findings in Sect. 5. Then, we discuss the related works and threats to the validity of this study in Sects. 6 and 7, respectively. Finally, we present the conclusion and future work in Sect. 8.

虽然存在一些研究调查了LLM生成代码中的错误（Fan 等, 2023；Liu 等, 2023；Pan 等, 2023），据我们所知，没有研究在实际编程任务（即涉及功能实现）中，考察LLM生成代码的错误模式。

本文通过研究由三种领先的大型语言模型（LLM）生成的代码中的错误特征来填补文献中的这一空白：CodeGen、盘古-Coder 和 Codex。识别出的特征随后被组织为一个分类法，并通过对从业者和研究人员的调查进行验证。我们的研究使用以下研究问题（RQs）进行：

RQ1 在由大型语言模型生成用于实际项目任务的代码中出现的错误有哪些特征?

RQ2 在使用大型语言模型（LLM）生成的代码中发现的错误模式在多大程度上与使用LLM的软件从业者和研究人员相关?

我们使用 CoderEval（Yu 等，2023）进行了研究，CoderEval 是一个包含 230 个 Python 函数的数据集，这些函数从 GitHub 上托管的 43 个真实开源项目中选取。我们使用每个选定的 LLM 为 CoderEval 中描述的不同任务生成代码。我们识别出所获得代码中的错误，并使用 CoderEval 提供的两种类型的提示（原始 docstring 或人工标注标签）手动分析了两组随机抽取的 333 个错误的特征。基于此，我们将研究结果整理成一个由 10 类错误组成的分类体系：误解、语法错误、低级错误、提示偏向代码、缺失边界情况、输入类型错误、虚构对象、属性错误、生成不完整以及未提示的考虑。为了评估获得的错误模式的有效性，我们进行了两项在线调查。第一项调查对象为 20 位从事 LLM 的软件从业人员和研究人员，旨在发现任何新的错误模式

本文做出了以下贡献：

– 我们研究了出现在大型语言模型生成代码中的错误模式及其普遍性。 – 我们根据大型语言模型生成代码中错误模式的特征提出了一种分类法。 – 我们通过调查验证我们的发现，并为研究人员和从业者提供见解。 – 我们将本研究使用的数据集公开发布在网上（The replication package 2023），供其他研究人员和从业者复现或在此基础上开展工作。

论文的其余部分安排如下：我们在第2节介绍研究背景。接下来，我们在第3节详细说明研究方法，包括对漏洞的实证研究和验证调查。然后，我们在第4.1节报告对LLM生成代码中漏洞的实证调查。验证调查结果在第4.2节呈现。我们在第5节对研究发现进行讨论。随后，我们分别在第6节和第7节讨论相关工作和本研究有效性威胁。最后，我们在第8节给出结论和未来工作。

2 Background

In this section, we introduce the three LLMs used in our study and describe the open coding methodology followed in this paper.

2.1 LLMs for Code Generation

Different LLMs have been introduced for automatic code generation in Software Engineering (SE). One highly potent LLM is OpenAI's Codex (Chen et al. 2021), which has been extensively employed in diverse code-related SE tasks (Nguyen and Nadi 2022; Lemieux et al. 2023; Chen et al. 2023). This closed-source GPT-based auto-regressive LLM (Brown et al. 2020) boasts up from 12 million to 12 billion parameters and undergoes fine-tuning on 54 million public repositories from GitHub. Notably, Codex is the model behind GitHub Copilot,[1] an in-IDE developer coding assistant, proficient at generating code based on user-provided context with the maximum input length of 4,096 tokens. A recent addition to GitHub Copilot is "Copilot Chat",[2] which is tuned with human feedback for dialog use cases. Copilot Chat proves versatile for a broad range of code-related tasks. It excels at generating code fragments, describing code snippets in natural language, generating unit tests, and fixing buggy code, all tailored to the specific context of the task at hand.

PanGu-Coder (Christopoulou et al. 2022) is an open-source pre-trained LLM for generating code from text. This model is based on PanGu-α architecture (Zeng et al. 2021), a unidirectional decoder-only transformer with an additional layer for querying added on top of it. PanGu-Coder was trained in two stages using Python programs extracted from GitHub. In the first stage (unsupervised), the model is pre-trained on raw programming language data containing docstrings/inline comments written in natural language. Next, this model is fine-tuned to generate code from text in a second stage (supervised). Two versions of this model were published and leveraged for code generation recently (Zan et al. 2022). These versions have respectively 317 million and 2.6 billion parameters, with a maximum input length of 1,024 tokens.

CodeGen (Nijkamp et al. 2022) is a family of LLMs trained for text-to-code generation. Based on a decoder-only architecture, this family of models is designed for multi-turn program synthesis, in which a user engages with the LLM by gradually feeding specifications in natural language to obtain a corresponding program. This collaborative interaction allows the user, in conjunction with the LLM, to iteratively build and refine a program in multiple steps. CodeGen models were trained on a natural language corpus and code extracted from GitHub (i.e., a multilingual code dataset, and a dataset of Python programs). Various CodeGen models come with 350 million to 16.1 billion parameters and a maximum input length of 2,048 tokens (Nijkamp et al. 2022). CodeGen open-source models have been employed for code generation tasks in many recent studies (Chen et al. 2022; Zan et al. 2022).

2.2 Open Coding

Open coding, also known as postformed code, is a qualitative research method used to analyze textual data (e.g., survey responses, and interview transcripts). In open coding, researchers start with a relatively unstructured approach, inspecting the data line by line and generating

[1] https://copilot.github.com/

[2] https://docs.github.com/en/copilot/github-copilot-chat

2 背景

在本节中，我们介绍了本研究中使用的三种大型语言模型，并描述了本文遵循的开放编码方法。

2.1 用于代码生成的大型语言模型

不同的LLM已被引入到软件工程（SE）中的自动代码生成。一个非常强大的LLM是OpenAI的Codex（Chen等, 2021），它已被广泛应用于各种与代码相关的SE任务（Nguyen和Nadi, 2022；Lemieux等, 2023；Chen等, 2023）。这个基于GPT的闭源自回归LLM（Brown等, 2020）参数数量从1200万到120亿不等，并且在来自GitHub的5400万个公共代码库上进行了微调。值得注意的是，Codex是GitHub Copilot的背后模型，[1] 一个集成在IDE中的开发者编码助手，能够基于用户提供的上下文生成代码，最大输入长度为4096个标记。GitHub Copilot的一个新功能是“Copilot Chat”，[2] 该功能通过人类反馈进行了对话使用场景的调优。Copilot Chat在广泛的代码相关任务中表现出多功能性。它擅长生成代码片段，用自然语言描述代码片段，生成单元测试，以及修复有错误的代码。

PanGu-Coder（Christopoulou 等，2022）是一个开源的预训练大语言模型（LLM），用于从文本生成代码。该模型基于 PanGu-α 架构（Zeng 等，2021），是一种仅有解码器的单向 Transformer，并在其顶层增加了一个用于查询的额外层。PanGu-Coder 使用从 GitHub 提取的 Python 程序进行了两阶段训练。在第一阶段（无监督阶段），模型在包含 docstring/内联注释的原始编程语言数据上进行预训练，这些注释是用自然语言书写的。接下来，在第二阶段（有监督阶段），该模型经过微调以从文本生成代码。最近发布并用于代码生成的该模型有两个版本（Zan 等，2022）。这两个版本分别具有 3.17 亿和 26 亿参数，最大输入长度为 1,024 个标记。

CodeGen（Nijkamp 等，2022）是一类用于文本到代码生成的大型语言模型（LLM）。基于仅解码器的架构，这类模型旨在进行多轮程序合成，其中用户通过逐步输入自然语言规格与 LLM 互动，以获取相应的程序。这种协作式交互允许用户与 LLM 一起通过多步骤迭代地构建和完善程序。CodeGen 模型是在自然语言语料库和从 GitHub 提取的代码（即多语言代码数据集以及 Python 程序数据集）上训练的。各种 CodeGen 模型的参数量从 3.5 亿到 161 亿不等，最大输入长度为 2048 个标记（Nijkamp 等，2022）。CodeGen 开源模型已被用于许多近期研究中的代码生成任务（Chen 等，2022；Zan 等，2022）。

2.2 开放编码

开放编码，也称为事后形成的编码，是一种用于分析文本数据（例如调查回应和访谈记录）的定性研究方法。在开放编码中，研究人员从相对无结构的方法开始，逐行检查数据并生成

[1] https://copilot.github.com/[2] https://docs.github.com/en/copilot/github-copilot-chat

codes or labels for themes and concepts identified in the data. The generated codes are subject to dynamic adjustments, i.e., they can be added, removed, or merged to develop a final codebook. This flexibility allows for a more adaptive and nuanced analysis compared to the rigid structure of preformed codes (Seaman 1999). Open coding has proven to be more useful than the preformed code approach in SE studies (Humbatova et al. 2020; Liu et al. 2023).

In open coding, two human reviewers independently start the process of coding on the same set of samples. During this phase, the reviewers explore patterns, similarities, and categories within the samples relevant to the qualitative variables (Seaman 1999), i.e., characteristics or root causes of bugs in our study. They assign discrete categories to each sample. The two reviewers then engage in a discussion session to exchange and deliberate on their identified categories, forming the initial codebook.

In each discussion, a third reviewer, not involved in the coding process, intervenes in cases of diverging opinions between the two reviewers, to finalize the labels. The two reviewers continue to independently label the remaining samples and address similarly any conflicts that arise during subsequent discussion sessions (Liu et al. 2023). Through this iterative process, the two reviewers independently label all the samples. At the end of the labeling process, they report the agreement reached through these independent labeling processes before entering into any negotiations (Liu et al. 2023). Different studies in SE adapted this technique to conduct qualitative research for different purposes including constructing a taxonomy (Humbatova et al. 2020; Liu et al. 2023).

3 Study Design

In this section, we outline our methodology designed for investigating bugs in code generated by LLMs. We first describe our method for studying bug patterns in code generated by LLMs. Next, we present the methodology followed to validate the results of our empirical analysis. Specifically, we address the following research questions:

RQ1: What are the characteristics of bugs occurring in code generated by LLMs for real-world project tasks?
RQ2: To what extent are the identified bug patterns in LLM-generated code relevant for software practitioners and researchers working with LLMs?

Figure 1 depicts the study methodology. We started by collecting data from a benchmark dataset (CoderEval) and selected a sample set. Next, we established a preliminary codebook by labeling 10% of our sample set (pilot study). Subsequently, we iteratively completed the labeling process and our codebook. We finally organized our findings to obtain a taxonomy and surveyed practitioners to validate the bug patterns in our taxonomy. In the remainder of this section, we delve into further details of our methodology to answer each RQ.

3.1 RQ1: What are the characteristics of bugs occurring in code generated by LLMs for real-world project tasks?

3.1.1 Data Collection

We used a benchmark dataset named CoderEval (Yu et al. 2023) to conduct our study. This dataset consists of 230 Python functions from 43 open-source projects and 230 Java

用于标识数据中主题和概念的代码或标签。生成的代码可以根据需要进行动态调整，即可以添加、删除或合并，以开发最终的代码表。这种灵活性相比预先设定代码的僵化结构，允许进行更具适应性和细微差别的分析（Seaman 1999）。在软件工程研究中，开放编码已被证明比预设代码方法更有用（Humbatova 等，2020；Liu 等，2023）。

在开放编码中，两位人工审核者独立地对相同的一组样本开始编码过程。在此阶段，审核者探索样本中与定性变量相关的模式、相似性和类别（Seaman 1999），即我们研究中缺陷的特征或根本原因。他们为每个样本分配离散类别。随后，两位审核者进行讨论交流，审议他们识别出的类别，形成初始代码本。

在每次讨论中，如果两位审核者之间出现分歧，一名未参与编码过程的第三方审核者会介入，以确定最终标签。两位审核者继续独立地标注其余样本，并在随后的讨论环节中以类似方式解决出现的任何冲突（Liu 等，2023）。通过这一迭代过程，两位审核者独立完成所有样本的标注。在标注过程结束时，他们会在进入任何协商之前报告通过这些独立标注过程达成的一致意见（Liu 等，2023）。软件工程（SE）中的不同研究采用该技术进行定性研究，以实现不同目的，包括构建分类法（Humbatova 等，2020；Liu 等，2023）。

3 研究设计

在本节中，我们概述了用于研究由大型语言模型生成的代码中的错误的方法。我们首先描述了研究大型语言模型生成代码中错误模式的方法。接下来，我们介绍了用于验证实证分析结果的方法。具体来说，我们将讨论以下研究问题：

RQ1: 在由大型语言模型（LLMs）为真实项目任务生成的代码中，出现的错误有哪些特征？ RQ2: 在LLM生成的代码中识别出的错误模式在从事LLM工作的软件开发者和研究人员中有多大相关性?

图1展示了研究方法。我们首先从一个基准数据集（CoderEval）收集数据，并选择了一个样本集。接下来，我们通过标注样本集的10%建立了初步编码本（试点研究）。随后，我们迭代完成了标注过程和编码本的完善。最后，我们整理研究结果以获得一个分类法，并通过调研实践者验证分类法中的错误模式。在本节余下部分，我们将深入探讨方法论的细节，以回答每一个研究问题（RQ）。

3.1 研究问题1：在大型语言模型为真实项目任务生成的代码中出现的漏洞有哪些特征?

3.1.1 数据收集

我们使用了一个名为 CoderEval（Yu 等，2023）的基准数据集来进行我们的研究。该数据集包括来自 43 个开源项目的 230 个 Python 函数和 230 个 Java 函数

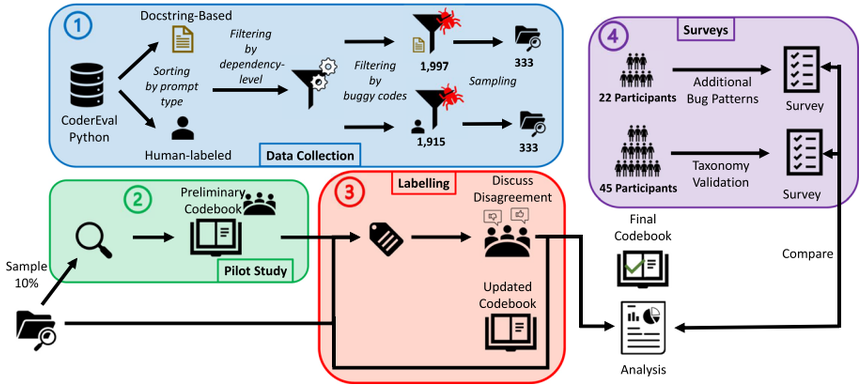


Fig. 1 High-level view of the followed methodology

functions from 10 open-source projects selected from GitHub. In this study, we focused on Python functions to reduce the workload induced by the manual labelling of bugs. We chose Python for several reasons: First, Python is a very common and widely used programming language across different domains.[3] Secondly, within the benchmark-based evaluation of LLMs, Python-based benchmarks are more represented than other programming languages (Jiang et al. 2024). Finally, this allowed us to reduce the workload necessary to build the taxonomy by focusing on one programming language. Each function in CoderEval is constituted of the function signature, a docstring describing the function, and the oracle code (i.e., correct reference solution to the function). Additional information is available in this benchmark such as the context of the function (like other functions being used, and files/projects in which the function is contained). CoderEval also contains a human-labeled description of each function which was obtained by showing the oracle code to a group of developers. Finally, for each of the functions, the CoderEval dataset contains 10 samples of code generated by each of the three LLMs under test for both types of prompt (docstring and human-labeled description). In our study, we will use both types of prompts to study how they influence the bug patterns.

The most recent version of CoderEval (Yu et al. 2024) uses ChatGPT (Schulman et al. 2022), CodeGen (Nijkamp et al. 2022) and PanGu-Coder (Christopoulou et al. 2022). However, when this study was started, CoderEval (Yu et al. 2023) included Codex (Chen et al. 2021) instead of ChatGPT (the other two LLMs were the same). As such, we used this version of their dataset in our analysis.[4] Two of these LLMs, CodeGen and PanGu-Coder, are open-source and have been used in previous studies that harness the power of LLMs for code generation tasks (Chen et al. 2022; Zan et al. 2022). The third one, Codex (Chen et al. 2021), is the model behind Copilot.[5] The code snippets generated by the LLMs were used to replace the oracle function sequentially. Upon replacement, the entry point code was executed, and the running output was captured, including return values, exceptions, errors, and console output. In case of no errors or exceptions, CoderEval compared the actual output with the expected output using unit tests. A difference between actual and expected output

[3] https://spectrum.ieee.org/top-programming-language-2020

[4] The dataset version is https://github.com/CoderEval/CoderEval/tree/ec1177750cf10b5faa414a0e76d1430e75141a44 and the paper version is https://arxiv.org/pdf/2302.00288v1

[5] https://github.com/features/copilot

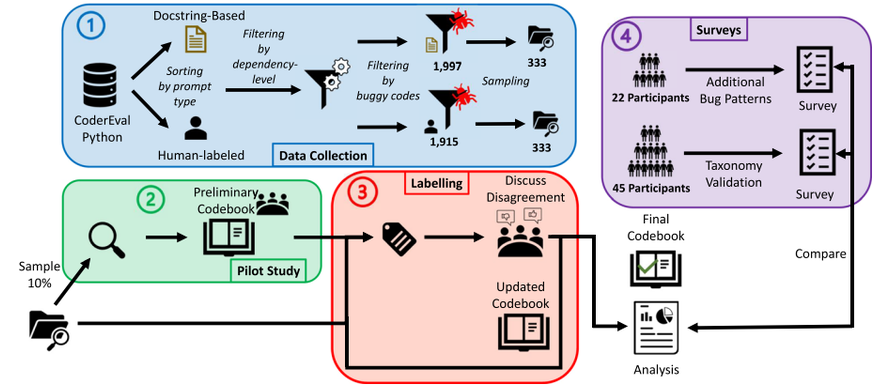


图1 所采用方法的高级视图

来自 GitHub 上 10 个开源项目的函数。在本研究中，我们专注于 Python 函数，以减少手动标注错误所带来的工作量。我们选择 Python 有几个原因：首先，Python 是一种非常常见且广泛使用的编程语言，应用于不同领域。其次，在基于基准的 LLM 评估中，基于 Python 的基准比其他编程语言更为常见（Jiang 等，2024）。最后，这使我们能够通过专注于一种编程语言来减少构建分类体系所需的工作量。CoderEval 中的每个函数由函数签名、描述函数的文档字符串以及 Oracle 代码（即函数的正确参考解法）组成。该基准中还提供了额外信息，例如函数的上下文（如使用的其他函数，以及函数所在的文件/项目）。CoderEval 还包含每个函数的人类标注描述。

最新版本的 CoderEval（Yu 等，2024）使用了 ChatGPT（Schulman 等，2022）、CodeGen（Nijkamp 等，2022）和盘古Coder（Christopoulou 等，2022）。然而，当本研究开始时，CoderEval（Yu 等，2023）包含的是 Codex（Chen 等，2021），而不是 ChatGPT（其他两个大型语言模型相同）。因此，我们在分析中使用了他们数据集的这个版本。[4] 这三个大型语言模型中，有两个，CodeGen 和盘古 Coder，是开源的，并且已在之前利用大型语言模型进行代码生成任务的研究中使用过（Chen 等，2022；Zan 等，2022）。第三个，Codex（Chen 等，2021），是 Copilot 背后的模型。[5] 由大型语言模型生成的代码片段被用来依次替换 oracle 函数。在替换后，执行入口点代码，并捕获运行输出，包括返回值、异常、错误和控制台输出。如果没有错误或异常，CoderEval 会将实际输出与预期输出进行比较。

[3] https://spectrum.ieee.org/top-programming-language-2020[4] 数据集版本是 https://github.com/CoderEval/CoderEval/tree/ec1177750cf10b5faa414a0e76d1430e75141a44, 论文版本是 https://arxiv.org/pdf/2302.00288v1[5] https://github.com/features/copilot

was considered a test failure. Conversely, an unexpected termination during execution was also treated as a test failure. The code snippets that led to a test failure were flagged as buggy in CoderEval. We used those flagged buggy samples in our empirical study.

The samples in CoderEval exhibit different "runnable levels", indicating the extent of dependency a function requires for effective execution. Out of the 6,900 Python samples in CoderEval (2,300 per LLM), our study was restricted to code fragments falling into one of the following runnable levels: "*self_runnable*" (without external dependencies), "*slib_runnable*" (dependent only on Python's standard libraries such as "os" or "json"), or "*plib_runnable*" (dependent only on Python's standard and public libraries such as "numpy"). We made this decision both to reduce the number of potential samples as well as because higher "runnable levels" would be more likely to result in the generation of code that fails to address the intended task, as highlighted in CoderEval's paper (Yu et al. 2023). Indeed, the higher this level, the more information is needed to properly understand how the function should work. This information is less likely to be correctly encompassed in the docstring for higher "runnable levels", to the point that even a programmer not privy to the actual function would struggle to code the functionality. As such, any bugs in the code generated from the prompt of those categories would not necessarily reflect a weakness of LLM in generating code but just a lack of proper information. Therefore, we did not consider them to mitigate potential bias in our results. Finally, as our focus was on observing bugs generated by LLMs, we considered only those generated code fragments that were flagged as buggy by CoderEval. These filterings resulted in 1,997 buggy code samples when using the docstring-based prompts and 1,915 buggy code samples when using the human-labeled prompts.

To streamline the manual labeling effort, we opted to sample our data. At a 95% confidence interval and an error rate of 5%, this necessitated the analysis of 323 samples when considering 1,997 buggy code samples. We thus sampled **333** code fragments for both types of prompts, that is **666** in total. To make the comparison relevant between the LLMs, in each case we sampled following a stratified sampling methodology: for a fixed type of prompt, for each LLM, we sampled 111 buggy generated code from each LLM available buggy code. Doing so allows us to have a representative sample of the distribution of bugs for each LLM (highlighted as ① in Fig. 1).

3.1.2 Manual Labeling and Classification

As no previous categorization exists for bugs in LLM-generated code based on real programming tasks, we conducted a manual analysis of our buggy samples employing an open coding procedure (Seaman 1999), similar to methodologies applied in prior studies on bug categorization (Humbatova et al. 2020; Liu et al. 2023; Tambon et al. 2024). The aim was to inductively construct bug categories in a bottom-up manner through manual analysis of buggy samples. This procedure allows us to identify various types of bugs based on our observations from the buggy samples. We used a shared document including the link to all buggy samples making sure that all reviewers could work together during the manual labeling process. The manual construction of the taxonomy involved three human reviewers, two of whom are senior Ph.D. students with experience in SE research. The third reviewer is a senior researcher with 10 years of research experience in the field of SE and AI.

We developed the codebook of our bug patterns in multiple rounds. We first started to label the buggy samples obtained from the docstring-based prompt and a preliminary pilot study was conducted. The first two authors independently coded 10% of the sample set to reduce the subjective bias. Each coder was tasked to analyze each buggy sample and label it with

被认为是测试失败。相反，在执行过程中出现的意外终止也被视为测试失败。导致测试失败的代码片段在CoderEval中被标记为有错误。我们在实证研究中使用了这些被标记为有错误的样本。

CoderEval 中的样本展示了不同的“可运行级别”，这表明一个函数在有效执行时所需要的依赖程度。在 CoderEval 中的 6,900 个 Python 样本（每个大型语言模型 LLM 为 2,300 个）中，我们的研究仅限于属于以下可运行级别之一的代码片段：“self_runnable”（无需外部依赖）、“slib_runnable”（仅依赖 Python 标准库，如“os”或“json”），或“plib_runnable”（仅依赖 Python 标准库和公共库，如“numpy”）。我们之所以做出这一决定，一是为了减少潜在样本的数量，二是因为较高的“可运行级别”更可能导致生成的代码无法完成预期任务，正如 CoderEval 的论文（Yu 等，2023）所强调的。实际上，级别越高，需要的信息就越多，以正确理解函数的工作方式。而这些信息在 docstring 中正确包含的可能性较低。

为了简化手动标注工作，我们选择对数据进行抽样。在95%的置信区间和5%的误差率下，考虑到1,997个有缺陷的代码样本，这需要分析323个样本。因此，我们为两种类型的提示各抽样了333个代码片段，总计666个。为了使不同大型语言模型（LLM）之间的比较具有参考意义，在每种情况下，我们都采用分层抽样的方法：对于固定类型的提示，对于每个LLM，我们从每个LLM可用的有缺陷代码中抽取111个生成的有缺陷代码。这样可以让我们拥有每个LLM的缺陷分布的代表性样本（如图1中所示的①）。

3.1.2 手动标注与分类

由于目前尚不存在针对基于真实编程任务的LLM生成代码中的错误的既有分类，我们对我们的错误样本进行了手动分析，采用了开放编码程序（Seaman 1999），类似于以往关于错误分类的研究方法（Humbatova 等 2020；Liu 等 2023；Tambon 等 2024）。其目的是通过对错误样本的手动分析，自下而上地归纳构建错误类别。此程序使我们能够根据对错误样本的观察识别各种类型的错误。我们使用了一个共享文档，其中包含所有错误样本的链接，确保所有审阅者在手动标注过程中能够协同工作。类别体系的手动构建涉及三位人工审阅者，其中两位是具有软件工程研究经验的高级博士生，第三位是具有10年软件工程与人工智能领域研究经验的高级研究员。

我们多轮次地开发了我们的缺陷模式代码本。我们首先开始标注从基于文档字符串的提示中获得的有缺陷样本，并进行了初步的试点研究。前两位作者独立对样本集的10%进行编码，以减少主观偏差。每个编码者的任务是分析每个有缺陷的样本并为其标注

one or more descriptive labels defined on their own, based on the bug pattern(s) observed in the sample. At the end of this round, the entire team convened to discuss the labels created, resolve conflicts, and merge the individual labels to create a consolidated set of categories, which served as the initial codebook for the next phase. In case of disagreement between the two coders, the third author would act as a tie-breaker (highlighted as ② in Fig. 1).

In the second phase, the remaining buggy samples were divided into six parts, each containing 15% of the remaining buggy codes, and we repeated the previous process: the two coders manually investigated all samples in each part independently, and then the entire team met to cross-check the labels and address conflicts while resolving discrepancies. If a new category emerged (i.e., a bug could not be classified under the existing categories) or a previous label was refined, the entire team gathered to discuss and incorporate the change into the codebook. We also made sure to re-label the affected samples from the previous rounds, if any. If a sample contained multiple types of bugs, it was classified into all identified bug patterns (i.e., using multiple labels). We had no constraint on the number of labels that could be assigned to a particular buggy snippet. Finally, the process was repeated, but this time with the buggy samples obtained using the human-labeled prompt but without an initial pilot study. The entire process required approximately 108 person-hours and resulted in a taxonomy comprising 10 bug patterns (highlighted as ③ in Fig. 1).

Given that we did not have any prior-defined labels and since we do not have a predefined number of labels for every buggy code snippet in our dataset, it is not possible to compute inter-rater agreement levels; which is typical for open coding. However, after finalizing categories, we realized that **77.5%** (for the docstring-based prompts) and **79.3%** (for the human-labeled prompt) of the bugs were labeled similarly during independent labeling by the two reviewers before solving disagreements. Disagreements generally arose when one of the coders either missed one particular bug pattern (e.g. A function name that was clearly biased by the prompt given to the LLM) that the other coder detected or when there were some blurry lines for a particular pattern (e.g. at what point is the LLM misinterpreting the task and not just missing some corner cases?). As an example, we give Listing 2 which presents a buggy code with the labels Syntax Error (missing parentheses) and a Non-Prompted consideration (the sign variable which is not asked in the conversion), two categories of bug pattern we uncovered in our labeling (see Sect. 4.1 for a description of the pattern). While both coders agreed on the Syntax Error, the coders disagreed on the Non-Prompted consideration, mainly on whether or not the handling of the sign variable was an attempt by the LLM to solve the task or something that was added by the LLM out of nowhere. After discussing and checking other buggy code fragments for the same task, the coders concluded that the sign variable was indeed not related to the task, solving the disagreement. Any conflicts were then thoroughly discussed until a 100% agreement was reached (Liu et al. 2023). We retained all the initial labels and comments on our shared document to facilitate any subsequent discussions. We provide in Table 1 each step of our process. These labels and comments are shared in our replication package available at The replication package (2023).

3.2 RQ2: To What Extent are the Identified Bug Patterns in LLM-generated Code Relevant for Software Practitioners and Researchers Working with LLMs?

To assess the relevance of the bug patterns identified in **RQ1**, we surveyed software practitioners and researchers working with LLMs (④ in Fig. 1).

一个或多个描述性标签根据样本中观察到的错误模式单独定义。在本轮结束时，整个团队会聚集讨论所创建的标签，解决冲突，并合并各自的标签以创建一个综合的类别集，这些类别集作为下一阶段的初始编码手册。如果两个编码者之间存在分歧，第三位作者将作为仲裁者（在图1中标注为②）。

在第二阶段，剩余的有缺陷样本被分为六部分，每部分包含剩余有缺陷代码的15%，我们重复了之前的流程：两名编码人员独立手动调查每一部分的所有样本，然后整个团队会面，以交叉检查标签并在解决分歧的同时处理冲突。如果出现了新类别（即某个缺陷无法归类到现有类别）或之前的标签被细化，整个团队会聚集讨论并将变更纳入编码手册。我们还确保重新标注之前轮次受影响的样本（如果有）。如果一个样本包含多种类型的缺陷，它会被分类到所有已识别的缺陷模式（即使用多个标签）。我们对分配给特定有缺陷代码片段的标签数量没有限制。最后，该流程被重复，但这次使用的是人类标注提示获得的有缺陷样本，但没有一个

鉴于我们没有任何事先定义的标签，并且由于我们没有为数据集中每个有缺陷的代码片段预定义的标签数量，因此无法计算评审者之间的一致性水平；这对于开放式编码来说是典型情况。然而，在最终确定类别后，我们发现，在解决分歧之前，两位评审员在独立标注时，有77.5%（针对基于文档字符串的提示）和79.3%（针对人工标注的提示）的错误被标注为相似。当出现分歧时，通常是因为其中一名编码员漏掉了另一名编码员检测到的某个特定错误模式（例如，一个函数名明显受到提供给大语言模型的提示的影响），或者某个特定模式存在一些模糊界限（例如，大语言模型在哪一点上是误解了任务，而不仅仅是遗漏了一些边缘情况？）。作为例子，我们给出清单2，其中展示了带有“语法错误（缺少括号）”和“非提示考虑”的标签的有缺陷代码。

3.2 研究问题2：在多大程度上识别出的 LLM 生成代码中的错误模式与从事 LLM 相关工作的软件从业者和研究人员相关?

为了评估在 RQ1 中识别的缺陷模式的相关性，我们调查了使用大语言模型（LLM）的软件从业者和研究人员（如图 1 中的④ 所示）。

Table 1 Coding process: initial agreements and conflicts throughout the rounds

Round	Analyzed Samples	Initial Agreements	Conflicts	New Categories
Pilot - Raw	33	26	7	8
1 - Raw	50	37	13	1 (Wrong Attribute)
2 - Raw	50	42	8	1 (NPC)
3 - Raw	50	34	16	0
4 - Raw	50	37	13	0
5 - Raw	50	43	7	0
6 - Raw	50	39	11	0
Total	333	258	75	10
1 - Human	50	35	15	0
2 - Human	50	38	12	0
3 - Human	50	37	13	0
4 - Human	50	43	7	0
5 - Human	50	37	13	0
5 - Human	50	41	9	0
6 - Human	33	29	4	0
Total	333	260	73	0

Raw: Docstring-based prompts, Human: Human-labeled prompts

3.2.1 Participants' Selection

To recruit participants for our surveys, we used several methods. The first one consisted in collecting the contacts (email addresses) of GitHub users who collaborated on repositories that contain code generated using various LLMs. We followed the methodology proposed by Fu et al. (2023) to identify GitHub repositories containing code generated using LLMs. Fu et al. (2023) employed various search keywords to collect GitHub repositories containing code generated by GitHub Copilot. We leveraged their shared dataset and broadened our list of participants, by adopting their methodology to collect additional repositories containing code-generated using Codex, Llama, Llama-2, CodeLlama, and ChatGPT. This process allowed us to collect a total of 113 repositories. Next, we employed the PyDriller (Spadini et al. 2018) library to extract the email addresses of the collaborators on each repository. In total, we obtained 200 unique email addresses of practitioners who collaborated on GitHub repositories containing LLM-generated code.

The second one was by adding a list of practitioners with a list of researchers who have published on code generation using LLMs. To obtain this list of researchers, we searched relevant LLM-based code generation papers over Google Scholar[6] using two keywords: "LLMs" and "code generation". We collected the first 75 papers returned by Google Scholar and manually inspected each of them to ensure their relevance i.e., that they leveraged LLMs for automatic code generation. This inspection process allowed us to identify 56 relevant papers that focused on code generation using LLMs. Next, we extracted the email addresses of the authors of each of these papers (who are assumed to be researchers working on generating code with LLMs). We obtained a total of 182 emails after removing duplicates.

[6] https://scholar.google.com

表1 编码过程：各轮次中的初步共识与冲突

圆	分析样本	初步协议	冲突	新类别
飞行员 - 原版	33	26	7	8
1 - 生	50	37	13	1（错误属性）
2 - 原始	50	42	8	1（NPC）
3 - 原始	50	34	16	0
4 - 生	50	37	13	0
5 - 生	50	43	7	0
6 - 生	50	39	11	0
总计	333	258	75	10
1 - 人类	50	35	15	0
2 - 人类	50	38	12	0
3 - 人类	50	37	13	0
4 - 人类	50	43	7	0
5 - 人类	50	37	13	0
5 - 人类	50	41	9	0
6 - 人类	33	29	4	0
总计	333	260	73	0

原始：基于文档字符串的提示，人类：人类标注的提示

3.2.1 参与者的选择

为了招募我们的调查参与者，我们使用了几种方法。第一种方法是收集在包含使用各种大语言模型（LLM）生成的代码的仓库中协作的 GitHub 用户的联系方式（电子邮件地址）。我们遵循 Fu 等人（2023）提出的方法来识别包含 LLM 生成代码的 GitHub 仓库。Fu 等人（2023）使用各种搜索关键词来收集包含 GitHub Copilot 生成代码的 GitHub 仓库。我们利用他们共享的数据集，并通过采用他们的方法来收集包含使用 Codex、Llama、Llama-2、CodeLlama 和 ChatGPT 生成代码的额外仓库，从而扩大了参与者名单。这个过程使我们总共收集了 113 个仓库。接下来，我们使用 PyDriller（Spadini 等人 2018）库提取每个仓库协作者的电子邮件地址。总计，我们获得了 200 个在 GitHub 仓库中协作的从业者的唯一电子邮件地址。

第二种方法是通过列出从事 LLM 代码生成研究的研究人员名单来实现。为了获得这份研究人员名单，我们在 Google Scholar 上搜索了与 LLM 代码生成相关的论文，使用了两个关键词："LLMs"和"代码生成"。我们收集了 Google Scholar 返回的前 75 篇论文，并手动检查每一篇论文以确保其相关性，即它们确实利用 LLM 进行自动代码生成。这个检查过程使我们识别出 56 篇专注于使用 LLM 生成代码的相关论文。接下来，我们提取了这些论文作者的电子邮件地址（假设他们是从事 LLM 代码生成研究的研究人员）。在去除重复后，我们总共获得了 182 个电子邮件地址。

[6] https://scholar.google.com

Overall, we successfully sent the survey questionnaire to 382 unique email addresses. We discuss the details about the participants and the response rate in Sect. 4.2.

Listing 2 Code generated by CodeGen and labeled as "Syntax Error".

```
def int_to_string(number:
        """
        Convert a number to a string, using the given alphabet.

The output has the most significant digit first.
        """
        string = ''
        if number < 0:
                number = −number
                sign = '−'
        else:
                sign = '+'
        while number!= 0:
                digit = number % 10
                number = number // 10
                string += str(digit)
        string = string[::−1]
        return sign + string
```

The third one was by posting our survey questionnaire on two relevant Reddit channels: *LocalLLaMA* and *MachineLearning*. Figure 7 in Appendix A shows our post on Reddit. The last method used was by collecting answers using paid crowd-workers platforms such as Prolific[7] as used in previous studies (Zid et al. 2024). We screened participants by requiring at least one year of self-reported Python programming experience, skills in development and testing, and experience with using LLM such as ChatGPT. We manually checked each response obtained to verify the quality of the response and avoid a participant replying at random when possible. While it is not possible to be sure that such a case arose, we did not observe patterns pointing towards a participant randomly filling the questionnaire (i.e. most of the time answering the same answer etc.) and there was no high deviation in the response of a particular respondent compared to the rest.

The questionnaires were dispatched along with a detailed message outlining the purpose, scope, and estimated completion time of the survey. We made it clear to the participants that the survey was anonymous, though participants had the option to provide their emails for additional communication and to receive a summary of the study.

3.2.2 Surveys with Practionners

Both survey forms and the anonymized responses that we collected are available in our replication package (The replication package 2023). Both surveys are divided into two parts and are inspired by similar empirical studies of bug patterns (Nikanjam et al. 2021; Humbatova et al. 2020; Tambon et al. 2024). In the first part, we asked demographic questions to the participants, as well as high-level questions about bugs in LLMs generated code regarding their frequencies and complexities associated with detecting or fixing the bugs (see Fig. 8 in Appendix B for more details).

In the second part, we ask specific questions for each survey. In the first survey, we wanted to gather feedback from practitioners about bug patterns they encountered in LLM-generated code without mentioning our taxonomy. To do so, we ask practitioners to provide at least

[7] https://www.prolific.com/

总体而言，我们成功地将调查问卷发送到了382个唯一的电子邮件地址。关于参与者和回应率的详细信息，我们在第4.2节中进行了讨论。

清单 2 由 CodeGen 生成的代码，并标记为“语法错误”。

```
def int_to_string(数字:
        """
        使用给定的字母表将数字转换为字符串。

输出的最高有效位在最前面。
        """
        字符串 =' '
        如果 number < 0:
                数字 = −数字
                签署 =' −'
        否则:
                签署 =' +'
        当 number!= 0:
                数字 = 数 % 10
                数字 = 数字 // 10
                字符串 +=str(数字)
        字符串 =字符串[::−1]
        返回 sign + 字符串
```

第三种方法是将我们的调查问卷发布在两个相关的 Reddit 频道上：LocalLLaMA 和 MachineLearning。附录 A 中的图 7 显示了我们在 Reddit 上的帖子。最后一种方法是使用付费众包平台（如 Prolific）收集答案，这与以往研究中使用的方法相同（Zid 等，2024）。我们通过要求参与者至少有一年的自报 Python 编程经验、开发和测试技能，以及使用大语言模型（如 ChatGPT）的经验来筛选参与者。我们手动检查每个获得的回答，以验证回答的质量，并尽可能避免参与者随机作答。虽然无法完全确定是否出现这种情况，但我们没有观察到参与者随意填写问卷的模式（例如大部分时间都回答相同的答案等），并且没有发现某个受访者的回答与其他受访者存在较大偏差。

问卷连同一条详细信息一起发送，说明了调查的目的、范围和预计完成时间。我们向参与者明确表示调查是匿名的，不过参与者可以选择提供他们的电子邮件以进行额外沟通并接收研究总结。

3.2.2 与从业者的调查

我们收集的调查问卷和匿名回复均可在我们的复现包中获取（复现包 2023）。两项调查均分为两部分，并受到类似错误模式实证研究的启发（Nikanjam 等, 2021; Humbatova 等, 2020；Tambon 等, 2024）。在第一部分，我们向参与者提出了人口统计学问题，以及有关 LLM 生成代码中错误的高级问题，涉及错误的频率和与检测或修复这些错误相关的复杂性（有关更多详细信息，请参见附录 B 中的图 8）。

在第二部分中，我们针对每个调查问卷提出具体问题。在第一次调查中，我们希望收集从业者关于他们在大型语言模型生成的代码中遇到的错误模式的反馈，而不提及我们的分类法。为此，我们要求从业者至少提供

[7] https://www.prolific.com/

two such bug patterns they encountered and to give a small description of what they were, as illustrated in Fig. 9. In the second survey, we wanted to gather the participants' feedback on several aspects of each pattern of bug identified. To do so, we defined the questions similarly for each pattern of bugs in the taxonomy. Specifically, for each pattern of bug, we present participants with a description of the bugs belonging to the pattern and provide an example code snippet; we use the same examples that will be presented in Sect. 4.1. Next, we presented the participants with several questions to be answered using a 5-level Likert scale. The full questionnaire is presented in Appendix B in Fig. 10.

The questions start by asking about the frequency at which the participants encountered the bug pattern (ranging from 1-Never to 5-Always). Next, we defined the following concepts to the participants:

i. Diagnosing: How hard would it be for the respondents to diagnose such an error in an LLM-generated code? Could they distinguish it easily or would it require running the code or using a debugger rendering the diagnostic hard? (1-Easy to 5-Hard)
ii. Complexity: Is the type of error rather trivial (i.e. not something a human developer with decent programming knowledge would do) or, on the contrary, does it denote some degree of complexity? (1-Trivial to 5-Complex)
iii. Fixing: If the respondents had to fix such a mistake, would it be hard? Would it just require adjusting some part of the code (i.e. easy), or would it require extensive refactoring (i.e. hard)? (1-Easy to 5-Hard)

Participants were then asked to answer from their own perception/experience of each bug pattern.

The survey also contained optional comment fields at the end of each section and the end of the survey. Participants had the opportunity to provide suggestions such as additional information related to the bug patterns in the study or bug patterns not described in the study.

3.2.3 Analysis of Survey Results

Once the results were collected for both surveys, we aggregated them. For the first survey, two authors manually went through the suggested bug patterns by each participant. We then tried to label the suggested patterns with one of ours or, if not possible, we flag the bug patterns as "Not present". This allows us to present an analysis and detail what bug patterns the respondents mentioned that we did not encounter and propose an explanation as to why.

For the second survey, for each bug pattern by the respondents, we aggregated the results (see Sect. 4.2). To do so, we followed a methodology used in similar studies (Tambon et al. 2024) leveraging weighted average. We processed the results for the frequency/diagnosing/-complexity/fixing of each bug pattern reported by the participants by multiplying the value of the Likert Scale with each percentage. That is to say, for instance, we would obtain a score of 2.79 for a bug pattern where respondents replied the following: 1 - (17.6%), 2 (26.5%), 3 (23.5%), 4 (23.5%), and 5 - (8.8%). We also complemented the explanation of the results with comments from respondents relevant to the description of the results. The goal was to capture the perceived difficulty in fixing/diagnosing and the complexity of the different bug patterns according to practitioners. Finally, we compared the weighted score obtained between two groups: academic respondents and industry respondents. The goal is to potentially highlight differences in the bug patterns faced by both groups.

Additionally, we performed an analysis comparing the frequency of reported bug patterns among respondents with those in our sample sets. We wanted to assess whether the bug patterns within our sample sets (extracted from CoderEval) matched LLM practitioners'

他们遇到的两种此类缺陷模式，并对它们进行了简要描述，如图 9 所示。在第二次调查中，我们希望收集参与者对每种识别出的缺陷模式的若干方面的反馈。为此，我们为分类法中的每种缺陷模式定义了类似的问题。具体来说，对于每种缺陷模式，我们向参与者展示属于该模式的缺陷描述，并提供示例代码片段；我们使用的示例与第 4.1 节中呈现的相同。接下来，我们向参与者提出了若干问题，要求使用五级李克特量表回答。完整问卷在附录 B 的图 10 中展示。

问题首先询问参与者遇到该漏洞模式的频率（从1-从不到5-总是）。接下来，我们向参与者定义了以下概念：

i. 诊断：对于受访者来说，诊断LLM生成的代码中的这种错误会有多难？他们能轻松区分吗，还是需要运行代码或使用调试器，从而使诊断变得困难？（1-容易到 5-困难） ii. 复杂性：这种错误类型是否相当简单（即拥有良好编程知识的人类开发者不会犯的错误），还是相反，它表示一定程度的复杂性？（1-简单 到 5-复杂） iii. 修复：如果受访者必须修复这种错误，会难吗？是否只需要调整代码的某个部分（即容易），还是需要进行大规模重构（即困难）？（1-容易 到 5-困难）

然后要求参与者根据自己对每种缺陷模式的感知/经验来回答。

调查在每个部分的末尾以及调查的末尾还包含可选的评论字段。参与者有机会提供建议，例如与研究中的错误模式相关的额外信息或研究中未描述的错误模式。

3.2.3 调查结果分析

一旦收集了两项调查的结果，我们就进行了汇总。对于第一项调查，两位作者手动审查了每位参与者提出的错误模式。然后我们尝试将提出的模式标注为我们已有的某个模式，或者如果无法匹配，我们将这些错误模式标记为“未出现”。这使我们能够呈现分析，并详细说明受访者提到但我们未遇到的错误模式，并提出可能的解释。

在第二次调查中，对于每个受访者提及的缺陷模式，我们汇总了结果（参见第4.2节）。为此，我们遵循了类似研究中使用的方法（Tambon 等，2024），利用加权平均值。我们通过将李克特量表的数值与各百分比相乘，处理参与者报告的每种缺陷模式的频率/诊断难度/复杂度/修复难度的结果。也就是说，例如，对于某个缺陷模式，如果受访者的回答如下：1 - (17.6%)，2 - (26.5%)，3 - (23.5%)，4 - (23.5%)，5 - (8.8%)，则我们将得到一个2.79的得分。我们还补充了参与者对结果描述的相关评论来解释结果。目的是根据从业者的观点，捕捉修复/诊断难度及不同缺陷模式的复杂性。最后，我们比较了两个群体——学术受访者和行业受访者——之间获得的加权得分。目的是增强

此外，我们还进行了分析，将受访者报告的错误模式频率与我们的样本集中的频率进行了比较。我们希望评估我们样本集（从CoderEval中提取）的错误模式是否与大型语言模型从业者的相匹配。

experience when dealing with bugs. The idea is to provide further evidence that our labeled categories are encountered in similar proportions by the respondents who used LLMs, not on a dataset such as CoderEval but as a tool in their development activities. Specifically, we evaluated if there was a correlation between the weighted average of the frequency of bug patterns reported by the participants and the proportion of each bug pattern found in the studied sample. To do so, we calculated the Spearman-ρ between those scores and the distribution of bug patterns.

4 Empirical Results

In this section, we delve into the bug patterns obtained from our analysis as well as the respondents' answers from our survey. We examine the nature, frequency, and distribution of these bugs across the three different LLMs. All collected artifacts and data generated during our study are accessible in our replication package (The replication package 2023).

4.1 RQ1: What are the Characteristics of Bugs Occurring in Code Generated by LLMs for Real-world Project Tasks?

4.1.1 Taxonomy

We organized our observations as a taxonomy. The taxonomy is organized into 10 identified categories (in color in Fig. 2) grouped into four main topics. We also provide three categories not found in our samples but reported by participants during our first survey (see Sect. 4.2.2).

In the following, we explain each of the 10 bug patterns observed; providing representative examples.

1. Misinterpretation Misinterpretation occurs when the generated code deviates from the intention of the prompt, that is the code *misinterprets* the functionality by being off the mark of the intended tasks. This happens when the model struggles to capture the specifications of the task expressed in the prompt. In Listing 3, we give an example task, "int_to_string", along

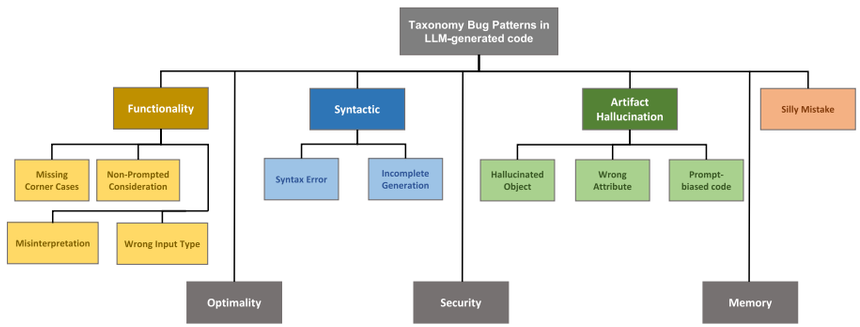


Fig. 2 Final taxonomy of bug patterns in code generated by LLM. We regroup our 10 categories into topic categories (e.g. Functionality) when possible. We also provide three topics (Optimality, Security and Memory) which were not identified in our samples but reported by participants during the first survey (see Sect. 4.2.2)

处理 bug 的经验。这个想法是提供进一步的证据，说明使用 LLM 的受访者遇到我们标记的类别的比例相似，而不是在像 CoderEval 这样的数据集上，而是在他们的开发活动中作为工具使用。具体来说，我们评估了参与者报告的 bug 模式频率加权平均值与研究样本中发现的每种 bug 模式比例之间是否存在关联。为此，我们计算了这些分数与 bug 模式分布之间的 Spearman 相关系数。

4 实证结果

在本节中，我们深入探讨了从分析中获得的错误模式以及受访者在调查中的回答。我们考察了这些错误在三种不同大型语言模型中的性质、频率和分布。本研究中收集的所有资料和生成的数据均可在我们的复现包中获取（复现包 2023）。

4.1 RQ1：在大型语言模型生成的用于实际项目任务的代码中，出现的错误有哪些特征?

4.1.1 分类法

我们将我们的观察整理为一个分类法。该分类法分为10个已识别的类别（在图2中以颜色表示），并归入四个主要主题。我们还提供了三类在我们的样本中未发现但在第一次调查中由参与者报告的类别（见第4.2.2节）。

在下文中，我们解释观察到的10种错误模式；并提供代表性示例。

1. 误解 误解发生在生成的代码偏离提示的意图时，也就是说代码因为偏离了预期任务而误解了功能。当模型难以捕捉提示中表达的任务规格时，就会发生这种情况。在清单3中，我们给出了一个示例任务，"int_to_string"，沿着

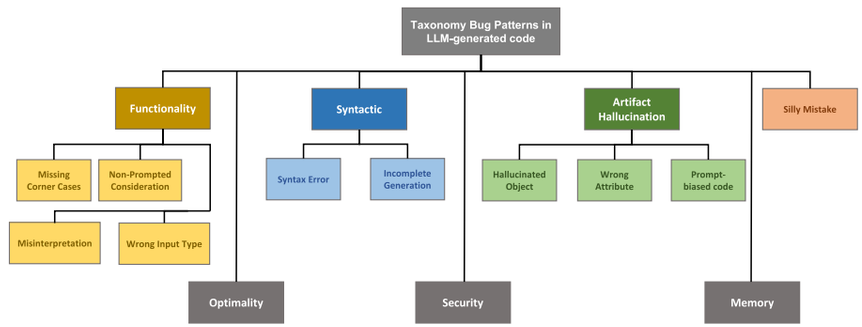


图2 由大型语言模型生成的代码中错误模式的最终分类。当可能时，我们将10个类别重新归入主题类别（例如功能性）。我们还提供了三个主题（优化、安全性和内存），这些主题在我们的样本中未被识别，但在第一次调查中由参与者报告（见第4.2.2节）。

with its corresponding reference solution (top). The function declaration and the docstring at the beginning of the solution (lines 1 to 6) represent the prompt and the rest (lines 7 to 15) is the reference solution. A code generated by PanGu-Coder for this prompt is presented at the bottom of Listing 3 (lines 17 and 18). This code is misaligned with the prompt since it simply treats the variable "number" as an index and returns the item at that index in the list "alphabet". However, the prompt describes a task to convert the variable "number" to a string using the given list "alphabet". Consequently, we label the contained bug as *Misinterpretation*. This bug pattern is among the most common in our analyzed dataset across all three models.

Listing 3 Reference Solution for the task "int_to_string', and code generated by PanGu-Coder labeled as "Misinterpretation".

```
1  def int_to_string(number: int, alphabet: List[str], padding: Optional[int] = None) −> str:
2      """
3      Convert a number to a string, using the given alphabet.
4
5      The output has the most significant digit first.
6      """
7      output = ‘‘’’
8      alpha_len = len(alphabet)
9      while number:
10         number, digit = divmod(number, alpha_len)
11         output += alphabet[digit]
12     if padding:
13         remainder = max(padding − len(output), 0)
14         output = output + alphabet[0] ∗ remainder
15     return output[::−1]
16 −−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−
17 def int_to_string(number: int, alphabet: str) −> str:
18     return alphabet[number]
```

2. Syntax Error Any code with a syntax error, such as a missing parenthesis or semicolon, is categorized as a *Syntax Error*. We presented earlier Listing 2 which showcases a code example with a syntax error in the first line generated by CodeGen for the same task, "int_to_string", as in Listing 3. In this case, the LLM failed to complete the function declaration (adding the closing parenthesis) at the end of the function signature which resulted in a syntax error. Although this bug pattern is not very common across the models in our study, CodeGen tended to generate more buggy code with specific syntax errors.

3. Silly Mistake We labeled codes presenting issues such as redundant conditions or unnecessary casting as *Silly Mistake*. While these types of issues may not introduce a bug on their own, they have the potential to easily lead to one. In Listing 4, there is an example generated by CodeGen where both the *if* and *else* conditions have the same statement. This is labeled as *Silly Mistake*. This bug pattern ranks among the top three most common patterns across the models.

Listing 4 Code generated by CodeGen and labeled as "Silly Mistake".

```
1  if not logfile.exists():
2          logging.basicConfig(
3                          filename=str(logf),
4                          format='%(asctime)s %(levelname)s %(name)s %(message)s',
5                          datefmt='%Y−%m−%d %H:%M:%S',
6                          level=logging.DEBUG,
7                  )
8  else:
```


以及其对应的参考解决方案（顶部）。解决方案开头的函数声明和文档字符串（第1到6行）表示提示，其余部分（第7到15行）是参考解决方案。PanGu-Coder 为此提示生成的代码展示在清单3底部（第17和18行）。由于该代码仅将变量“number”视为索引并返回列表“alphabet”中该索引处的项，它与提示不一致。然而，提示描述的任务是使用给定的列表“alphabet”将变量“number”转换为字符串。因此，我们将其中的错误标记为“误解”。这种错误模式在我们分析的三个模型的数据集中都是最常见的之一。

列出任务“int_到_字符串”的参考解决方案（Listing 3），以及由盘古-Coder生成并标记为“误解”的代码。

```
1  def int_到_string(number: int, alphabet: List[str], padding: Optional[int] = None) −>str:
2      """
3      使用给定的字母表将数字转换为字符串。
4
5      输出的最高有效位在最前面。
6      """
7      输出 = ‘ ’’ ’
8      alpha_长度 =长度(字母表)
9      当 number:
10         数字，位数 = divmod(数字, 字母_长度)
11         输出 += 字母[数字]
12     如果有填充:
13         余数 = max(填充 − 输出长度, 0)
14         输出 = 输出 + 字母表[0] ∗ 余数
15     返回输出[::−1]
16 −−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−−
17 def int_到_字符串(number: int, alphabet: str) −> str:
18     返回字母[数字]
```

2. 语法错误 任何包含语法错误的代码，如缺少括号或分号，都被归类为语法错误。我们之前展示了列表 2，它展示了一个在第一行就有语法错误的代码示例，这段代码由 CodeGen 为同一个任务“int_to_string”生成，如列表 3 所示。在这种情况下，LLM 未能完成函数声明（在函数签名末尾添加关闭的括号），导致了语法错误。尽管这种错误模式在我们研究的模型中并不常见，但 CodeGen 更倾向于生成带有特定语法错误的 buggy 代码。

3. 愚蠢错误 我们将那些呈现诸如冗余条件或不必要类型转换等问题的代码标记为愚蠢错误。虽然这些类型的问题本身可能不会引入错误，但它们很容易导致错误。在示例 4 中，有一个由 CodeGen 生成的例子，其中 if 和 else 条件下的语句相同。这被标记为愚蠢错误。这种错误模式在各模型中排名前三的常见模式之一。

列表 4 由 CodeGen 生成并标记为“愚蠢的错误”的代码。

```
1 如果 notlogfile 不存在(): 2 logging.basicConfig( 3 filename=str(logf), 4
format=’%(asctime)s %(levelname)s %(name)s %(message)s’, 5
datefmt=’%Y−%m−%d %H:%M:%S’, 6 level=logging.DEBUG, 7 ) 8 否则:
```

```
        logging.basicConfig(
                filename=str(logf),
                format='%(asctime)s %(levelname)s %(name)s %(message)s',
                datefmt='%Y-%m-%d %H:%M:%S',
                level=logging.DEBUG,
            )
```

4. Prompt-biased Code This pattern occurs when the generated code presents some correctness and/or generalization issues due to the LLM excessively relying on provided examples or particular terms in the prompt while implementing a function. Listing 5 illustrates this issue. Lines 1 to 9 constitute the prompt, while lines 10 to 13 show the code generated by Codex for the task described in the prompt. The code should return the polygon for given vertices. The length of the vertices can be varied. However, lines 5 to 8 contain an example of vertices with length 4 and the list of its expected outputs, given to clarify the task provided in the prompt. In the code in Listing 5 generated by Codex, the code is accurate only for the specific example provided in the prompt, when the length of the vertex is 4, and will fail in other cases. This code is labeled as a "Prompt-biased code".

Listing 5 Code generated by Codex and labeled as "Prompt-biased Code".

```
def vertex3tuple(vertices):
    """returns 3 points for each vertex of the polygon. This will include the vertex and
   the 2 points on both sides of the vertex:

        polygon with vertices ABCD
        Will return
        DAB, ABC, BCD, CDA -> returns 3tuples
        #A B C D -> of vertices
    """
    return [(vertices[-1], vertices[0], vertices[1]),
                (vertices[0], vertices[1], vertices[2]),
                (vertices[1], vertices[2], vertices[3]),
                (vertices[2], vertices[3], vertices[0])]
```

5. Missing Corner Case This bug occurs when the generated code operates correctly, except for overlooking certain corner cases. Listing 6 provides an example of this bug pattern along with its corresponding reference solution (top). Lines 17 to 24 represent the prompt and line 25 shows the code generated by Codex. The task involves implementing a function that checks if a given host is the localhost. Although the Codex-generated code checks for various representations of localhost, the reference solution is expected to verify additional potential options, such as *socket.gethostname()*, which were present in the oracle given for the task described in the prompt. Listing 6 is labeled as *Missing Corner Case*.

Listing 6 Reference Solution for the task "is_local" and code generated by Codex, labeled as "Missing Corner Case".

```
import socket
def is_local(host):
    """
    Checks if the host is the localhost

    :param host: The hostname or ip
    :return: True if the host is the localhost
    """
```



```
9 logging.basicConfig( 10 filename=str(logf), 11 format='%
(asctime)s %(levelname)s %(name)s %(message)s', 12 datefmt='%Y-%
m-%d%H:%M:%S', 13 level=logging.DEBUG, 14 )
```

4. 提示偏倚代码 这种模式发生在生成的代码由于大语言模型在实现功能时过度依赖提供的示例或提示中的特定术语，从而出现一些正确性和/或泛化问题。列表 5 展示了这个问题。第 1 到 9 行构成提示，而第 10 到 13 行显示 Codex 为提示中描述的任务生成的代码。该代码应返回给定顶点的多边形。顶点的长度可以变化。然而，第 5 到 8 行包含一个长度为 4 的顶点示例及其预期输出列表，以阐明提示中提供的任务。在 Codex 生成的列表 5 中的代码，对于提示中提供的特定示例（顶点长度为 4）是准确的，但在其他情况下会失败。该代码被标记为"提示偏倚代码"。

列出由 Codex 生成并标记为"提示偏向代码"的第 5 号代码。

```
def 顶点三元组(顶点):
    "为多边形的每个顶点返回3个点。这将包括该顶点和
   顶点两边的两个点:

        顶点为 ABCD 的多边形
        会回来
        DAB、ABC、BCD、CDA ->返回三元组
        #A B C D ->的顶点
    """
    返回 [(顶点[-1], 顶点[0], 顶点[1]),
                (顶点[0], 顶点[1], 顶点[2]),
                (顶点[1], 顶点[2], 顶点[3]),
                (顶点[2], 顶点[3], 顶点[0])]
```

5. 缺失的边缘情况 该错误发生在生成的代码正常运行，但忽略了某些边缘情况时。清单6提供了这种错误模式的示例以及相应的参考解决方案（上方）。第17行到第24行表示提示，第25行显示Codex生成的代码。该任务涉及实现一个函数，用于检查给定主机是否为本地主机。尽管Codex生成的代码检查了本地主机的各种表示方式，但参考解决方案应验证其他潜在选项，例如socket.gethostname()，这些选项在提示中描述的任务的oracle中存在。清单6被标注为缺失的边缘情况。

列表 6 "is_local" 任务的参考解决方案以及由 Codex 生成的代码，标记为"缺失的边缘情况"。

```
导入 socket
def is_local(主机):
    """
    检查主机是否为本地主机

    :param host: 主机名或IP
    :return: 如果主机是本地主机则返回 True
    """
```

```
    return host in ["127.0.0.1",
                    "localhost",
                    socket.gethostname(),
                    # just in case socket.gethostname() does not work we also try the following:
                    platform.node(),
                    socket.gethostbyaddr(socket.gethostname())[0]
                    ]
```

```
import socket
def is_local(host):
    """
    Checks if the host is the localhost

    :param host: The hostname or ip
    :return: True if the host is the localhost
    """
    return host in ['localhost', '127.0.0.1', '::1']
```

6. Wrong Input Type We use this label when the code contains an incorrect input type in a correct function call. For example in Listing 7, the task describes implementing a function to reduce a given list of classes to its ordered minimum equivalent. Lines 5 to 8 represent the code generated by CodeGen to address this task. In line 8, the Python built-in function *min* is called on a List of ClassType inputs which raises an error. We labeled this buggy code example as *Wrong Input Type*. It is worth noticing that this is not the only label for this buggy code example in Listing 7.

Listing 7 Code generated by CodeGen and labeled as "Wrong Input Type".

```
def minimalBases(classes):
        '''
        Reduce a list of base classes to its ordered minimum equivalent
        '''
        if len(classes) == 1:
                return classes[0]
        else:
                return min(classes)
```

7. Hallucinated Object We apply this label when the code contains an object that is not defined (i.e., the LLM is hallucinating). For instance, Listing 8 represents the code generated by Codex for the task "make_find_paths"that involves transforming all the paths given in the find-paths variable into glob patterns. The generated function calls a function named "find_path_to_glob" (line 14), which is not defined. *Hallucinated Object* is among the third common bug patterns across the models.

Listing 8 Code generated by Codex and labeled as "Hallucinated Object".

```
def make_find_paths(find_paths):
    '''
    Given a sequence of path fragments or patterns as passed to '--find', transform all
    path fragments into glob patterns. Pass through existing patterns untouched.

    For example, given find_paths of:

      ['foo.txt', 'pp:root/somedir']

```


9 返回主机是否在 [中"127.0.0.1", 10 "localhost", 11 socket.gethostname(), 12 # 以防 socket.gethostname() 不起作用，我们也尝试以下方法： 13 platform.node(), 14 socket.gethostbyaddr(socket.gethostname())[0]15]

16 _____

17 导入 socket 18 定义 is_local(host): 19 """ 20 检查主机是否为本地主机 21 22 :param host: 主机名或 IP 23 :return: 如果主机是本地主机则返回 True 24 """ 25 返回主机是否在 ['localhost', '127.0.0.1', '::1'] 中

6. 错误的输入类型 当代码在正确的函数调用中包含了错误的输入类型时，我们会使用此标签。例如在示例 7 中，任务描述的是实现一个函数，将给定的类列表简化为其有序的最小等价形式。第 5 到 8 行表示 CodeGen 为解决此任务而生成的代码。在第 8 行，Python 内置函数 min 被用于 ClassType 类型的列表，这会引发错误。我们将此有问题的代码示例标记为“错误的输入类型”。值得注意的是，这并不是示例 7 中此有问题代码示例的唯一标签。

代码清单 7 由 CodeGen 生成，并标记为“错误的输入类型”。

```
def 最小基类(类集合):
        '''
        将基类列表缩减为其有序的最小等效项
        '''
        如果 len(classes) == 1:
                返回类[0]
        否则:
                返回 min(classes)
```

7. 幻觉对象 当代码中包含未定义的对象（即，LLM 出现幻觉）时，我们会应用此标签。例如，列表 8 表示 Codex 为任务“make_find_paths”生成的代码，该任务涉及将 find-paths 变量中给定的所有路径转换为全局模式。生成的函数调用了一个名为“find_path_to_glob”的函数（第 14 行），但该函数未被定义。幻觉对象是各模型中第三常见的错误模式之一。

列表 8 由 Codex 生成并标记为“虚构对象”的代码。

```
def make_find_paths(find_paths):
    '''
    给定传递给 '--find' 的一系列路径片段或模式，转换所有
    将路径片段转换为通配符模式。现有模式保持不变。

    例如，给定 find_路径的:

      ['foo.txt', 'pp:root/somedir']

```

```
    ... transform that into:

      ['sh:**/*foo.txt*/**', 'pp:root/somedir']
    '''
    return [find_path_to_glob(x) for x in find_paths]
```

8. Wrong Attribute This label is applied when the code contains an incorrect/non-existent attribute for an object or module. For example, in Listing 9, there is code generated by PanGu-Coder for the same task presented in Listing 1 in the introduction, named "parser_flag". The prompt describes the task as "Given an argparse.ArgumentParser instance, returns its argument flags in a space-separated string." The code generated by PanGu-Coder tries to use 'flags' as an attribute of the input 'parser', which is not a correct attribute according to the information provided in the prompt. The code sample in Listing 9 is labeled as "Wrong Attribute".

Listing 9 Code generated by PanGu-Coder and labeled as "Wrong Attribute".

```
def parser_flags(parser):
    '''
    Given an argparse.ArgumentParser instance, return its argument flags in a
    space-separated string.
    '''
    return''.join('--{0}'.format(f) for f in parser._flags)
```

9. Incomplete Generation This label is applied when the generated code is incomplete or is reduced to, for example, a 'pass' statement or an empty function. This bug pattern also includes cases where the LLM stops completing the code in the middle of the function either because of token limits or because it reached an <end_of_sequence> token without properly finishing the code. Listing 10 generated by CodeGen is an example of this category.

Listing 10 Code generated by CodeGen and labeled as "Incomplete Generation".

```
def write_configuration(config_filename, rendered_config, mode=0o600, overwrite=False):
        """
        Given a target config filename and rendered config YAML, write it out to file. Create any
containing directories as needed. But if the file already exists and overwrite is False,
abort before writing anything.
        """
        if os.path.exists(config_filename) and overwrite:
                print("Overwriting " +
                #
```

10. Non-Prompted Consideration (NPC) This label is applied when the generated code contains statements that are unrelated to the task specified in the prompt, leading to errors. For instance, in Listing 1, which was presented as an example in the introduction, there is a code generated by Codex, for the task named "parser_flag". The prompt involves "Given an argparse.ArgumentParser instance, returns its argument flags in a space-separated string." As we already discussed in the introduction, although the prompt does not specify sorting the flags, at line 14, the generated function sorts the list of flags before joining them, which is not in line with the given prompt.


```
    … 将其转换为:

      ['sh:**/*foo.txt*/**', 'pp:root/somedir']
    '''
    返回 [查找_路径_到_glob(x) 对于 x 在 查找_路径]中
```

8. 错误属性 当代码包含对象或模块的错误/不存在的属性时，会应用此标签。例如，在清单9中，存在由PanGu-Coder为引言中清单1所示的相同任务生成的代码，命名为"parser_flag"。提示将任务描述为"给定一个argparse.ArgumentParser实例，返回其参数标志的空格分隔字符串。" PanGu-Coder生成的代码试图将输入的"parser"的'flags'作为一个属性来使用，但根据提示中提供的信息，这不是一个正确的属性。清单9中的代码示例被标记为"错误属性"。

列表 9 由盘古编码器生成并标记为"错误属性"的代码。

```
def 解析器_标志(解析器):
    '''
    给定一个 argparse.ArgumentParser 实例，返回其参数标志
    空格–分隔的字符串。
    '''
    return''.join('--{0}'.format(f) for f in parser._flags)
```

9. 不完整生成 当生成的代码不完整，或者被缩减为例如"pass"语句或空函数时，会应用此标签。该错误模式还包括这样一些情况：LLM在函数中途停止生成代码，这可能是因为令牌限制，或者因为达到了 <序列结束_令牌_而没有正确完成代码。CodeGen生成的清单10是此类别的一个例子。

列出由 CodeGen 生成并标记为"生成不完整"的 10 个代码。

```
def 写入_配置(config_文件名, 渲染_配置, 模式=0o600, 覆盖=False):
        """
        给定目标配置文件名和渲染后的配置 YAML，将其写入文件。创建任何
根据需要包含目录。但如果文件已存在且覆盖为 False，
在写任何东西之前中止。
        """
        如果 os.path.exists(config_filename) 并且 overwrite:
                print("覆盖 \" +\"")
                #
```

10. 非提示考虑（NPC） 当生成的代码包含与提示中指定的任务无关的语句，导致错误时，会应用此标签。例如，在引言中作为示例展示的清单 1 中，有 Codex 为名为"parser_flag"的任务生成的代码。提示内容为"给定一个 argparse.ArgumentParser 实例，返回其参数标志，以空格分隔的字符串形式"。正如我们在引言中已经讨论过的，尽管提示没有指定对标志进行排序，但在第 14 行，生成的函数在连接标志之前对标志列表进行了排序，这与给定的提示不符。

4.1.2 Analysis of the Bug Patterns

The bug patterns have been defined based on the generated buggy codes in our studied samples. As mentioned in Sect. 3, some of the buggy code samples are labeled in multiple categories due to the presence of different types of bugs or because a bug could be labeled as an overlap of multiple categories.

Bug Patterns across LLMs - Description Table 2 presents the distribution of bug patterns extracted from the sampled and labeled buggy fragments, across different categories for three LLMs and two prompt types. Some categories are more prevalent in specific models. For instance, in the case of a robust model like Codex, the most common bug pattern is *Missing Corner Case* (**23.53%** of the samples generated with docstring-based prompts and **28.02%** of the samples generated with human-labeled prompts). *Missing Corner Case* occurs when the model generates code that addresses the task description but fails on one or a few exceptional test inputs due to a minor oversight, such as forgetting some particular cases. Listing 6 presents an example of this type. Conversely, in the case of relatively weaker and open-source models like PanGu-Coder or CodeGen, the most common category is *Misinterpretation* for both prompt types whereas for Codex, it is the second most common category. This bug pattern occurs when the code generated by the model does not align with the prompt and deviates from the intended purpose of the task. This result demonstrates that changing the prompt from a docstring-based format to a human-labeled prompt does not significantly improve the comprehension of the models over the intended purpose of the tasks.

Among all models, CodeGen has the highest number of buggy samples in the *Silly Mistake* and *Syntax Error* categories, independently of the type of prompts used. In particular, CodeGen had a tendency to regenerate the code from scratch (signature included) which would cause such an issue as presented in Listing 2. We did not observe the same problem with other LLMs. Codex, as a stronger model, exhibits the highest number of samples in the *Non-Prompted Consideration (NPC)* category, regardless of the prompt types. This cat-

Table 2 The distribution of bug patterns in the generated code by three different LLMs, using two different type of prompts (**Raw**: docstring-based prompts, **Human**: human-labeled prompts)

Bug Pattern	CodeGen		PanGu-Coder		Codex		Total	
	Raw	Human	Raw	Human	Raw	Human	Raw	Human
Misinterpretation	**22.84%**	**31.43%**	**24.11%**	**32.49%**	**15.03%**	**24.18%**	**20.77%**	**29.42%**
Syntax Error	9.14%	**12.57%**	4.96%	1.02%	3.27%	0.55%	6.11%	4.51%
Silly Mistake	**14.72%**	9.14%	4.26%	3.55 %	7.84%	1.65 %	**9.57%**	4.69%
Prompt-biased code	8.12%	2.86%	3.55%	**12.18%**	7.19%	6.59%	6.52%	7.40%
Missing Corner Case	5.58%	11.43%	**19.86%**	**15.23%**	**23.53%**	**28.02%**	**15.27%**	**18.23%**
Wrong Input Type	6.60%	2.86%	8.51%	8.63%	2.61%	6.04%	5.91%	5.96%
Hallucinated Object	5.08%	2.86%	**14.89%**	**12.18%**	10.46%	9.89%	**9.57%**	8.48%
Wrong Attribute	7.61%	6.29%	9.93%	8.63%	8.50%	**11.54%**	8.55%	**8.84%**
Incomplete Generation	**13.71%**	**18.29%**	3.55%	3.05%	9.80%	3.85%	**9.57%**	8.12%
NPC	6.60%	2.29%	6.38%	3.05%	**11.76%**	7.69%	8.15%	4.33%
Total	100%	100%	100%	100%	100%	100%	100%	100%

We put the top categories per LLM in bold

4.1.2 缺陷模式分析

这些缺陷模式是基于我们研究样本中生成的有缺陷代码定义的。如第3节所述，由于存在不同类型的缺陷，或者某个缺陷可以被标记为多个类别的重叠，一些有缺陷的代码样本被标记为多个类别。

跨大语言模型的错误模式 - 描述表 2 展示了从采样和标记的有缺陷代码片段中提取的错误模式在不同类别中的分布，涉及三种大语言模型和两种提示类型。一些类别在特定模型中更为常见。例如，对于像 Codex 这样稳健的模型，最常见的错误模式是缺失角落情况（使用基于文档字符串的提示生成的样本占 23.53%，使用人工标记提示生成的样本占 28.02%）。缺失角落情况发生在模型生成的代码能够处理任务描述，但由于一些小疏忽（例如忘记某些特定情况）而在一个或几个特殊测试输入上失败。列表 6 给出了这种类型的示例。相反，对于像盘古-Coder 或 CodeGen 这样相对较弱且开源的模型，在两种提示类型下最常见的类别都是误解，而对于 Codex，它是第二常见的类别。这种错误模式发生

在所有模型中，CodeGen 在“愚蠢错误”和“语法错误”类别中具有最多的错误样本，这与所使用的提示类型无关。特别是，CodeGen 有从头重新生成代码（包括函数签名）的倾向，这会导致如清单 2 所示的问题。我们在其他大型语言模型中没有观察到相同的问题。作为一个更强的模型，Codex 在“非提示考虑”（NPC）类别中表现出样本数量最多，无论提示类型如何。这个类别

表2 三种不同大型语言模型生成代码中的错误模式分布，使用两种不同类型的提示（Raw：基于文档字符串的提示，Human：人工标注提示）

漏洞模式	代码生成		盘古编程器		法典		总计	
	Raw	人类	Raw	人类	Raw	人类	Raw	人类
误解	**22.84%**	**31.43%**	**24.11%**	**32.49%**	**15.03%**	**24.18%**	**20.77%**	**29.42%**
语法错误	9.14%	**12.57%**	4.96%	1.02%	3.27%	0.55%	6.11%	4.51%
愚蠢的错误	**14.72%**	9.14%	4.26%	3.55 %	7.84%	1.65 %	**9.57%**	4.69%
提示偏向代码	8.12%	2.86%	3.55%	**12.18%**	7.19%	6.59%	6.52%	7.40%
缺失的边角情况	5.58%	11.43%	**19.86%**	**15.23%**	**23.53%**	**28.02%**	**15.27%**	**18.23%**
输入类型错误	6.60%	2.86%	8.51%	8.63%	2.61%	6.04%	5.91%	5.96%
幻觉物体	5.08%	2.86%	**14.89%**	**12.18%**	10.46%	9.89%	**9.57%**	8.48%
错误属性	7.61%	6.29%	9.93%	8.63%	8.50%	**11.54%**	8.55%	**8.84%**
生成不完整	**13.71%**	**18.29%**	3.55%	3.05%	9.80%	3.85%	**9.57%**	8.12%
NPC	6.60%	2.29%	6.38%	3.05%	**11.76%**	7.69%	8.15%	4.33%
总计	100%	100%	100%	100%	100%	100%	100%	100%

我们将每个大型语言模型的顶级类别加粗

egory pertains to situations where the model considers additional conditions or statements that were not requested in the prompt, leading to bugs. For example, the model might convert the final output of a function into an integer, which was not required in the prompt, or check if the output is a string and then apply further functionality (see Listing 1). For both prompt types, the second most common bug pattern in code generated by PanGu-Coder is *Missing Corner Case* and, the third most common bug pattern is *Hallucinated Object* with the docstring-based prompt (**14.89**%), while with the human-labeled prompt, it is a tie between *Hallucinated Object* and *Prompt-biased Code* (**12.18**%). *Hallucinated Object* occurs, for example, when the model erroneously hallucinates the existence of a function that does not exist (see Listing 8). Our observations reveal that when the model struggles to address the prompt, it sometimes resorts to calling an auxiliary function that is not defined, assuming that this function accomplishes the primary objective of the task. Usually, the function's name is generated based on the words used in the prompt. In our proposed taxonomy, *Hallucinated Object* and *Prompt-biased Code* are both subcategories of a higher-level category named *Artifact Hallucination*. For code generated by PanGu-Coder, changing the prompt type reduces the occurrence of bug patterns in *Hallucinated Object* but increases the number of bug patterns in *Prompt-biased Code*.

When considering all three studied models, *Misinterpretation* is the most common bug pattern (**20.77%** for samples generated with docstring-based prompts and **29.42%** for samples generated with human-labeled prompts). The second place belongs to *Missing Corner Cases* with *Hallucinated Object*, *Incomplete Generation* and *Wrong Attribute* following closely in the third position. It is noteworthy that the last three mentioned bug patterns should rarely be observed in code written by humans. For example, human developers typically do not frequently call undefined functions (*Hallucinated Object*) or access attributes on objects that do not exist (*Wrong Attribute*). Some of these bug patterns can be detected by Python Linters embedded in various IDEs, such as VS Code, which further reduces the likelihood of encountering them in human-written code.

The distribution of bug patterns between types of prompts reveals that *Misinterpretation* and *Missing Corner Cases* bug patterns proportion increase while other bug patterns proportion decreases or remain stable. As such, buggy code generated using human-labeled prompts seems to at least contain fewer "trivial" mistakes (such as *Syntax Error*, *Silly Mistake* etc.) than buggy code generated using docstring-based prompts. While this discrepancy could be explained by the sampling error due to the random sampling methodology, the poorer quality of the docstring-based prompts could explain this observation, as outlined by Yu et al. (2023) in the CoderEval dataset "we find that the quality of original natural language descriptions [the docstrings] is rather unstable and inconsistent, so we wonder if they are a good choice as the prompt". Note however that this does not necessarily translate for all LLMs. Indeed, in the case of CodeGen, the proportion of *Incomplete Generation* or *Syntax Error* bug patterns increased.

Bug Patterns across LLMs - Statistical Test As we have categorical distributions, we use the Fisher-exact test (Fisher et al. 1922) to evaluate whether there is a difference between the distributions of bug patterns across LLMs for the codes generated using docstring-based prompts. The test across all LLMs reports a significant (p-value < 0.01) difference. Moreover, using the Cramer-V (Cramér 1999) for effect size returns a medium effect size (0.26) (Cohen 2013). This is similarly the case by applying the test by pair (e.g., distribution of CodeGen vs Codex etc.), except when comparing Codex to PanGu-Coder where the p-value is < 0.05 and effect size is small. As such, the distributions of bugs across LLMs are statistically significantly different.

该类别涉及模型考虑了提示中未要求的额外条件或语句，从而导致错误的情况。例如，模型可能会将函数的最终输出转换为整数，而提示中并未要求，或者检查输出是否为字符串然后应用进一步的功能（见清单 1）。对于这两类提示，PanGu-Coder 生成代码中第二常见的错误模式是缺失边界情况，而第三常见的错误模式是基于文档字符串的提示下出现的虚构对象（14.89%），而在人工标注的提示下，则是虚构对象和提示偏差代码并列（12.18%）。虚构对象通常发生在模型错误地虚构了一个不存在的函数（见清单 8）。我们的观察显示，当模型难以处理提示时，它有时会调用未定义的辅助函数，假设

在考虑所有三种研究模型时，误解是最常见的错误模式（使用基于文档字符串的提示生成的样本中为20.77%，使用人工标注提示生成的样本中为29.42%）。第二位是缺失的边界情况伴随虚构对象，紧随其后的是生成不完整和属性错误，排名第三。值得注意的是，最后提到的这三种错误模式在人类编写的代码中很少出现。例如，人类开发者通常不会频繁调用未定义的函数（虚构对象）或访问不存在对象的属性（属性错误）。其中一些错误模式可以通过嵌入在各种IDE中的Python代码检查器（如VS Code）检测到，这进一步降低了在人工编写代码中遇到这些错误的可能性。

不同类型提示之间的错误模式分布显示，误解（Misinterpretation）和缺失边界情况（Missing Corner Cases）错误模式的比例增加，而其他错误模式的比例则下降或保持稳定。因此，使用人工标注提示生成的有缺陷代码似乎至少包含比使用基于文档字符串提示生成的有缺陷代码更少的“琐碎”错误（例如语法错误、愚蠢错误等）。虽然这种差异可以用随机抽样方法引起的抽样误差来解释，但基于文档字符串的提示质量较差也可以解释这一观察结果，正如 Yu 等人（2023）在 CoderEval 数据集中的说明：“我们发现原始自然语言描述（v1）——即文档字符串（v2）的质量相当不稳定且不一致，因此我们想知道它们是否是作为提示的好选择。” 需要注意的是，这并不一定适用于所有大型语言模型（LLM）。事实上，在 CodeGen 的情况下，生成不完整或语法错误的比例

跨大型语言模型（LLMs）的错误模式——统计检验 由于我们拥有类别分布，我们使用Fisher精确检验（Fisher等，1922）来评估使用基于文档字符串提示生成的代码的错误模式在不同LLMs之间的分布是否存在差异。对所有LLMs的检验报告了显著的（p值为 < 0.01）差异。此外，使用Cramer-V（Cramér 1999）计算效应量返回中等效应量（0.26）（Cohen 2013）。通过成对检验（例如CodeGen与Codex等的分布）也得出类似结果，唯一例外是比较Codex与PanGu-Coder时，p值为 < 0.05，效应量为小。因此，不同LLMs之间的错误分布在统计上存在显著差异。

When analyzing the distribution of bug patterns obtained in the code generated using human-labeled prompts, we observe results consistent with those from raw-docstring prompts. Specifically, there is a significant difference (p-value < 0.01) with a medium effect size across all LLMs, as well as in pair comparisons between CodeGen and Codex, and CodeGen and PanGu-Coder. In the case of a comparison between Codex and PanGu-Coder we find a p-value < 0.05 and a small effect size. To further contrast the difference, we compute the Fisher-exact test for the same LLM across prompt type (raw docstring vs human label): we find a statistical significance in all cases (p-value < 0.05) with a small effect size. As such, the prompt type induces a change in bug patterns, moreover, the distribution of bug patterns varies with the LLM used.

Bug Patterns Across Dependency Levels - Description As discussed in Sect. 3.1.1, our results are limited to tasks with runnable levels no higher than *"plib_runnable"* (dependent only on Python's standard and public libraries, such as "numpy"). The goal of this limitation is to mitigate potential biases in our results that may arise from the lack of information on dependencies not covered in the prompt. However, to further our analysis, we assess whether the three selected runnable levels in our sample set cause any bias toward specific bug patterns. To conduct this analysis, we report the distribution of bug patterns in different runnable levels of tasks across LLMs and different prompt types (Docstring-based/Human-labeled), as shown in Table 3.

The results, as indicated in Table 3, show a varied distribution of bug patterns across different runnable levels. Similar to the result in Table 2, the *Misinterpretation* category contains the highest number of buggy code snippets across all dependency levels and prompt types. The second most frequent bug pattern is *Missing Corner Case* for both prompt types at the *"slib_runnable"* and *"plib_runnable"*. However, at the *"self_runnable"* level the second most common bug pattern is *Missing Corner Case* for human-labeled prompts, while for docstring-based prompts, the *Wrong Attribute* pattern ranks second. While higher runnable levels are more likely to cause generating buggy code by LLMs, as highlighted in CoderEval

Table 3 Distribution of bug patterns across different runnable levels (levels of task dependencies) with two different types of prompts

Bug Pattern	self_runnable		slib_runnable		plib_runnable		Total	
	Raw	Human	Raw	Human	Raw	Human	Raw	Human
Misinterpretation	**16.29%**	**28.17%**	**24.86%**	**31.02%**	**23.53%**	**29.07%**	**20.77%**	**29.42%**
Syntax Error	7.69%	6.75%	4.32%	3.24%	5.88%	1.16%	6.11%	4.51%
Silly Mistake	8.60%	4.76%	**11.35%**	4.62%	8.24%	4.65%	**9.57%**	4.69%
Prompt-biased Code	8.14%	8.33%	6.49%	6.02%	2.35%	8.14%	6.52%	7.40%
Missing Corner Case	**10.41%**	**15.87%**	**17.84%**	**18.98%**	**22.35%**	**23.26%**	**15.27%**	**18.23%**
Wrong Input Type	6.79%	5.16%	4.86%	6.02%	5.88%	8.14%	5.91%	5.96%
Hallucinated Object	9.95%	7.14%	9.73%	**12.50%**	8.24%	2.33%	**9.57%**	8.48%
Wrong Attribute	**13.12%**	**9.52%**	2.16%	6.48%	**10.59%**	**12.79%**	8.55%	**8.84%**
Incomplete Generation	9.50%	8.33%	10.27%	6.94%	8.24%	10.47%	**9.57%**	8.12%
NPC	9.50%	5.95%	8.11%	4.17%	4.71%	0.00%	8.15%	4.33%
Total	100%	100%	100%	100%	100%	100%	100%	100%

(**Raw**: docstring-based prompts, **Human**: human-labeled prompts). We put the top categories per LLM in bold

在分析使用人工标注提示生成的代码中获得的错误模式分布时，我们观察到的结果与使用原始文档字符串提示得到的结果一致。具体来说，所有大型语言模型（LLM）中均存在显著差异（p 值 < 0.01）且具有中等效应量，在 CodeGen 与 Codex 以及 CodeGen 与 PanGu-Coder 的成对比较中也表现出相同特征。在 Codex 与 PanGu-Coder 的比较中，我们发现 p 值为 < 0.05 且效应量较小。为了进一步对比差异，我们对同一 LLM 在不同提示类型（原始文档字符串 vs 人工标注）下计算了费舍尔精确检验：所有情况均具有统计显著性（p 值 < 0.05），效应量较小。因此，提示类型会引起错误模式的变化，此外，错误模式的分布还随所使用的 LLM 而变化。

不同依赖级别的错误模式 - 描述 如第 3.1.1 节所讨论的，我们的结果仅限于可运行级别不高于“plib_runnable”的任务（仅依赖于 Python 的标准库和公共库，例如 “numpy”）。这种限制的目的是为了减轻由于提示中未覆盖的依赖信息缺失可能对我们的结果产生的潜在偏差。然而，为了进一步分析，我们评估在样本集中选择的三个可运行级别是否会导致对特定错误模式的偏向。为了进行此分析，我们报告了不同可运行级别任务中错误模式在各大语言模型和不同提示类型（基于文档字符串/人工标注）中的分布，如表 3 所示。

如表3所示，结果显示不同可运行级别的缺陷模式分布各不相同。与表2的结果类似，误解类别在所有依赖级别和提示类型中包含最多的有缺陷代码片段。第二常见的缺陷模式是在“slib_runnable”和“plib_runnable”级别上，两种提示类型中均为缺失边界情况。然而，在“self_runnable”级别，对于人工标注的提示，第二常见的缺陷模式是缺失边界情况，而对于基于文档字符串的提示，第二位的是错误属性模式。正如CoderEval所强调的，更高的可运行级别更可能导致LLM生成有缺陷的代码。

表3 不同可运行级别（任务依赖级别）下两种不同类型提示的错误模式分布

错误模式	自运行_		slib 可运行_		plib_可运行的		总计	
	Raw	人类	Raw	人类	Raw	人类	Raw	人类
误解	**16.29%**	**28.17%**	**24.86%**	**31.02%**	**23.53%**	**29.07%**	**20.77%**	**29.42%**
语法错误	7.69%	6.75%	4.32%	3.24%	5.88%	1.16%	6.11%	4.51%
愚蠢的错误	8.60%	4.76%	**11.35%**	4.62%	8.24%	4.65%	**9.57%**	4.69%
提示偏向代码	8.14%	8.33%	6.49%	6.02%	2.35%	8.14%	6.52%	7.40%
缺失的边角情况	**10.41%**	**15.87%**	**17.84%**	**18.98%**	**22.35%**	**23.26%**	**15.27%**	**18.23%**
输入类型错误	6.79%	5.16%	4.86%	6.02%	5.88%	8.14%	5.91%	5.96%
幻觉物体	9.95%	7.14%	9.73%	**12.50%**	8.24%	2.33%	**9.57%**	8.48%
错误属性	**13.12%**	**9.52%**	2.16%	6.48%	**10.59%**	**12.79%**	8.55%	**8.84%**
生成不完整	9.50%	8.33%	10.27%	6.94%	8.24%	10.47%	**9.57%**	8.12%
NPC	9.50%	5.95%	8.11%	4.17%	4.71%	0.00%	8.15%	4.33%
总计	100%	100%	100%	100%	100%	100%	100%	100%

(原始：基于文档字符串的提示，人类：人工标注的提示)。我们将每个大语言模型的顶级类别加粗显示

benchmark (Yu et al. 2023), the results in Table 3 do not demonstrate that increasing the dependency level of the task up to *"plib_runnable"* impacts the bug patterns in code snippets for most of the bug patterns in both prompt types. However, when using human-labeled prompts, the number of buggy code snippets across all runnable levels decreases in the *Silly Mistake*, *Syntax Error*, and *NPC* categories. For instance, with human-labeled prompts and as the dependency level of tasks increases to *"plib_runnable"*, the proportion of buggy code snippets in the *Syntax Error* category drops to 1.16% (compared to 5.88% with docstring-based prompts), and for *NPC*, it decreases to zero (from 4.71% with docstring-based prompts). Likewise, we observe an increasing trend with runnable level and prompt types for *Missing Corner Case* or *Prompt-biased code*. Similar trends are not observable for other types of bugs at different runnable levels. Therefore, considering the runnable levels in our dataset, our proposed bug patterns are not influenced by a specific runnable level.

Bug Patterns Across Dependency Levels - Statistical Test Similarly to distributions across LLMs, we compute the Fisher-exact test for the distributions across dependency-level when dealing with code generated with the docstring-based prompts. In that case, the test is only significant (p-value < 0.01) when comparing all dependency-levels or when comparing self-contained to slib-runnable dependency levels. In both cases, the Cramer-V representing the effect size is small (0.19 and 0.26 respectively). In the case of comparing self-contained to plib-runnable or slib-runnable and plib-runnable, we find no significance. As such, the distributions of bugs across dependency-level might not be different. The only difference we can highlight is between bugs at the self-contained (i.e., no library used) and slib-runnable (i.e., at most a standard library used) dependency-level.

When using the distribution of bug patterns obtained from the codes generated using human-labeled prompts, we find the inverse of the raw-label prompts: there is a statistically significant difference between self-contained and plib-runnable on one hand and between slib-runnable and plib-runnable (p-value < 0.05). Similarly, we find that the comparison across all LLMs is significant (p-value < 0.05). In all cases, the Cramer-V returns a small effect size. We however do not have a significant difference between self-contained and slib-runnable. As such, the type of prompt seems to have some impact on the bug patterns expressed at different dependency levels. To further verify this, we then proceed to calculate the Fisher-exact test for the same dependency-level across prompt type (raw docstring vs human-labeled): we find a significant difference in the case of self-contained (p-value < 0.05) and slib-runnable (p-value $= 0.056 < 0.01$). In both cases, the effect size is small. We do not find any significant difference for plib-runnable.

Findings 1: Most buggy samples in CoderEval exhibit *Misinterpretation* and *Missing Corner Cases* bug patterns, yet there are also a non-trivial number of other bug patterns such as *Hallucinated Object* or *Wrong Attribute*. Several bug patterns do not seem to be typical of human developers, either because they show a strange behavior (e.g., *Silly Mistake* with duplicated if/else condition) or because they would be easily detected using an IDE. The prompt types affect the distribution of bug patterns across LLMs and across dependency-level, as seen by the difference between the results obtained with human-labeled and docstring-based prompts.

Bug Patterns Across Task To further study bug patterns in LLM-generated code on the CoderEval benchmark, we look into what bug patterns are present, for each task in the

基准（Yu 等，2023），表 3 中的结果并未表明，将任务的依赖级别提高到“plib_runnable”会影响大多数类型的错误模式在两种提示类型的代码片段中的表现。然而，当使用人工标注的提示时，在 Foolish Mistake（愚蠢错误）、Syntax Error（语法错误）和 NPC 分类中，各个 runnable 级别的 buggy 代码片段数量都有所减少。例如，使用人工标注的提示，并且随着任务依赖级别提高到“plib_runnable”，语法错误类别的 buggy 代码片段比例降至 1.16%（而基于 docstring 的提示为 5.88%），NPC 类别降至零（而基于 docstring 的提示为 4.71%）。同样地，对于缺失的边角案例或提示偏置代码，随着 runnable 级别和提示类型的提高，也观察到类似的增加趋势。在不同 runnable 级别下，其他类型的错误没有观察到类似趋势。因此，考虑到我们数据集中的 runnable 级别，o

跨依赖级别的错误模式 - 统计检验 类似于跨大型语言模型 (LLM) 的分布，我们在处理使用基于文档字符串提示生成的代码时，计算依赖级别分布的 Fisher 精确检验。在这种情况下，仅在比较所有依赖级别或比较自包含与 slib 可运行依赖级别时，检验结果才显著（p 值 < 0.01）。在这两种情况下，表示效应大小的 Cramer-V 都很小（分别为 0.19 和 0.26）。在比较自包含与 plib 可运行、或 slib 可运行与 plib 可运行时，我们未发现显著性。因此，错误在不同依赖级别的分布可能没有差异。我们唯一可以强调的差异是自包含（即未使用任何库）与 slib 可运行（即最多使用标准库）依赖级别的错误之间的差异。

在使用从人类标注提示生成的代码中获得的错误模式分布时，我们发现原始标签提示的情况恰好相反：自包含（self-contained）与 plib 可运行（plib-runnable）之间，以及 slib 可运行（slib-runnable）与 plib 可运行之间存在统计显著差异（p 值 < 0.05）。同样，我们发现所有大型语言模型（LLM）之间的比较也是显著的（p 值 < 0.05）。在所有情况下，Cramer-V 返回的效应量都很小。然而，自包含与 slib 可运行之间没有显著差异。因此，提示类型似乎对不同依赖级别表达的错误模式有一定影响。为了进一步验证这一点，我们接着计算了相同依赖级别下不同提示类型（原始 docstring 与人类标注）之间的 Fisher 精确检验：在自包含（p 值 < 0.05）和 slib 可运行（p 值 $= 0.056 < 0.01$）的情况下，我们发现存在显著差异。在这两种情况下，效应量都很小。我们没有发现任何显著

发现 1：CoderEval 中的大多数有缺陷样本表现出误解和缺失边界情况的错误模式，但也存在相当数量的其他错误模式，例如虚构对象或错误属性。一些错误模式似乎并非人类开发者的典型行为，要么因为它们表现出奇怪的行为（例如带有重复 if/else 条件的愚蠢错误），要么因为使用 IDE 就可以轻易检测到。提示类型会影响 LLMs 以及依赖级别上错误模式的分布，这可以通过人工标注提示和基于文档字符串的提示获得的结果差异看出。

不同任务中的错误模式 为了进一步研究 LLM 生成代码在 CoderEval 基准上的错误模式，我们考察每个任务中存在的错误模式

benchmark, across LLMs. The goal is to identify if some tasks in the dataset are more likely to lead to a specific bug pattern across all LLMs which could indicate some potential shortcomings in the prompts or if bug patterns are evenly distributed across different tasks in the CoderEval benchmark. To do so, we counted in our sample set the number of times a particular bug pattern was exhibited, regardless of the employed LLM for each task. We then normalized values per task to know what bug patterns per task were more prevalent. Results are presented as a heatmap in Fig. 3.

Overall, the various bug patterns are distributed somewhat evenly across different tasks, as indicated by the darker areas in both heatmaps, Figs. 3 and 4. This suggests that the tasks are generally not biased toward a particular bug pattern that we identified. However, we note some very light spots on each heatmap (greater than 0.9), highlighting tasks where the

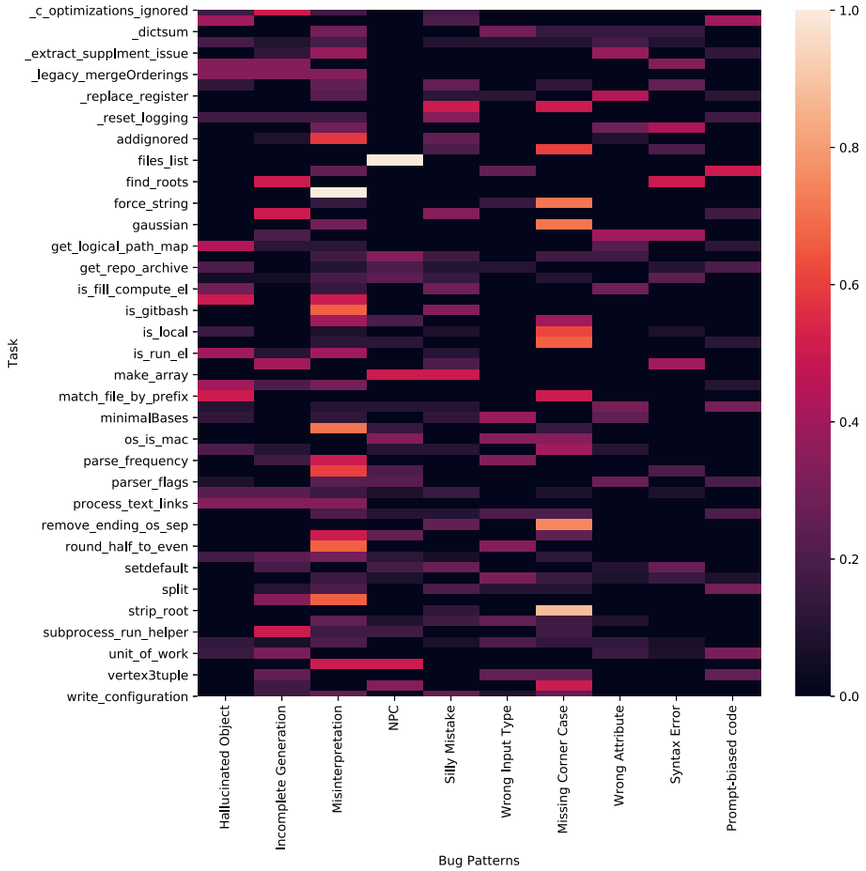


Fig. 3 The heatmap illustrates the distribution of bug patterns on various tasks across all LLMs for the docstring-based prompt. The values are normalized over lines per task. The brighter areas represent a higher number of code samples of a specific task categorized into a particular bug pattern. The black areas indicate the lack of code samples in the bug pattern for a specific task

基准测试，跨越大型语言模型（LLMs）。目标是确定数据集中是否有某些任务更容易导致所有LLM出现特定的错误模式，这可能表明提示存在潜在不足，或者错误模式在CoderEval基准的不同任务中是否均匀分布。为此，我们在样本集中统计了每个任务中出现特定错误模式的次数，而不考虑使用的LLM。然后我们对每个任务的数值进行了归一化，以了解每个任务中哪些错误模式更为常见。结果以热图形式呈现在图3中。

总体而言，如图3和图4的热图中较深的区域所示，各种缺陷模式在不同任务中的分布相对均匀。这表明这些任务通常不会偏向我们识别的特定缺陷模式。然而，我们注意到每个热图上都有一些非常浅的点（大于0.9），突出了某些任务，其中的

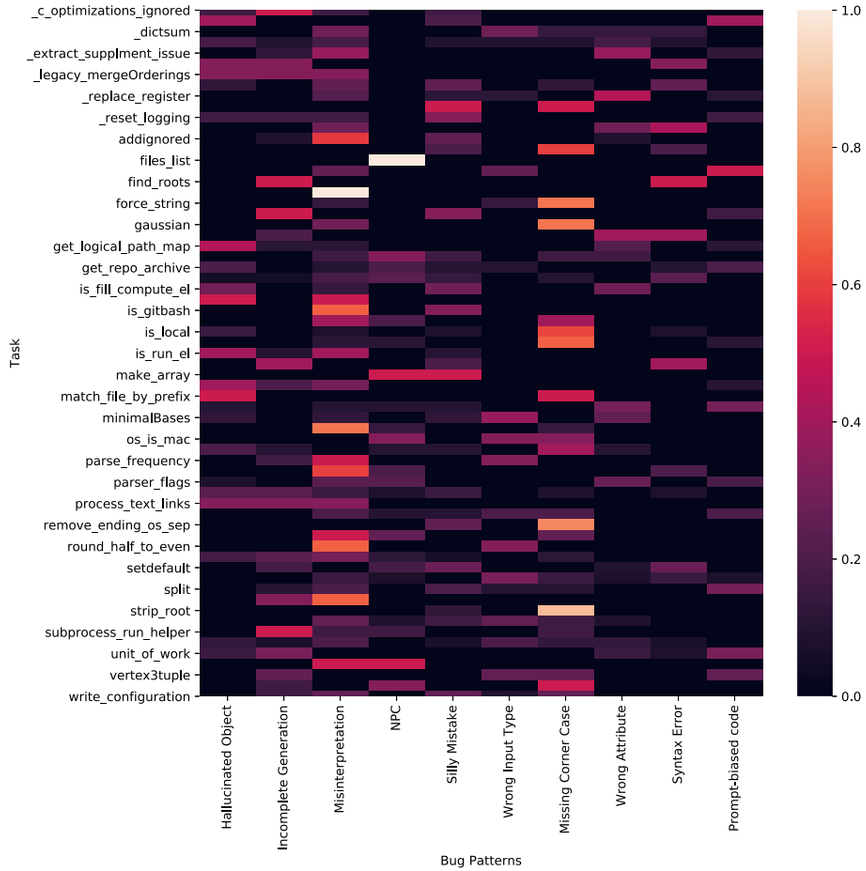


图3 热力图显示了在基于文档字符串的提示下，各大语言模型在不同任务上的错误模式分布。数值已按每个任务的代码行数进行归一化。较亮的区域表示特定任务中属于特定错误模式的代码样本数量较多。黑色区域表示在特定任务中该错误模式的代码样本缺失

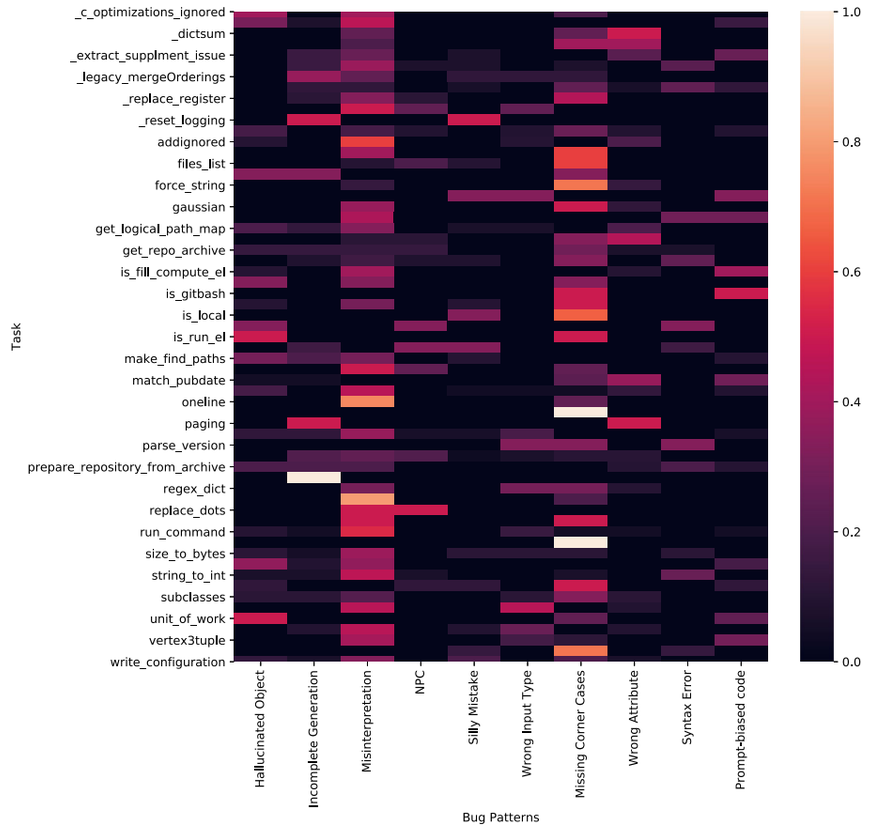


Fig. 4 The heatmap illustrates the distribution of bug patterns on various tasks across all LLMs for the human-labeled prompt. The values are normalized over lines per task. The brighter areas represent a higher number of code samples of a specific task categorized into a particular bug pattern. The black areas indicate the lack of code samples in the bug pattern for a specific task

majority of code samples are categorized under a particular bug pattern. For example, for the task named “fix_namespace_prefix_w” where all the buggy code snippets generated by different LLMs are categorized as *Misinterpretation*. This classification stems from the fact that the prompt associated with this task, “Convert text that defaults to ‘w:st=‘’ ’ to ‘w-st=‘’ ’, does not accurately represent the functionality revealed by the oracle and corresponding test cases. However, all models generated at least one correct solution based on the same prompt, which passed all the test cases. Regarding the distribution of bug patterns across tasks when using human-labeled prompts (see Fig. 4), we observe similar trends. For example, under this type of prompt, all the buggy code snippets for only three tasks fall under a single bug pattern. However, the prevalence of specific bug patterns for a given task varies between the two prompt types. For instance, in the task “file_list”, the majority of code samples exhibit the *NPC* bug pattern when generated using docstring-based prompts. In contrast, with human-labeled prompts, less than 50% of the samples display this pattern, with most instead showing the *Missing Corner Cases* bug pattern. Similarly, as another example, for the task “setdefault”, all buggy code snippets are classified as *Missing Corner Cases* when

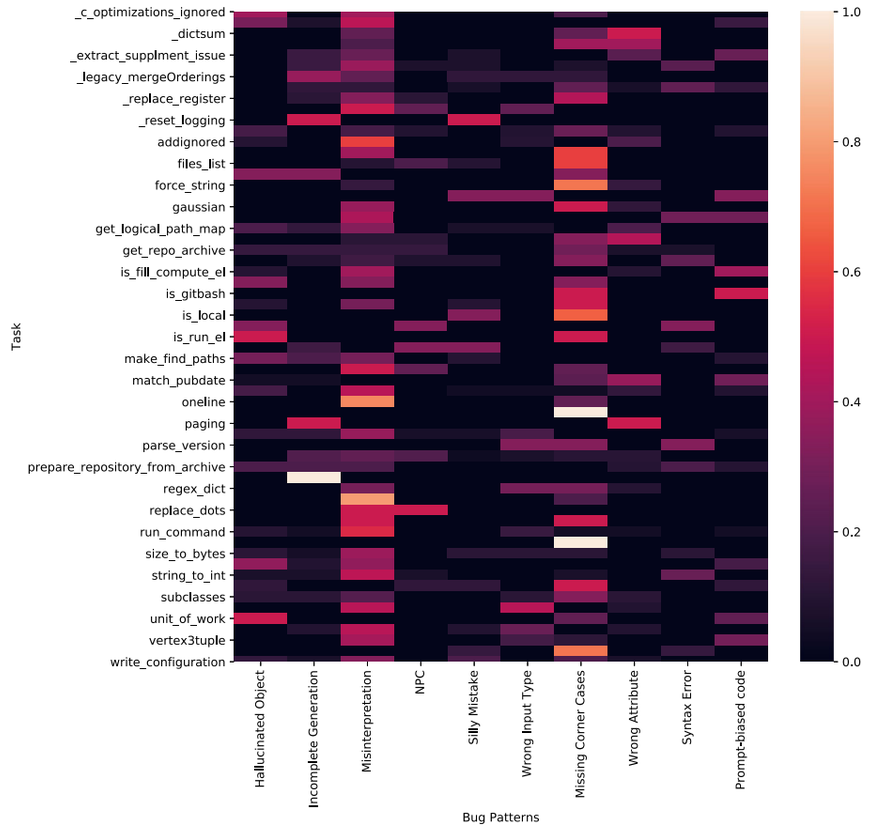


图4 热力图展示了在人工标注提示下，各大语言模型（LLM）在不同任务上的漏洞模式分布。数值已按每个任务的代码行数进行归一化。较亮的区域表示特定任务中属于特定漏洞模式的代码样本数量较多。黑色区域表示在特定任务中该漏洞模式的代码样本缺失。

大多数代码示例被归类到特定的错误模式下。例如，对于名为“fix_namespace_prefix_w”的任务，所有由不同大型语言模型生成的有错误的代码片段都被归类为误解。这种分类来源于与该任务相关的提示“将默认值为‘w:st=’的文本转换为‘w-st=’”，它并不能准确表示由oracle和相应测试用例揭示的功能。然而，所有模型基于相同的提示至少生成了一个正确的解决方案，并通过了所有测试用例。关于使用人工标注提示时任务间错误模式的分布（参见图4），我们观察到类似的趋势。例如，在这种类型的提示下，只有三个任务的所有有错误代码片段属于单一错误模式。然而，对于给定任务，特定错误模式的普遍性在两种提示类型间存在差异。例如，在任务“file_list”中，

using human-labeled prompts. However, with docstring-based prompts, the buggy examples are distributed across multiple categories, such as *Wrong Attribute*, *Silly Mistake*, or *NPC*.

Additionally, as shown in Fig. 3, the majority of bug patterns are concentrated in a specific type for only a few tasks. For example, in the "force_string" task, up to 72% of the bug patterns generated by different models are categorized as *Missing Corner Case* due to an additional newline character in the prompt. The prompt for this task reads, "This function returns the bytes object corresponding to 'obj' / in case it is a string using UTF-8.". The content after the backslash appears as a new line in the prompt, creating ambiguity in the prompt's objective for all LLMs. In the buggy code snippets, the function attempts to return the bytes object corresponding to 'obj'. In cases where 'obj' is a string, it decodes it to UTF-8. However, upon checking the oracle, it becomes evident that the intended purpose of the prompt is to return the corresponding UTF-8 if 'obj' is an instance of a byte object.

Findings 2: The bug patterns are distributed over different tasks in the CoderEval dataset. However, our results reveal that the prompt types may impact the distribution of buggy samples within a specific task.

4.2 RQ2: To What Extent are the Identified Bug Patterns in LLM-generated Code Relevant for Software Practitioners and Researchers Working with LLMs?

For both surveys, we obtained the following number of participants: for the first survey, **22** participants using the platform Prolific; for the second survey, we obtained **45** participants, 34 via solicitation with email/Reddit and 11 via Prolific. The solicitation via email was open for two weeks and we achieved a return rate of 8.9%, which is in line with many SE surveys conducted outside specific companies (Nardone et al. 2023; Nikanjam et al. 2021). We first give results of the questions concerning the demographics and experience of the participants. We then detail the results regarding the first survey with the bug patterns reported that are present/absent in our taxonomy. Finally, we detail the results regarding the second survey with the frequency of reported bug patterns, as well as the analysis of the reported difficulty of diagnosing/fixing and the perceived complexity of the bug patterns.

4.2.1 Participants' Demographics and Experience with LLM bugs

All the participants answered the demographic questions. For the first survey:

- Professions: 5 Developers, 5 Software Engineers, 3 Data analysts, 4 IT-related jobs (QA tester, SysAdmin etc.) and 5 non-related (Manager, Student etc.)
- ChatGPT was used by **20** participants. **72**% of the participants reported only non-open-source LLMs such as ChatGPT, Bard (Chowdhery et al. 2023) or Claude (Anthropic 2013). **6** participants cited open-source LLMs such as LLama or CodeGen.
- In terms of the programming language used for LLMs, besides Python, the participants responded: **77%** for JavaScript, **50%** for C++ and Java and **36%** for C. These languages are among the most used programming languages according to different programming language indexes (gitindex 2023; tiobeindex 2023). Other programming languages were reported in smaller proportions: Go, C# or Rust.

使用人工标注的提示。然而，对于基于文档字符串的提示，错误示例分布在多个类别中，例如错误属性、愚蠢的错误或 NPC。

此外，如图3所示，大多数缺陷模式集中在少数任务的特定类型中。例如，在“force_string”任务中，不同模型生成的缺陷模式中多达72%被归类为缺少边界情况，这是由于提示中额外的换行符导致的。该任务的提示为：“This function returns the bytes object corresponding to 'obj' / in case it is a string using UTF-8.”。反斜杠后的内容在提示中显示为新的一行，使得所有大语言模型对提示的目标产生歧义。在有缺陷的代码片段中，该函数试图返回对应'obj'的字节对象。在'obj'是字符串的情况下，将其解码为UTF-8。然而，经过检查参考规范后，可以清楚地看出，该提示的预期目的是如果'obj'是字节对象的实例，则返回对应的UTF-8。

发现 2：错误模式分布在 CoderEval 数据集的不同任务中。然而，我们的结果显示，提示类型可能会影响特定任务中错误样本的分布。

4.2 研究问题2：在多大程度上，在大语言模型生成的代码中识别出的缺陷模式与使用大语言模型的软件从业者和研究人员相关?

对于两项调查，我们获得了以下参与者数量：第一项调查中，通过Prolific平台有22名参与者；第二项调查中，我们获得了45名参与者，其中34人通过电子邮件/Reddit招募，11人通过Prolific平台参与。电子邮件招募开放了两周，我们达到了8.9%的回复率，这与许多在特定公司之外进行的软件工程调查（Nardone 等，2023；Nikanjam 等，2021）一致。我们首先给出有关参与者人口统计和经验的问题结果。然后，我们详细说明第一项调查中的结果，即报告的错误模式在我们的分类法中是存在还是缺失的。最后，我们详细说明第二项调查的结果，包括报告的错误模式的频率，以及诊断/修复难度的分析和对错误模式复杂性的感知分析。

4.2.1 参与者的人口统计信息及其在大型语言模型缺陷方面的经验

所有参与者都回答了人口统计学问题。对于第一次调查：

– 职业：5名开发人员，5名软件工程师，3名数据分析师，4个 IT 相关职位（QA 测试员、系统管理员等）以及5个非相关职位（经理、学生等） – 20 名参与者使用了 ChatGPT。72% 的参与者报告仅使用非开源大语言模型（LLM），如 ChatGPT、Bard（Chowdhery 等，2023）或 Claude（Anthropic，2013）。6 名参与者提到使用开源 LLM，如 LLama 或 CodeGen。 – 在 LLM 编程语言方面，除了 Python 外，参与者的回答如下：77% 使用 JavaScript，50% 使用 C++ 和 Java，36% 使用 C。这些语言均为不同编程语言指数中最常用的编程语言之一（gitindex 2023；tiobeindex 2023）。其他编程语言的使用比例较小：Go、C# 或 Rust。

For the second survey:

- Professions: For the academic field, there were 12 Ph.D. students, 4 Researchers, 8 Undergraduate/Graduate students and 1 Lecturer. For the industry field, there were 11 Developers, 4 Data scientists, two Technical artists, one Software Engineer, one Software Tester, and a CTO.
- ChatGPT was used by **41** participants. **75**% of the participants reported only non-open-source LLMs such as ChatGPT, Bard (Chowdhery et al. 2023) or Claude (Anthropic 2013). **11** participants cited open-source LLMs such as LLama or CodeGen.
- In terms of the programming language used for LLMs, besides Python, the participants responded: **47%** for JavaScript, **29%** for C++ and Java and **22%** for C. These languages are among the most used programming languages according to different programming language indexes (gitindex 2023; tiobeindex 2023). Other programming languages were reported in smaller proportion: Go (9%), C# (9%), SQL (7%), Rust (4%) or HTML/CSS (4%). Four respondents (three PhD students and one Data scientist) reported exclusively using LLMs only to program in Python language.

Note that the sum of responses for LLMs and programming languages goes above the number of responses as one could provide multiple answers.

For the second survey, before asking participants specific questions about the categories of bugs, we collected information about their general experience dealing with bugs in LLM-generated code. **71%** of the participants reported that they often or always experience bugs when using LLMs to generate code. No respondent said they *never* faced bugs in LLM-generated code. We then asked the participants about the complexity of the bugs they encountered in their LLM code generation activities and **64%** of them mentioned that the bugs had a medium or more difficulty in terms of fixing, with only **27%** of participants reporting that their encountered bugs were *easy* to fix. We also asked the participants how they proceeded to fix the encountered bugs and **33%** of them reported that they fixed the bugs manually by reading the stack traces and googling the error messages. The remaining participants reported using a combination of manual inspection with re-prompting the LLM to correct the mistake. No participant declared only using re-prompting based on the error message, similar to approaches presented in Schäfer et al. (2023). This highlights the fact that debugging LLM-generated code is still predominantly a manual task. We believe that the bug patterns proposed in this paper are an important first step toward the development of efficient automatic debugging tools for LLM-generated code.

4.2.2 Bug Patterns Observed by Respondents in LLM-generated Code

In the first survey, respondents were simply asked to list bug patterns they encountered, without having access to the taxonomy. After analyzing the respondents' answers, we first observed that 68% of the participants reported as bug patterns some forms of *Misinterpretation*/*Missing Corner Cases* and 55% of the participants reported encountering some form of *Syntax Error*. Participants also mentioned bug patterns linked to what we qualified as *Wrong Input Type*, *Wrong Attribute* and *Hallucinated Object*. Bug patterns such as *Silly Mistake* or *NPC* were not mentioned explicitly, but given that respondents said they encountered them in the second survey (see next Section), participants not reporting them in this survey might just mean they might not be the most striking bug patterns to them compared to *Syntax Error* or *Misinterpretation*.

对于第二次调查：

– 职业：在学术领域，有12名博士生，4名研究人员，8名本科/研究生学生和1名讲师。在工业领域，有11名开发者，4名数据科学家，2名技术美术人员，1名软件工程师，1名软件测试员，以及1名首席技术官（CTO）。

– 共有41名参与者使用了ChatGPT。75%的参与者报告只使用非开源的大型语言模型（LLM），如ChatGPT、Bard（Chowdhery 等，2023）或Claude（Anthropic，2013）。11名参与者提到了开源LLM，如LLama或CodeGen。 – 在LLM使用的编程语言方面，除了Python，参与者的回答如下：47%使用JavaScript，29%使用C++ 和Java，22%使用C。这些语言都是根据不同编程语言索引（gitindex 2023；tiobeindex 2023）最常用的编程语言之一。其他编程语言的使用比例较小：Go（9%）、C#（9%）、SQL（7%）、Rust（4%）或HTML/CSS（4%）。四名受访者（三名博士生和一名数据科学家）报告仅使用LLM来进行Python语言编程。

请注意，LLM 和编程语言的回答总和超过了回答数量，因为可以提供多个答案。

在第二次调查中，在询问参与者关于错误类别的具体问题之前，我们收集了他们处理由大语言模型（LLM）生成代码的错误的整体经验。71%的参与者报告说，当使用LLM生成代码时，他们经常或总是遇到错误。没有受访者表示他们从未在LLM生成的代码中遇到过错误。随后，我们询问参与者在LLM代码生成活动中遇到的错误的复杂性，其中64%的参与者提到，这些错误在修复方面具有中等或更高的难度，仅有27%的参与者报告他们遇到的错误容易修复。我们还询问参与者他们是如何修复遇到的错误的，33%的参与者报告说他们通过阅读堆栈跟踪和搜索错误信息手动修复错误。其余参与者则报告使用手动检查结合重新提示LLM来纠正错误的方式。没有参与者

4.2.2 受访者在大型语言模型生成代码中观察到的错误模式

在第一次调查中，受访者只是被简单地要求列出他们遇到的缺陷模式，而没有接触到分类法。在分析了受访者的回答后，我们首先观察到，68%的参与者报告了一些误解/遗漏角落情况形式的缺陷模式，55%的参与者报告遇到了一些语法错误形式的缺陷模式。参与者还提到了与我们定义为错误输入类型、错误属性和虚构对象相关的缺陷模式。像愚蠢错误或NPC这样的缺陷模式没有被明确提及，但鉴于受访者在第二次调查中表示遇到过它们（见下一节），参与者在本次调查中未报告它们可能只是意味着与语法错误或误解相比，它们对他们来说可能不是最显著的缺陷模式。

Besides bug patterns identified in our samples, participants reported several other bug patterns. Their absence from our taxonomy can be explained by two factors: 1) lack of sufficient instances in our samples to warrant an individual category, 2) patterns that can not be identified in CoderEval/Python. For the first case, several respondents identified bug patterns such as infinite loops, out-of-bound arrays or overflow recursive calls. In our samples, we similarly observed some individual instances, but there were not enough samples of these patterns to justify a separate sub-category. The bugs with a limited number of samples were labelled in a more general category (e.g. Wrong Input Type, etc.). Regarding the second case, several bug patterns pertain to issues not covered in CoderEval, such as code optimality or security vulnerabilities. These bug patterns are absent from CoderEval samples because the benchmark does not focus on such concerns. For example, CoderEval prompts do not require an optimal solution for a task, and the oracle provided may not represent the most optimal solution. One respondent also reported bug patterns dealing with inconsistency between the generated codes and the rest of the codebase. This can not directly be observed in our case as the prompts do not rely on an external code base. However, Hallucinated Objects could be encompassing this use case if we were to deal with this category: that is, objects not present in the codebase but that the LLM-generated code will still try to use. Finally, some respondents pointed out some bug patterns that are language-specific: for instance, null-pointer exceptions in C or some specific API call in React. In our case, using Python, we obviously did not encounter such issues. Thus, to expand our taxonomy, we further add three new categories, based on the participants' answers, which were mentioned at least twice by different participants. Namely, Optimality, Security and Memory issues (see Fig. 2).

4.2.3 Frequency of Bugs Patterns

In the second survey, participants were asked to validate our observed bug patterns and provide an analysis of their characteristics. Figure 5 summarizes the obtained aggregated results in the "Frequency" column. For each bug pattern, we report the following: the frequency at which participants declared they encountered the bug pattern, what is the perceived difficulty in

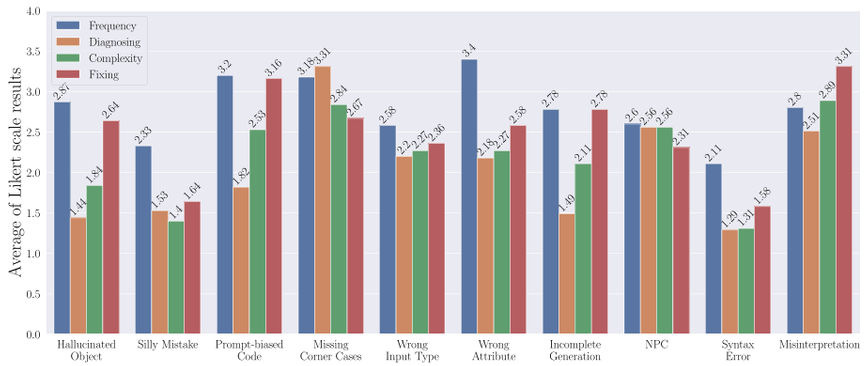


Fig. 5 Aggregated results of the validation survey. Questions related to the frequency of encounter of bug patterns, the difficulty to diagnose and fix them as well as the complexity of the bug. 1 represents never/easy/trivial/low and 5 represents always/hard/complex/high

除了在我们的样本中识别出的错误模式外，参与者还报告了其他几种错误模式。它们未出现在我们的分类法中可以由两个因素解释：1）我们的样本中缺乏足够的实例来支持单独的类别，2）在 CoderEval/Python 中无法识别的模式。对于第一种情况，一些受访者识别出了诸如无限循环、数组越界或递归调用溢出的错误模式。在我们的样本中，我们也观察到了一些个别实例，但这些模式的样本数量不足以证明设立单独的子类别。样本数量有限的错误被归入更一般的类别（如错误的输入类型等）。关于第二种情况，一些错误模式涉及 CoderEval 未涵盖的问题，如代码优化或安全漏洞。这些错误模式在 CoderEval 样本中缺失，因为该基准不关注这些问题。例如，CoderEval 的提示不 r

4.2.3 缺陷模式的频率

在第二次调查中，参与者被要求验证我们观察到的缺陷模式并提供其特征分析。图5总结了“频率”栏中获得的汇总结果。对于每种缺陷模式，我们报告以下内容：参与者声明遇到该缺陷模式的频率，以及感知的难度

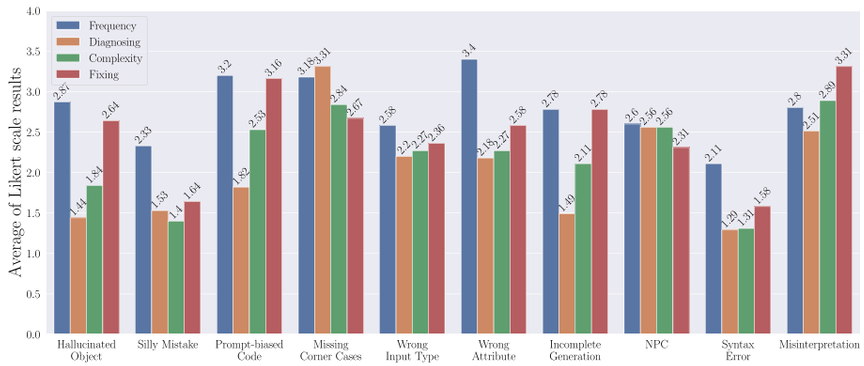


图5 验证调查的汇总结果。涉及遇到错误模式的频率、诊断和修复难度以及错误复杂性的问题。1代表从不/容易/简单/低，5代表总是/困难/复杂/高

diagnosing such a bug pattern, what is the perceived complexity of the bug pattern (i.e., is it a trivial mistake a competent developer would not do or does it denotes some complexity), and what is the perceived difficulty to fix such a bug pattern. For each of those metrics, we report the weighted average results giving the proportion of respondents that answered 1/2/3/4/5 given the Likert scale as described in Sect. 3.2.3. For instance, the *Missing Corner Cases* bug pattern has a score of **3.18** and is among the most frequent bug patterns encountered by participants. All categories in the taxonomy have been encountered and acknowledged by the participants. The most encountered categories are *Wrong Attributes*, *Prompt-biased Code*, or *Missing Corner Cases* with an aggregated score above 3. Thus, participants report that they often encounter bugs from these categories. One participant reported that they observed *Prompt-biased Code* bugs when asking LLMs to generate code using a prompt with several examples of unit tests: "*I face this problem when docstring given too many examples, and the prompt requires LLMs to pass at least all the test cases*". The categories of bugs that were encountered the least are *Syntax Error* and *Silly Mistake* with scores respectively of **2.11** and **2.33**. Thus, most participants reported that they never or rarely encountered these two categories of bugs. This distribution is in line with the results obtained in RQ1, for Codex (one of the most popular LLMs) using the CoderEval dataset: *Syntax Error*, *Silly Mistake*, and *Wrong Input Type* are among the least frequent bug categories in Codex-generated code (see Table 2). Moreover, the *Missing Corner Case* is the most frequent bug category in code generated using Codex. To confirm this observation, we conducted a correlation analysis as described in Sect. 3.2.2 between the frequency of occurrence of bug patterns reported by respondents and the frequency of bug patterns observed in our sample set. The highest correlation is obtained for Codex when using the human-label prompt (ρ **= 0.75**, strong correlation) which seems plausible as we have seen at the beginning of this section that a large majority of the respondents worked using ChatGPT and/or Codex. When using the raw docstring-based prompt, the correlation is ρ = **0.39**. For other LLMs, the correlations are for PanGu and CodeGen **0.64** and **-0.13** respectively on human-labeled prompts and **0.30** and **-0.32** on the raw docstring-based prompts. As such, the frequency of the bug patterns reported by the respondents is similar to the distribution of the bug patterns sampled using human-labeled prompts and the Codex model in our sample set.

We then set out to compare the frequency between academic and industrial practitioners. Figure 6 presents the results. Concerning the frequency of bug patterns encountered, we used a Fisher-exact test between the academic and industry practitioner groups for each of our bug patterns. That is, we compared how participants between each group replied in terms of frequency (1 to 5) for a given bug pattern. In all cases, we did not find a statistical significance (p-value $>$ 0.05) between two groups. As such, the frequencies of encounters between participants' groups can not be said to be significantly different. However, we do note some small discrepancies between the two groups: in general, academic practitioners reported being more likely to encounter our bug patterns than industry respondents. The only categories where it was not the case are Prompt-biased and Missing Corner Cases. We can explain this difference possibly due to several factors: First, based on our results, academic respondents tended to use open-source LLM more often than industry respondents where ChatGPT and Copilot were more prevalent. As open-source LLM tends to be less performant than commercial ones, this could explain why certain categories are observed more by academic practitioners than industrial. For instance, we saw that CodeGen (open-source) was more likely than Codex to exhibit an *Incomplete Generation* bug pattern, and industry practitioners reported seeing less of this pattern than academic practitioners. We can potentially confirm it by looking at the correlations between bug patterns as reported in our labeling and the frequency of both academic and industry practitioners: the correla-

诊断这种错误模式时，该错误模式的感知复杂度是多少（即，它是一个有能力的开发人员不会犯的微不足道的错误，还是表明某种复杂性），以及修复这种错误模式的感知难度是多少。对于每个指标，我们报告了加权平均结果，给出按照第3.2.3节描述的李克特量表回答1/2/3/4/5的受访者比例。例如，缺失边界情况（Missing Corner Cases）错误模式得分为3.18，是参与者遇到的最常见错误模式之一。分类法中的所有类别都已被参与者遇到并认可。最常遇到的类别是错误属性（Wrong Attributes）、提示偏倚代码（Prompt-biased Code）或缺失边界情况（Missing Corner Cases），其综合得分都超过3。因此，参与者报告他们经常遇到这些类别的错误。一个参与者报告称，当使用带有多个指令的提示让大型语言模型生成代码时，他们观察到了提示偏倚代码（Prompt-biased Code）错误。

然后，我们着手比较学术界和工业界从业者之间的频率。图6展示了结果。关于遇到缺陷模式的频率，我们对学术组和工业实践者组的每一种缺陷模式使用了Fisher精确检验。也就是说，我们比较了每个组的参与者在给定缺陷模式下的频率（1到5）的回答。在所有情况下，我们都未发现两组之间存在统计学显著性差异（p值 $>$ 0.05）。因此，不能说参与者组之间的遇到频率有显著差异。然而，我们确实注意到两组之间存在一些小差异：总体上，学术从业者报告更有可能遇到我们的缺陷模式，而工业受访者则不然。唯一不是这种情况的类别是'提示偏倚'和'缺失的边角情况'。我们可能可以通过几个因素解释这一差异：首先，根据我们的结果，学术受访者

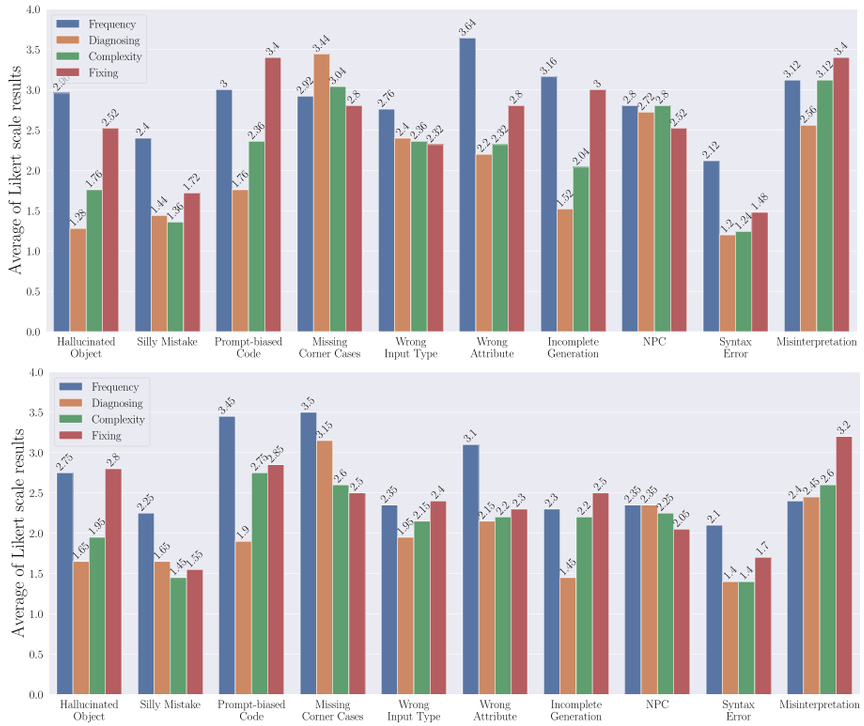


Fig. 6 Aggregated results of the validation survey per affiliation of the practitioners: (**Top**) Academic, (**Bottom**) Industry. Questions related to the frequency of encounter of bug patterns, the difficulty to diagnose and fix them as well as the complexity of the bug. 1 represents never/easy/trivial/low and 5 represents always/hard/-complex/high

tion (on human-labeled prompt samples) is **0.83** for industry practitioners against **0.54** for academic practitioners when checking the distribution of Codex. Moreover, the correlations between CodeGen bug distribution and industrial practitioners are negative for both types of prompts, showing that industrial practitioners reported an opposing bug patterns trend to what we observed with CodeGen in our data. CodeGen generates a lot of incomplete code, demonstrating its limited capability for code generation tasks. Secondly, as industry and academic practitioners leverage LLM for different tasks and aims, some bug patterns may be less observed because of that: for instance, one respondent (industry) declared about *Prompt-biased Code* that: "*this is common, especially if you are not specific that the solution you need is a general one and not only for that specific conditions.*"

4.2.4 Diagnosing, Complexity, and Fixing of Bug Patterns

Figure 5 summarizes the obtained results in the "Diagnosing", "Complexity" and "Fixing" columns which were defined to participants (see Sect. 3.2.2). In terms of diagnosing the bugs, most bug patterns were reported by the respondents as easy to diagnose (score < 2.5),

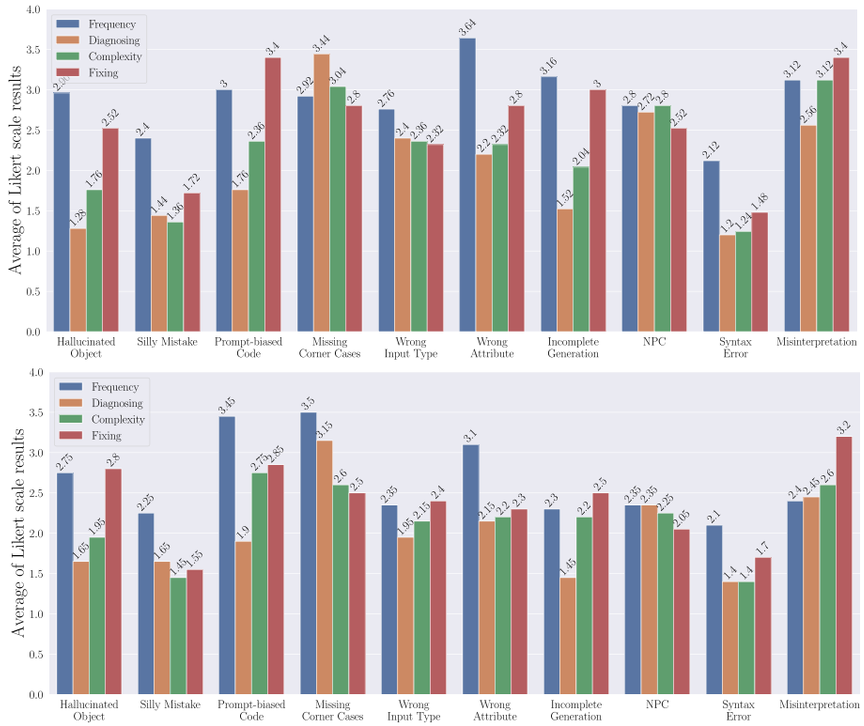


图6 按从业者隶属机构汇总的验证调查结果：（上）学术界，（下）工业界。问题涉及遇到错误模式的频率、诊断和修复它们的难度以及错误的复杂性。1表示从不/容易/简单/低，5表示总是/困难/-复杂/高

在检查 Codex 的分布时，人类标注提示样本上的 индустрии 从业者的相关系数为 0.83，而学术从业者为 0.54。此外，CodeGen 的漏洞分布与工业从业者之间的相关性在两类提示下都是负的，这表明工业从业者报告的漏洞模式趋势与我们在数据中观察到的 CodeGen 情况相反。CodeGen 生成了大量不完整的代码，显示了其在代码生成任务上的有限能力。其次，由于工业和学术从业者利用大语言模型（LLM）的任务和目标不同，一些漏洞模式可能因此观察得较少：例如，一位受访者（工业）关于提示偏向的 Code 说："这是常见的，特别是如果你没有明确说明你需要的是通用解决方案而不仅仅是针对特定条件的解决方案。"

4.2.4 缺陷模式的诊断、复杂性与修复

图 5 总结了在"诊断"、"复杂性"和"修复"列中获得的结果，这些列是为参与者定义的（见第 3.2.2 节）。在错误诊断方面，大多数错误模式被受访者报告为易于诊断（得分 < 2.5）。

especially as several respondents pointed out that: [the bugs are] "*Easy to diagnose when using an IDE with real-time code checking*". According to our survey's respondents, the only bug patterns that are not as easy to diagnose are *Misinterpretation*, *Non-Prompted Consideration*, and *Missing Corner Cases*. In the case of *Non-Prompted Consideration*, a respondent highlighted that: "*I have observed some difficult cases that non-requested behaviour is in the middle of the generated code*". *Missing Corner Cases* is the only category reported as being quite hard to diagnose by respondents (score $>$ 3). This result could be expected since the *Missing Corner Cases* category encompasses code that will compile correctly, but break unit tests, as highlighted by one of the participants: "*It is a risk to become complacent to LLM code generation because sometimes code looks reasonable (in this case). Still, you might be unaware of all the ways it could fail*". Another respondent emphasized the importance of efficient test cases when addressing bugs from the *Missing Corner Cases* category, nothing that: "*It is very dependent on the evaluation test case set. From its short natural language description, like docstring, it's hard to cover all its possible corner cases*". When comparing the difference between industry and academic respondents when diagnosing (see Fig. 6), both groups tend to report a similar level of difficulty. We note a major difference for the Wrong Input Type where academic respondents tend to spend a longer time to diagnose such bug patterns. It possibly takes less time for industry practitioners as they reported using more typed language than academic practitioners did. This could make diagnosing such a bug pattern easier ("*Usually not an issue, really easy to fix and spot in statically typed languages*").

Regarding how likely a developer is to make such a mistake (i.e., "complexity" of the bug patterns), respondents mostly consider the identified bug patterns to have low complexity, with *Syntax Error* and *Silly Mistake* types being the least complex. This adds to our intuition that some bug patterns in LLM-generated code may be different from human-made bugs. Indeed, as most developers use IDE, *Syntax Error*s or similar mistakes are pretty easy to catch. Similarly *Silly Mistake*s, such as the example presented in Listing 4, are not a bug that developers would likely do, and, even if done, they are easy to detect. On the contrary, *Misinterpretation*s and *Missing Corner Cases* bug patterns are more complex and more similar to a type of error that a developer would make. This is highlighted by the respondents, as both categories have the highest aggregated score. One survey participant commented that: "*For example, I would very much forget about IPv6 (don't we all?:)) and miss that corner case myself. So it is both my lack of knowledge and implicitly trusting the LLM output that can make this such an insidious bug to diagnose and solve*". Thus, the complexity of such bugs could arise from both the nature of the error and developers' excessive reliance on LLMs, which amplifies the impact and intricacy of the issue. When comparing the differences in how industry and academic respondents perceive complexity (see Fig. 6), both groups tend to make similar judgments overall. However, we observed two notable distinctions: one with NPC and the other with Misinterpretation, where academic respondents consistently rated the bug patterns as more complex. This disparity might be attributed to industry respondents likely having more practical experience addressing such issues or perceiving them as less problematic.

Regarding the difficulty of fixing the bugs from the different categories, the survey participants consider the bugs to be moderately difficult to fix; mirroring their responses in the first part of the survey, when we asked about the difficulty of fixing LLM-generated bugs. Only *Syntax Error*s and *Silly Mistake*s are considered easy to fix with an aggregated score of $<$ 2. Bugs from the *Misinterpretation* category are reported to be more difficult to fix than the other categories by the survey's participants. This result is consistent with the observation

尤其是正如几位受访者指出的： [这些漏洞] "在使用具有实时代码检查功能的IDE时容易诊断"。根据我们调查的受访者，唯一不那么容易诊断的漏洞模式是误解、非提示性考虑和缺失的边缘情况。在非提示性考虑的情况下，一位受访者指出："我观察到一些困难的情况，非请求的行为出现在生成代码的中间部分"。缺失的边缘情况是唯一被受访者报告为相当难以诊断的类别（得分 $>$ 3）。这一结果在预期之内，因为缺失的边缘情况类别包含那些可以正确编译但会破坏单元测试的代码，正如一位参与者强调的："对LLM代码生成变得自满是有风险的，因为有时代码看起来合理（在这种情况下），但你可能不知道它可能失败的所有方式"。另一位受访者强调了重要性

关于开发者犯这种错误的可能性（即 bug 模式的"复杂性"），受访者大多认为已识别的 bug 模式复杂性较低，其中语法错误和愚蠢错误类型的复杂性最低。这增加了我们的直觉，即 LLM 生成代码中的某些 bug 模式可能与人类产生的错误不同。实际上，由于大多数开发者使用 IDE，语法错误或类似错误相当容易发现。同样，像列表 4 中展示的示例这样的愚蠢错误，并不是开发者很可能犯的错误，即使犯了，也很容易被检测到。相反，误解和缺失边界情况的 bug 模式更复杂，更类似于开发者可能会犯的错误类型。这一点在受访者中得到了突出体现，因为这两类的综合评分最高。一位调查参与者评论道："例如，我很可能会忘记 IPv6（我们不是都这样吗？:））并且会遗漏那个边界情况"

关于修复不同类别错误的难度，调查参与者认为这些错误的修复难度属于中等；这与他们在调查第一部分中关于修复大语言模型生成的错误难度的回答一致。只有语法错误和愚蠢的错误被认为容易修复，其综合得分为 $<$ 2。调查参与者报告称，误解类别的错误比其他类别的错误更难修复。这一结果与观察结果相一致。

we made during the labeling phase. *Misinterpretation* bugs generally lead to a code that is way different compared to the prompt specification. In that case, re-prompting or extensive manual effort is needed to fix the generated code fragment. For some bug categories, like *Hallucinated Objects*, participants suggested that providing additional information in the prompt can often help the LLMs to correct the mistake: "*Most often time after providing a header or function definition, the model can correctly create the method or function*". However, in several cases, especially for bugs in the *Misinterpretation* category, it is not enough to provide additional information: "*It depends on whether you know what you are expecting the code to look like. If so, this is easy to spot, and probably you're better off manually writing the code. I have tried to prompt-engineer, but I find that it's a waste of my time. Sometimes, you will get code generated that misinterprets the prompt, but maybe you are not aware of what the generated solution looks like so you have to read the generated code intently or figure out the library calls that are generated to understand that it misunderstood the prompt*". When comparing the differences between industry and academic respondents in fixing the bug patterns (see Fig. 6), we observed some notable distinctions between the two groups on bug patterns such as Prompt-biased code or NPC. Academic respondents generally reported greater difficulty in addressing these bug patterns, which could be attributed to differences in experience levels and problem-solving approaches. For instance, many industry respondents indicated that, rather than attempting to fix the code manually, they would simply regenerate a solution or adjust the prompt. For example, one respondent commented on the Prompt-biased bug pattern: "*this is a case where I would usually re-prompt the LLM as opposed to trying to fix it myself, as it's clear that it just took the docstring instructions too literally.*"

Finally, as we reported in Sect. 4.1, some LLMs like Codex (and so Copilot), tend to generate additional code that is not explicitly requested in the task's description. This may be linked to the different settings of the LLM: an IDE plugin for Copilot/Codex or a web browser/standalone for CodeGen and PanGu-Coder. This was echoed by one of the respondents of our validation survey who also observed this kind of behavior being more prevalent in Copilot-based models compared to the others: "*There is a difference in LLM-generated code quality between using an in-IDE plugin such as copilot, and using a browser-based tool such as chat gpt or google bard. I find that copilot has more of a tendency to recommend code that I don't need, or code that is redundant; [...]*".

Findings 3: The surveyed practitioners encountered the different bug patterns of our empirical taxonomy. In particular, the frequency of bug patterns reported by practitioners correlates with the frequency observed for bug patterns generated by *Codex* using the human-labeled prompts in our sample set. Practitioners reported that bug patterns are generally easy to diagnose and fix but requires manual effort. *Misinterpretation* and *Missing Corner Case* bug patterns are the only exceptions: being harder to fix and diagnose.

5 Lessons Learned

This study shed light on the nature of bugs occurring in code generated using LLMs. We observed that bugs contained in LLMs-generated code seem to differ from bugs in human-

我们在标注阶段发现的。误解性错误通常会导致生成的代码与提示规范差异很大。在这种情况下，需要重新提示或大量手动操作来修复生成的代码片段。对于某些错误类别，比如“幻觉对象”，参与者建议在提示中提供额外信息通常可以帮助大语言模型纠正错误：“通常在提供头文件或函数定义后，模型可以正确地创建方法或函数”。然而，在一些情况下，特别是对于误解类别的错误，仅提供额外信息是不够的：“这取决于你是否知道希望代码是什么样子。如果知道，这很容易发现，可能你最好手动编写代码。我尝试过提示工程，但发现这浪费了我的时间。有时，你会得到生成的代码误解了提示，但也许你会……

最后，正如我们在第4.1节中报道的，一些大语言模型（如Codex（以及Copilot））倾向于生成在任务描述中未明确要求的额外代码。这可能与大语言模型的不同设置有关：Copilot/Codex使用IDE插件，而CodeGen和盘古Coder使用网页浏览器或独立应用程序。我们的验证调查中的一位受访者也表达了类似看法，他观察到这种行为在基于Copilot的模型中比其他模型更普遍：“在使用像Copilot这样的IDE插件与使用基于浏览器的工具如ChatGPT或Google Bard之间，LLM生成的代码质量存在差异。我发现Copilot更倾向于推荐我不需要的代码，或者冗余的代码；*[…]*”。

发现 3：被调查的从业者遇到了我们经验分类法中的不同缺陷模式。特别是，从业者报告的缺陷模式频率与在我们的样本集中使用人工标注提示生成的 Codex 缺陷模式的观察频率相关。从业者报告称，缺陷模式通常容易诊断和修复，但需要人工努力。唯一的例外是误解和遗漏边缘情况的缺陷模式：它们更难修复和诊断。

五个教训

这项研究揭示了使用大型语言模型生成的代码中出现的错误的性质。我们观察到，大型语言模型生成的代码中的错误似乎与人类生成的代码中的错误有所不同。

written code. Previous studies (Moradi Dakhel et al. 2023) highlighting differences in bug repair cost-such as repair batch sizes and average time to fix when comparing human-written code with LLM-generated code motivated our work to develop a taxonomy of bugs specific to the code generated by LLMs. While this study does not directly compare bugs in human-written and LLM-generated code, our findings uncover bug types that appear to be uncommon in human-written code. In SE, various studies have focused on developing bug taxonomies for human-generated code, either across multiple programming languages (Catolino et al. 2019) or for specific languages or domains (Le Goues et al. 2015; Pan et al. 2009; Humbatova et al. 2020). For example, one study (Tan et al. 2014) on open-source software projects categorized bugs based on their root causes into three main categories: "memory", "concurrency", and "semantic" bugs. The "semantic bugs" category in this study is the most relevant to the LLM-generated bug patterns in our taxonomy. This category includes subcategories like "missing features/cases", which is similar to our "Missing Corner Cases" pattern, "Typo", comparable to our "Syntax Error", and "Other wrong functionality implementation", which aligns with our "Misinterpretation" category. Another study, BugJS (Gyimesi et al. 2021), focused on categorizing bug patterns in JavaScript code, and developed four general categories of bugs in JavaScript code with multiple subcategories. Within these categories, bug types labeled "Incomplete feature generation" and "Incorrect feature generation" partially intersect with our "Misinterpretation" and "Missing Corner Cases" patterns, respectively.

While some intersections exist between our proposed taxonomy and previous bug taxonomies for human-written code, these taxonomies do not capture the unique characteristics of some of the LLM-generated bugs, which often arise from the stochastic and generative nature of LLMs. For instance, categories like "Hallucinated Object", "Incomplete Generation", and "Syntax Error" are detectable by IDEs and have not been reported in human-generated code. LLM-specific bug types such as "Prompt-biased" and "Hallucinated Objects" reflect the distinct behaviour of LLMs, such as generating non-existent objects, and "Incomplete Generation", which may stem from the model's limited context length. Another category, "Silly Mistake", includes buggy patterns that may not necessarily trigger an error but are "silly" in nature, such as redundant repetitions of 'if/else' conditions, which seem rarely found in human code. Additionally, our observations on "Wrong Attribute" errors show that the incorrect attributes generated by LLMs often do not correspond to any existing attributes of the objects. In many cases, these names were influenced by keywords in the prompt, a situation uncommon in human code, as developers typically rely on library documentation or IDE for attribute calls to prevent accidentally calling a name as an attribute that is not valid.

Thus, our proposed taxonomy of LLM-generated bugs extends available taxonomies by highlighting the bug patterns inherited from the generative and probabilistic aspects unique to LLMs. While previous taxonomies can be applied to LLM-generated buggy code to some extent, our taxonomy enriches these frameworks with categories specific to LLMs. Although we did not directly compare human-written and LLM-generated buggy code, our results point towards meaningful differences in the nature and frequency of bugs between the two, opening the way for future studies to perform a more controlled comparison.

In light of our manual labeling and validation surveys, we provide several lessons learned for both LLMs' users and researchers in the following.

书写的代码。先前的研究（Moradi Dakhel 等，2023）强调了在修复错误成本上的差异——例如比较人工编写代码与大语言模型（LLM）生成代码时的修复批量大小和平均修复时间——这促使我们的工作开发了针对 LLM 生成代码的错误分类法。尽管本研究没有直接比较人工编写代码与 LLM 生成代码中的错误，但我们的发现揭示了一些在人工编写代码中似乎不常见的错误类型。在软件工程（SE）领域，各种研究集中于为人工生成的代码开发错误分类法，可以跨多种编程语言（Catolino 等，2019）或者针对特定语言或领域（Le Goues 等，2015；Pan 等，2009；Humbatova 等，2020）。例如，一项关于开源软件项目的研究（Tan 等，2014）根据错误的根本原因将错误分为三大类："内存"、"并发"和"语义"错误。本研究中的"语义错误"类别与 LLM 最相关。

虽然我们提出的分类法与以往针对人工编写代码的缺陷分类法存在一些交集，但这些分类法并未捕捉到某些大语言模型（LLM）生成的缺陷的独特特征，这些缺陷通常源于LLM的随机性和生成性。例如，"幻觉对象"、"生成不完整"和"语法错误"等类别可以被IDE检测到，但在人类生成的代码中未被报告。LLM特有的缺陷类型如"提示偏差"和"幻觉对象"反映了LLM的独特行为，比如生成不存在的对象，而"生成不完整"可能源于模型的上下文长度有限。另一个类别"愚蠢错误"包括可能不会触发错误但本质上"愚蠢"的缺陷模式，例如'if/else'条件的冗余重复，这在人工代码中似乎很少出现。此外，我们对"属性错误"的观察显示，不正确的属性

因此，我们提出的LLM生成的漏洞分类法通过突出生成功能模型独有的生成性和概率性方面继承的漏洞模式，扩展了现有的分类法。虽然之前的分类法在一定程度上可以应用于LLM生成的有漏洞代码中，但我们的分类法通过加入LLM特有的类别丰富了这些框架。尽管我们没有直接比较人类编写和LLM生成的有漏洞代码，但我们的结果显示两者在漏洞的性质和频率上存在有意义的差异，为未来研究进行更有控制的比较打开了道路。

根据我们的手动标注和验证调查，我们在以下内容中为大语言模型的用户和研究人员提供了若干经验教训。

5.1 Lessons Learned for LLM Users

(Lack of) Information in the prompt can drastically alter model output

CoderEval's tasks comprise a function signature with a docstring. While this might be enough to some extent for a human, LLMs seem to struggle with this somewhat limited information. One issue we observed in our sample set concerns the library or even the Python version. Indeed, there is little control over which version of a given programming language or library the LLMs will use in the generated code if it is not indicated explicitly in the prompt. We came across examples of generated code that would only work for a certain version of Python while the prompt and the oracle code were pointing to another version. For instance, when parsing XML files, the LLMs would sometimes use the function "getchildren()" which is deprecated for certain versions of Python. This is something that some respondents in our survey reported: ("*Sometimes, the LLM would generate functions that are deprecated. It is not necessarily hallucinating, it was just trained on an older version of the Python package you are using*"). Interestingly, by looking at other samples of the same task not necessarily in our sample set, we found that the usage of this function by LLMs was not systematic and some samples were correctly using the expected (more recent) function. Hence, having an older version in the generated code can not only be explained by deprecated training data. Similarly, on tasks requiring a different coding between Python 2 and 3, the LLMs would often only consider one of them (e.g., using the API for Python 3 for both the Python 2 and Python 3 cases). Moreover, given the difference we observed between bug patterns in code generated using human-labeled prompts and docstring-based prompts, it shows also that *how* the prompt is formulated can have an impact on how over the bug patterns a given generated code will contain.

Recommendation: Re-prompting or adding additional information (for instance Chain Of Thoughts Wei et al. 2022; Li et al. 2023 or examples of test cases Liu et al. 2024) can help the LLM in the code generation process. This is noted by a respondent of our survey: "*Most often time after providing a header or function definition, the model can correctly create the method or function*". For instance, in the cases we observed in our sample set, with the mismatch between the desired version of the library by the user and the obtained version in the LLM's code, providing the concrete version of the libraries can help. By doing so in the prompt, the user can constrain the LLM to generate accordingly. For instance, Guilherme and Vincenzi (2023) generated Java test cases for a particular Java/JUnit version by explicitly mentioning it in the prompt.

More testing, less trusting Even when given enough information, LLMs might generate faulty codes. In that case, having access to test cases to properly test the generated code is important. This however can be quite a manual effort. In a regression test setting, using automated test case generation tools such as EvoSuite (Fraser and Arcuri 2014) can help. Leveraging LLMs themselves was also shown to be a potential way to generate unit tests (Liu et al. 2023; Tang et al. 2023). Nonetheless, one should be cautious even when test cases are used. For instance, in the case of *Misinterpretation* or *Missing Corner Case*, the error might not be obvious, and potential existing test cases might not catch the error in the LLMs' generated code. For instance, Liu et al. (2023) showed that existing test cases in HumanEval

5.1 LLM 用户的经验教训

提示中的（缺乏）信息可能会极大地改变模型输出

CoderEval 的任务包含一个带有文档字符串的函数签名。虽然对于人类来说这在某种程度上可能已经足够，但大型语言模型（LLMs）似乎在处理这些有限信息时有些困难。我们在样本集中观察到的一个问题涉及库甚至 Python 版本。事实上，如果在提示中没有明确指出，LLMs 几乎无法控制生成代码时使用的编程语言或库的版本。我们遇到过生成代码仅适用于某个特定 Python 版本的例子，而提示和标准答案代码指向的是另一个版本。例如，在解析 XML 文件时，LLMs 有时会使用"getchildren()"函数，而该函数在某些 Python 版本中已被弃用。这也是我们的一些调查受访者提到的情况：（"有时，LLM 会生成已被弃用的函数。这不一定是幻觉，它只是基于 Python 库的旧版本进行训练。"

建议：重新提示或添加额外信息（例如 Chain Of Thoughts Wei 等人 2022；Li 等人 2023 或测试用例示例 Liu 等人 2024）可以帮助大语言模型（LLM）在代码生成过程中。这一点在我们调查的受访者中有所体现："大多数情况下，在提供了头文件或函数定义之后，模型可以正确地创建方法或函数。"例如，在我们观察的样本集中，当用户期望的库版本与 LLM 生成代码中获得的版本不匹配时，提供具体的库版本会有所帮助。通过在提示中做到这一点，用户可以约束 LLM 按照指定生成代码。例如，Guilherme 和 Vincenzi （2023）通过在提示中明确提及特定的 Java/JUnit 版本来生成 Java 测试用例。

更多测试，少一些信任 即使提供了足够的信息，大型语言模型（LLM）也可能生成有缺陷的代码。在这种情况下，能够访问测试用例以正确测试生成的代码是很重要的。然而，这可能需要相当多的人工工作。在回归测试环境中，使用自动化测试用例生成工具如 EvoSuite（Fraser 和 Arcuri 2014）可以提供帮助。利用 LLM 本身也被证明是一种生成单元测试的潜在方法（Liu 等，2023；Tang 等，2023）。尽管如此，即使使用测试用例，也应保持谨慎。例如，在误解或遗漏边界情况的情况下，错误可能并不明显，而现有测试用例可能无法捕捉 LLM 生成代码中的错误。例如，Liu 等（2023）显示了 HumanEval 中的现有测试用例

dataset (Chen et al. 2021), which encompasses simpler tasks compared to CoderEval, would not catch all existing mistakes in LLMs' generated code. Worse, if no tests are available, users either have to trust the LLMs' generated code or manually read and analyze the obtained code to make sure there are no errors. The first option introduces a high risk depending on the context in which the code should be used while the second one adds a tedious and time-consuming task for the developers, proportional to the code complexity. With the previous points mentioned, it is obvious that LLM-generated code should not be too trusted as it can easily be error-prone. If more experienced developers are less likely to fall into this trap, novice users or people unfamiliar with the particular task requested in the prompt can easily trust generated code, especially if it "looks" correct, as highlighted in one comment from our survey participants: "*It is a risk to become complacent to LLM code generation, because sometimes code looks reasonable (in this case), but you might be unaware of all the ways it could fail*". Some previous studies also pointed out that more novice users tend to over-rely on the code (Scoccia 2023; Kazemitabaar et al. 2023; Moradi Dakhel et al. 2023) which can have serious negative effects.

Recommendation: When possible, users should make use of test cases. Both in the prompt to improve the generation process of the LLMs (Liu et al. 2024), as well as to check the generated code for potential errors. In any case, users (especially novice users) should be mindful when reusing LLMs generated code by trusting the solution is correct. One could draw an analogy with StackOverflow where several studies (Fischer et al. 2017; Zhang et al. 2018; Verdi et al. 2022) showed that users tend to rely on those forums while the code present here might not be always reliable.

Knowing how and when to use LLMs In some cases, if too many details are needed for the LLMs to generate a proper code, it might be more straightforward to write the function directly. For instance, Vaithilingam et al. (2022) showed that users leveraging Copilot for coding tasks were not necessarily faster than users without Copilot for the reasons we observed in our study: the code generated might be buggy, oftentimes not in a straightforward way (*Misinterpretation*/*Missing Corner Case*), or the users might have a hard time understanding the logic of the generated code. This is best put by one of our survey's respondents: "*It depends on whether you know what you are expecting the code to look like. If so, this is easy to spot, and probably you're better off manually writing the code. I have tried to prompt and engineer, but I find that it's a waste of my time. Sometimes, you will get code generated that misinterprets the prompt, but maybe you are not aware of what the generated solution looks like so you have to read the generated code intently or figure out the library calls that are generated to understand that it misunderstood the prompt*". Moreover, straight code generation might not be recommended in all cases, such as we analyzed in our study. An alternative would be to relax the problem to instead do *code completion* as commented by a survey's participant: "*In most of my usage scenarios, I generate code with LLM based on existing code. It could be different from letting LLMs generate code from scratch.*". In that case, the user can start coding the functionality and then use the LLM to complete some lines to lower the workload. The additional context would help the model in making more accurate and less error-prone generations. Even in the case of the generated code containing an error, it will likely be easier for a developer to debug as part of the logic was implemented by said developer and the generated part should also follow a similar logic. This, however, requires some context to work with, which might hamper the usefulness of the method. Another way to

数据集 (Chen 等, 2021) 包含的任务相比 CoderEval 更简单，因此无法捕捉 LLM 生成代码中的所有现有错误。更糟糕的是，如果没有可用的测试，用户要么不得不信任 LLM 生成的代码，要么手动阅读和分析获得的代码以确保没有错误。第一种选择根据代码使用的具体场景会带来较高风险，而第二种选择则会增加开发者的繁琐且耗时的任务，其时间消耗与代码复杂性成正比。基于以上几点，很明显不应该过度信任 LLM 生成的代码，因为这些代码很容易出错。如果经验更丰富的开发者不太可能落入这一陷阱，那么对于新手用户或对提示中请求的特定任务不熟悉的人来说，尤其是当代码“看起来”正确时，很容易就会信任生成的代码，正如我们调查参与者的一条评论所强调的：“对 LLM 代码生成变得自满是一种风险。”

建议：在可能的情况下，用户应使用测试用例。这既可以在提示中使用，以改进大型语言模型（LLM）的生成过程（Liu 等，2024），也可以用于检查生成的代码是否存在潜在错误。在任何情况下，用户（尤其是新手用户）在重复使用 LLM 生成的代码时，都应注意不要盲目信任其解决方案是正确的。这可以类比于 StackOverflow，多项研究（Fischer 等，2017；Zhang 等，2018；Verdi 等，2022）显示，用户倾向于依赖这些论坛，但其上的代码并不总是可靠的。

了解如何以及何时使用大型语言模型（LLMs）在某些情况下，如果生成适当代码需要的细节过多，直接编写函数可能会更直接。例如，Vaithilingam 等人（2022）表明，利用 Copilot 进行编码任务的用户不一定比没有 Copilot 的用户更快，原因与我们研究中观察到的类似：生成的代码可能存在漏洞，且经常不是直接可用的（误解/遗漏的边缘情况），或者用户可能难以理解生成代码的逻辑。正如我们的一位调查受访者所言：“这取决于你是否知道自己期望代码是什么样的。如果知道，这很容易识别，可能你手动写代码更好。我尝试过提示和设计，但发现这是浪费时间。有时，你会得到与提示不符的生成代码，但可能你并没有意识到生成的解决方案l

mitigate full code generation would be to use LLMs to kick-start coding, saving time on more redundant or simple parts. This has the advantage of not necessarily needing code context and letting the users have more control over the code. This was another point mentioned in the study of Vaithilingam et al. (2022): even when they did not observe a significant time advantage while using LLMs, developers still preferred them because they could kick-start the coding process and avoid searching for solutions on the Internet.

Recommendation: On one hand, if the user has a precise idea of what needs to be done, the advantage of generating the code brought by the LLMs might not offset the time it takes to correctly prompt the LLMs and to debug/understand the generated code by the user. In that case, manual coding can be more recommended. Alternatively, code completion can be used, as the additional code context provided by the users can both guide the LLMs and lead to a code that is more understandable by the users. On the other hand, if the user has no idea how to tackle the task, LLMs can serve as a code kickstart.

5.2 Lessons Learned for Researchers

Following our results and analysis, we provide additional lessons learned that could be future potential research venues for the community regarding bugs in the code generated by LLMs:

Repairing the code generated by LLMs One way to mitigate bugs in LLM-generated code would be to directly repair them, ideally before the code is returned to the user. Automatic Program Repair (APR) tools (Monperrus et al. 2018) have been used to fix traditional software programs and several studies (Li et al. 2023; Moon et al. 2023) started applying them to code generated by LLMs. These studies, however, made use of simpler datasets (e.g., HumanEval Chen et al. 2021 or LeetCode Leetcode contest 2023, which are not based on programming tasks extracted from real-world projects) and still failed to find several mistakes. Hence, we recommend analyzing existing bugs in practical tasks similarly to what we did, which could be beneficial to guide such repair. This could help improve existing APR tools for LLMs generated code or complement prompt for LLMs to self-check generated code against identified bug patterns.

Proposing code features related benchmarks for testing code LLMs We observed that LLMs-generated code can contain different bug patterns and that, depending on the code dependency level, LLMs might be more prone to certain errors. Thus, creating standardized benchmark datasets and evaluation metrics specifically tailored for assessing code-related features, the performance of bug detection and triaging methods for LLM-generated code. This will enable fair comparison and evaluation of different testing approaches. Existing benchmarks already focusing on code repair such as CodeXGlue (Lu et al. 2021) or HumanEvalPack (Muennighoff et al. 2023), on dependency level such as CoderEval (Yu et al. 2023) or ClassEval (Du et al. 2023) or even on code efficiency such as EffiBench (Huang et al. 2024) are important to be considered. Hence, promoting new benchmarks or complementing the above benchmark with additional features such as different bug patterns, dependency levels or other code-related features could help in better assessing LLMs shortcomings.

Interpretable AI for code generation Besides potential APR techniques, fixing and diagnosing the bugs in LLMs generated code remain manual tasks. This task can be quite intensive depending on the nature of the bug patterns introduced such as *Missing Corner Case* or *Misin-*

缓解完全代码生成的方法是使用大型语言模型（LLMs）来启动编码，从而在更冗余或简单的部分节省时间。这有一个优点，即不必依赖代码上下文，同时让用户对代码有更多的控制权。这也是Vaithilingam等人（2022年）研究中提到的另一点：即使在使用LLMs时他们没有观察到显著的时间优势，开发者仍然更喜欢使用它们，因为可以启动编码过程，避免在互联网上寻找解决方案。

推荐：一方面，如果用户对需要完成的任务有明确的想法，由大语言模型生成代码所带来的优势可能不足以抵消正确提示大语言模型以及调试/理解生成代码所花费的时间。在这种情况下，手动编码可能更为推荐。或者，可以使用代码补全，因为用户提供的额外代码上下文既可以指导大语言模型，也可以生成用户更易理解的代码。另一方面，如果用户不知道如何处理该任务，大语言模型可以作为代码的启动工具。

5.2 研究人员的经验教训

根据我们的结果和分析，我们提供了额外的经验教训，这些教训可能成为社区关于由大型语言模型生成的代码中的漏洞的未来潜在研究方向：

修复由大语言模型生成的代码 减少大语言模型生成代码中的错误的一种方法是直接修复这些代码，最好在将代码返回给用户之前进行。自动程序修复（APR）工具（Monperrus 等, 2018）已被用于修复传统软件程序，并且一些研究（Li 等, 2023；Moon 等, 2023）开始将其应用于大语言模型生成的代码。然而，这些研究使用了较为简单的数据集（例如 HumanEval Chen 等, 2021 或 LeetCode Leetcode contest 2023，这些数据集并非基于从真实项目中提取的编程任务），仍然未能发现若干错误。因此，我们建议像我们所做的那样分析实际任务中的现有错误，这可能有助于指导此类修复。这可以帮助改进针对大语言模型生成代码的现有 APR 工具，或补充提示，让大语言模型根据已识别的错误模式自检生成的代码。

提出与代码特性相关的基准以测试代码大模型（LLMs） 我们观察到，大模型生成的代码可能包含不同的错误模式，并且根据代码的依赖级别，大模型可能更容易出现某些错误。因此，创建专门用于评估代码相关特性、LLM生成代码的错误检测和分类方法性能的标准化基准数据集和评估指标，这将有助于公平地比较和评估不同的测试方法。现有的基准已经关注代码修复，例如 CodeXGlue（Lu 等，2021）或 HumanEvalPack （Muennighoff 等，2023）、依赖级别例如 CoderEval （Yu 等，2023）或 ClassEval（Du 等，2023），甚至代码效率例如 EffiBench （Huang 等，2024），都是需要考虑的重要内容。因此，推广新的基准或通过增加诸如不同错误模式、依赖级别或其他代码相关特性来补充上述基准是很有必要的。

用于代码生成的可解释人工智能 除了潜在的APR技术之外，修复和诊断大型语言模型生成代码中的漏洞仍然是人工任务。根据引入的漏洞模式的性质，这项任务可能相当繁重，例如缺失的边缘情况或错误信息。

terpretation as we described in our study. As such, employing interpretability techniques for LLMs could help in providing insights into potential fixes or assisting in bug detection. Such techniques analyze how input (i.e., prompt) variations affect a model's output (Li et al. 2024). By monitoring the effect of presence or absence of different combinations of input elements, these approaches facilitate uncovering the relative contribution of each element to the output, enabling model analysis without necessitating access to its internal architecture (Liu et al. 2023). Thus, applying interpretable AI techniques to LLMs to enhance transparency in code generation is also an important research direction. For instance, Ji et al. (2023) analyzed how prompt changes impact code generation.

6 Related Works

Our work focuses on identifying bug patterns in code generated by LLMs based on their types, to support and potentially accelerate bug triaging activities in the software engineering field. Therefore, in this section, we discuss the literature related to bug classification studies, as well as research relevant to bugs in LLM-generated code.

6.1 Bug Patterns

Previous research has employed various methodologies to analyze bugs across different programming languages and software systems and proposed a taxonomy of bugs.

For instance, Catolino et al. (2019) manually labeled bugs by analyzing the titles and summaries of bug reports across various languages, defining a taxonomy of bug patterns in different repositories on GitHub. They investigated the frequency of each bug type, the time required to fix them, and the topics discussed in bug reports, regardless of the programming language. Their analysis resulted in the identification of 9 general bug types, such as "configuration issues", "network issues", "database-related issues", and "security issues", which are common types across different programming languages.

ManyBugs and Introclass benchmarks (Le Goues et al. 2015) consist of different buggy code in C programming language and their corresponding bug patterns. In ManyBugs, the authors categorized 185 defects in C programs based on defect symptoms. This classification was performed by examining developers' annotations, analyzing bug reports, commit comments in the fix versions, modifications in human patches, and associated witness test cases. The defects were categorized into four general groups, including "incorrect output", "segmentation errors", and "fatal errors". Additionally, the Introclass dataset, containing 998 instances of buggy programs from six student assignments in C, was classified based on the error messages from failing test cases, also resulting in similar four categories.

Similarly, Tan et al. (2017) classified bugs by examining the syntactic differences between buggy and patched programs, using this method for its automation potential, which is particularly useful for large datasets where manual inspection is time-consuming. They employed GumTree to extract syntactic differences at the Abstract Syntax Tree (AST) level and proposed defect classes based on mutant operators like "Insert Return" or "Replace Logical Operator" which are required to convert a buggy code into the correct patch. These defect types were based on buggy code in C programs and were later used to classify buggy code generated by Codex in Java programming language (Fan et al. 2023).

正如我们在研究中描述的那样的解释。因此，使用用于大语言模型（LLM）的可解释性技术可以帮助提供潜在修复方案的见解或辅助错误检测。这类技术分析输入（即提示）变化如何影响模型的输出（Li 等，2024）。通过监控不同输入元素组合的存在或缺失的效果，这些方法有助于揭示每个元素对输出的相对贡献，从而在不需要访问其内部架构的情况下对模型进行分析（Liu 等，2023）。因此，将可解释的人工智能技术应用于大语言模型以增强代码生成的透明性也是一个重要的研究方向。例如，Ji 等（2023）分析了提示变化如何影响代码生成。

6 相关工作

我们的工作重点是根据类型识别由大型语言模型生成的代码中的错误模式，以支持并可能加速软件工程领域的错误分类活动。因此，在本节中，我们讨论了与错误分类研究相关的文献，以及与大型语言模型生成代码中的错误相关的研究。

6.1 错误模式

以往的研究采用了各种方法来分析不同编程语言和软件系统中的错误，并提出了一种错误分类法。

例如，Catolino 等人（2019）通过分析各种语言的缺陷报告的标题和摘要，手动标注了缺陷，并定义了 GitHub 上不同代码库中缺陷模式的分类法。他们研究了每种缺陷类型的频率、修复所需时间以及缺陷报告中讨论的主题，而不考虑编程语言。他们的分析结果识别出了 9 种通用缺陷类型，例如"配置问题"、"网络问题"、"数据库相关问题"和"安全问题"，这些都是不同编程语言中常见的缺陷类型。

ManyBugs 和 Introclass 基准（Le Goues 等，2015）由不同的 C 语言有缺陷代码及其相应的缺陷模式组成。在 ManyBugs 中，作者根据缺陷症状对 C 程序中的 185 个缺陷进行了分类。这一分类是通过检查开发者的注释、分析错误报告、修复版本中的提交注释、人为补丁的修改以及相关的见证测试用例来完成的。这些缺陷被分类为四个一般类型，包括"输出不正确"、"分段错误"和"致命错误"。此外，Introclass 数据集包含来自六个 C 语言学生作业的 998 个有缺陷程序实例，依据失败测试用例的错误信息进行分类，同样得出了类似的四类。

同样，Tan 等人（2017）通过检查有缺陷程序和修补程序之间的语法差异来对 bug 进行分类，使用这种方法是因为它具有自动化潜力，这在手动检查耗时的大型数据集中尤其有用。他们使用 GumTree 在抽象语法树（AST）级别提取语法差异，并基于像"插入返回"或"替换逻辑运算符"等变异操作符提出了缺陷类别，这些操作符是将有缺陷代码转换为正确补丁所必需的。这些缺陷类型基于 C 程序中的有缺陷代码，后来被用于分类 Codex 在 Java 编程语言中生成的有缺陷代码（Fan 等人，2023）。

Pan et al. (2009) conducted a similar study focusing on Java programs, where they developed a taxonomy of bug patterns based on the syntax components that changed between the buggy code and corresponding fixes. Their research resulted in the identification of 27 bug fix patterns, including "if-related" and "method-call" patterns which require conversion of the buggy code into a fixed version.

Furthermore, research into Deep Neural Network (DNN) software has shown that it often differs structurally from general-purpose software. To address these differences, a bug taxonomy specific to Deep Learning (DL) frameworks was proposed (Humbatova et al. 2020). In this study, researchers manually studied bug reports in DL projects, examining Python source files from well-known repositories like TensorFlow or Keras. They applied open-card manual labeling to issue reports, pull requests, commit messages, and discussions related to bugs in DL frameworks. They focused on issues relevant to the usage of the DL framework. This resulted in a taxonomy covering bugs related to the structure and properties of DL models, comprising five high-level categories and different subcategories at each level.

Building on these efforts, our proposed taxonomy for buggy code generated by LLMs can be seen as an extension of the traditional bug taxonomy for general-purpose software. These two taxonomies, traditional and LLM-specific, can complement each other in future studies and offer a more comprehensive framework for bug categorization and analysis.

6.2 Bugs in LLM-generated Code

While various studies have incorporated LLMs for diverse programming tasks, our focus is specifically on studies that investigated bugs in LLM-generated code. We also discuss studies proposing LLM-based programming assistant tools and studies that examine the quality of code generated by LLMs.

Vaithilingam et al. (2022) conducted a human study involving 24 participants to investigate the user experience of using Copilot for completing programming tasks. Their study revealed that participants using Copilot had a lower success rate in accurately completing these tasks compared to those using Intellisense.[8] This occurred primarily because the participants struggled to detect and correct the errors contained in the code generated by Copilot. Participants faced difficulties correcting the bugs generated by Copilot, to the point that they preferred writing the code from scratch instead of repairing the bugs contained in Copilot's code.

Moradi Dakhel et al. (2023) assessed the quality of code generated by Copilot for solving different fundamental algorithmic problems such as searching and sorting. Moradi et al. also compared the quality of code generated by Copilot as an AI pair programming assistant for certain programming tasks with the quality of human-written code. They also examined the effort required to repair bugs in code generated by Copilot using an APR tool, comparing it with effort required to fix bugs in human-written code. Their results highlight some differences in the cost of repairing buggy Copilot-generated code compared to those generated by humans. Their results also suggest that this difference could be due to Copilot occasionally overlooking specific details in the prompt.

Mastropaolo et al. (2023) studied the robustness of LLM-based programming assistant tools and investigated the extent to which the input provided to Copilot as a prompt affects the generated output. To conduct this study, they employed a Deep Learning based paraphrasing technique to rephrase task descriptions and assess the correctness of the rephrased descriptions through manual evaluation by the authors. Their results indicate that rephrasing

[8] https://visualstudio.microsoft.com/services/intellicode/

潘等人（2009）进行了一个类似的研究，重点关注 Java 程序，他们基于从错误代码到相应修复代码之间变化的语法组件开发了一个错误模式分类法。他们的研究结果识别出了 27 种错误修复模式，包括需要将错误代码转换为修复版本的"if 相关"和"方法调用"模式。

此外，对深度神经网络（DNN）软件的研究表明，它在结构上通常与通用软件不同。为了解决这些差异，提出了一种针对深度学习（DL）框架的特定缺陷分类法（Humbatova 等, 2020）。在这项研究中，研究人员手动研究了 DL 项目中的缺陷报告，检查了来自知名库如 TensorFlow 或 Keras 的 Python 源文件。他们对与 DL 框架缺陷相关的问题报告、拉取请求、提交信息和讨论应用了开放卡片手动标注方法。他们关注与 DL 框架使用相关的问题。这导致了一种分类法，涵盖了与 DL 模型的结构和属性相关的缺陷，包括五个高层类别以及每个层级的不同子类别。

在这些努力的基础上，我们提出的针对大语言模型生成的有缺陷代码的分类法可以被视为通用软件传统缺陷分类法的延伸。这两种分类法，即传统分类法和特定于大语言模型的分类法，可以在未来的研究中相互补充，并为缺陷分类和分析提供一个更全面的框架。

6.2 LLM生成代码中的错误

虽然各种研究已经将大型语言模型（LLMs）应用于不同的编程任务，但我们的重点特别在于那些研究LLM生成代码中错误的研究。我们还讨论了提出基于LLM的编程助手工具的研究，以及研究LLM生成代码质量的研究。

Vaithilingam 等人（2022）进行了一项涉及 24 名参与者的人体研究，以调查使用 Copilot 完成编程任务的用户体验。他们的研究显示，使用 Copilot 的参与者在准确完成这些任务方面的成功率低于使用 Intellisense 的参与者。这主要是因为参与者难以发现并纠正 Copilot 生成代码中的错误。参与者在纠正 Copilot 生成的 bug 时遇到了困难，以至于他们宁可从头编写代码，也不愿修复 Copilot 代码中的 bug。

Moradi Dakhel 等人（2023）评估了 Copilot 生成的代码在解决不同基础算法问题（如搜索和排序）方面的质量。Moradi 等人还比较了 Copilot 作为 AI 配对编程助手在某些编程任务中生成的代码质量与人类编写代码的质量。他们还研究了使用 APR 工具修复 Copilot 生成代码中漏洞所需的努力，并将其与修复人类编写代码中漏洞所需的努力进行比较。他们的结果突出显示了修复 Copilot 生成的有缺陷代码与人类生成的代码在成本上的一些差异。他们的结果还表明，这种差异可能是由于 Copilot 有时会忽略提示中的特定细节所致。

Mastropaolo 等人（2023）研究了基于大语言模型的编程助手工具的稳健性，并调查了提供给 Copilot 作为提示的输入在多大程度上影响生成的输出。为了进行这项研究，他们采用了一种基于深度学习的改写技术来重述任务描述，并通过作者的人工评估来评估改写描述的正确性。他们的结果表明，改写

[8] https://visualstudio.microsoft.com/services/intellicode/

the task description influences the distribution of solutions generated by Copilot across categories of *failing test*, *passing test*, *syntax error*, and *no solution*. Thus, there is a potential loss in accuracy (moving generated code snippets from *passing test* category to *failing test* or *syntax error*) associated with the description of the prompt.

While the findings of these studies shed light on the bug-proneness of code generated by LLMs, none of them have thoroughly investigated the bug patterns and–or the characteristics of such bugs. Only a handful of studies have delved into the types of bugs observed in the code generated by LLMs, and we will discuss them in the following.

Honarvar et al. (2023) conducted a study to assess the robustness of four different LLMs in addressing what they termed as instances or neighborhood questions. To create a set of neighborhoods for a template question, they replaced variables in 60 different programming tasks with input tests collected from their test oracle and generated different instances for a single programming question. In the evaluation process, rather than just labeling a code as buggy, they assigned a correctness score to indicate the number of instances of a template question successfully handled by the code generated by an LLM. Their findings revealed that there are buggy code snippets generated by LLMs that failed only in a few instances. Subsequently, they categorized the reasons behind failures based on observed error types into classes such as *syntax error*, *runtime error*, *assertion error*, and *wrong number of arguments*.

Fan et al. (2023) aimed to enhance the reliability of code generated by Codex by fixing its generated buggy code. They employed both an APR tool and also leveraged Codex to repair the buggy code. To assess the feasibility of applying APR to repair buggy code from Codex, they initially analyzed common mistakes found in solutions generated by Codex. Two authors manually attempted to fix the bugs in the Codex-generated code, creating fixing patches by referencing other human-provided solutions for the same task on Leetcode (Leetcode contest 2023). The authors categorized buggy snippets using a predefined set of defect categories, derived from a benchmark study on human buggy code for programming competition (Tan et al. 2017). This classification was inspired by the type of fix required for the buggy code, such as *operator mutation* or *variable mutation*. They concluded that the buggy code generated by Codex shared similar mutant operators for fixing as those found in human buggy code. However, their results on repairing buggy code revealed that Codex outperformed APR tools (which are also inspired by human bugs) in fixing its own buggy code, and they recommended enhancements to APR tools to overcome this limitation. Their conclusions are not confirmed by our study. Indeed, while *Misinterpretation* and *Missing Corner Case* bug patterns, which relate more to something human developers would do (as validated by the survey in Sect. 4.2), are still present, multiple non-human like mistakes occur even on a more advanced model like Codex (e.g., *Wrong Attribute*, *Hallucinated Object*). This can be explained by the fact that CoderEval is based on actual programs mined from GitHub and not coding competition snippets such as those extracted from LeetCode contests.

Liu et al. (2023) conducted an analysis of code quality issues in the code generated by ChatGPT for a specific set of LeetCode problems (Leetcode contest 2023). They utilized static analysis tools, such as Pylint for Python, to examine common characteristics of the generated code. They applied open card sorting to discuss and group the ChatGPT-generated code based on their results on test cases and the outcome of static analysis tools. After grouping the code snippets they assigned a name to each group based on common error patterns, leading to four different categories of quality issues in ChatGPT-generated code: *compilation and runtime errors*, *wrong output* (linked to assertion errors), *code style and maintainability*, and *performance and efficiency*. Within each category, they counted error types from the messages

任务描述会影响 Copilot 生成的解决方案在失败测试、通过测试、语法错误和无解决方案类别中的分布。因此，与提示的描述相关，可能会出现准确性损失（将生成的代码片段从通过测试类别移动到失败测试或语法错误）。

虽然这些研究的发现揭示了由大型语言模型生成的代码容易出现错误，但它们都没有彻底调查这些错误的模式和/或这些错误的特征。只有少数研究深入探讨了在大型语言模型生成的代码中观察到的错误类型，我们将在下文中讨论这些研究。

Honarvar 等人（2023）进行了一项研究，以评估四种不同大语言模型（LLM）在处理他们所称的实例或邻域问题时的鲁棒性。为了为一个模板问题创建一组邻域，他们在 60 个不同的编程任务中用从测试oracle收集的输入测试替换变量，并为单个编程问题生成不同的实例。在评估过程中，他们并没有仅仅将代码标记为有错误，而是分配了一个正确性分数，以显示由 LLM 生成的代码成功处理模板问题实例的数量。他们的研究发现，LLM生成的有错误的代码片段有些仅在少数实例中失败。随后，他们根据观察到的错误类型将失败原因分类为语法错误、运行时错误、断言错误以及参数数量错误等类别。

Fan 等人（2023）旨在通过修复 Codex 生成的有缺陷的代码来提高其生成代码的可靠性。他们既使用了 APR 工具，也利用 Codex 来修复有缺陷的代码。为了评估将 APR 应用于修复 Codex 生成的有缺陷代码的可行性，他们最初分析了 Codex 生成的解决方案中常见的错误。两位作者手动尝试修复 Codex 生成代码中的错误，通过参考 Leetcode 上为相同任务提供的其他人类解决方案（Leetcode 竞赛 2023）来创建修复补丁。作者使用预定义的缺陷类别集合对有缺陷的代码片段进行了分类，该集合来源于对编程比赛中的人类有缺陷代码的基准研究（Tan 等人，2017）。这种分类受修复有缺陷代码所需类型的启发，例如操作符变异或变量变异。他们得出结论，Codex 生成的有缺陷代码在修复过程中所需的变异操作符与人类有缺陷代码中发现的类似。然而

刘等人（2023）对 ChatGPT 为特定一组 LeetCode 问题（LeetCode 竞赛 2023）生成的代码中的代码质量问题进行了分析。他们利用静态分析工具，例如针对 Python 的 Pylint，来检查生成代码的常见特征。他们应用开放式卡片分类方法，根据测试用例的结果和静态分析工具的输出，对 ChatGPT 生成的代码进行了讨论和分组。在对代码片段进行分组后，他们根据常见的错误模式为每个组命名，从而得出 ChatGPT 生成代码中的四类质量问题：编译和运行错误、错误输出（与断言错误有关）、代码风格与可维护性以及性能与效率。在每个类别中，他们统计了消息中的错误类型。

raised by static analysis tools and reported the distribution of error types. Their results revealed that code snippets passing all test cases exhibited fewer quality issues, although they were not entirely free of such issues. While we did not study quality issues (as we focused on bugs rather than code quality issues), the prevalence of *Misinterpretation* and *Missing Corner Case* categories in our dataset are in line with the prevalence of *Wrong Output* category in their study. Thus, we reached similar observations in more practical tasks (compared to LeetCode competitions which are simpler). Moreover, their *Compilation and Runtime Error* in the LeetCode tasks match one or more of our categories, for instance, the *Wrong Attribute* or *Wrong Input Type*. Many bug categories found in our study were not found in their study, for example, *Hallucinated Object* or *Non-Prompted Consideration*. This difference is likely due to the nature of the studied tasks: CoderEval tasks are more practical (as they were directly extracted from GitHub) than the programming competition tasks on the LeetCode. This gives way to more complex errors such as *Prompt-biased Code* or *Hallucinated Object* as the docstring is less detailed and the task more complex in CoderEval. We believe that our study and presented bug patterns not only confirm their findings but also improve their results by exploring bug patterns in more practical tasks and not limited to the error types reported by static analysis tools.

Jesse et al. (2023) examined the proneness of LLMs in generating Simple, Stupid Bugs (SStuBs). Stupid bugs are bugs that require single statement fixes but are challenging to locate. To conduct the study, they used a dataset of SStuBs in Java. They used the code before the error line as a prompt and presented it to the LLM and examined whether the model completed the code by regenerating the bug or generating the fix. The results revealed that the LLMs used in their study produced more SStuBs than the correct code. Additionally, SStuBs generated by Codex took significantly longer to be fixed, underscoring the challenge posed by the prevalence of SStuBs and the potential difficulty in distinguishing between natural and incorrect outputs of LLMs. The study also demonstrated that adding comments to the code can, to some extent, assist the model in generating fewer SStuBs. While the focus of this study was on Java, not all equivalent SStuBs in Python were observed in our sample set. However, we categorized bugs with similar characteristics as *Missing Corner Cases*, where a small change or a single statement modification can transform the buggy code into correct code.

The study by Pan et al. (2023) is focused on analyzing the effectiveness of LLMs in code translation and is the only study that as part of their results developed a systematic bug taxonomy of buggy code generated by LLMs when employed for code translation tasks. The characteristics outlined in their taxonomy are particularly relevant to the challenges associated with translating code from one language to another such as *Mismatch of API behavior between source and target*, *Removal of logic from the source code*, and *Mismatch of behavior after replacing API call*.

To the best of our knowledge, our study is the first to systematically analyze the characteristics of LLM-generated buggy code and construct a bug taxonomy based on the observed bug patterns. Furthermore, we are the first to evaluate the appropriateness of our proposed taxonomy on LLM-generated buggy code through a survey study targeting users of LLMs. In contrast to prior studies that based their analyses on programming tasks collected from platforms like LeetCode, our analysis consists of the buggy code generated by three different LLMs using real-world programming tasks collected from the CoderEval benchmark dataset. We assert that our findings contribute to a more comprehensive understanding of bugs in LLM-generated code.

由静态分析工具提出并报告了错误类型的分布。他们的结果显示，通过所有测试用例的代码片段出现的质量问题较少，尽管并非完全没有此类问题。虽然我们没有研究质量问题（因为我们专注于错误而非代码质量问题），但我们数据集中误解类型和缺失边界情况类型的普遍性与他们研究中错误输出类别的普遍性一致。因此，我们在更实际的任务中（相比之下 LeetCode 竞赛较为简单）得出了类似的观察结果。此外，他们在 LeetCode 任务中发现的编译错误和运行时错误与我们的一种或多种类别相匹配，例如错误属性或输入类型错误。我们研究中发现的许多错误类别在他们的研究中没有发现，例如虚构对象或未提示的考量。这种差异很可能是由于所研究任务的性质不同：CoderEval 任务更加实际（因为

Jesse 等人（2023）研究了大语言模型（LLMs）生成简单愚蠢的错误（SStuBs）的倾向。愚蠢的错误是指只需单条语句即可修复但难以定位的错误。为进行该研究，他们使用了一个 Java 语言 SStuBs 数据集。他们使用错误行之前的代码作为提示，呈现给 LLM，并检查模型是通过重新生成错误代码还是生成修复代码来完成代码。结果显示，他们研究中使用的 LLM 生成的 SStuBs 比正确代码更多。此外，Codex 生成的 SStuBs 修复所需时间明显更长，这凸显了 SStuBs 的普遍存在所带来的挑战，以及区分 LLM 的自然输出与错误输出的潜在困难。研究还表明，在代码中添加注释在某种程度上可以帮助模型生成更少的 SStuBs。虽然本研究的重点是 Java，但在我们的样本集中并未观察到所有等效的 Python SStuBs。然而

Pan 等人（2023 年）的研究侧重于分析大型语言模型（LLMs）在代码翻译中的有效性，并且是唯一一项在其结果中开发了针对 LLM 在代码翻译任务中生成的有缺陷代码的系统性错误分类的研究。他们分类中概述的特征与将代码从一种语言翻译到另一种语言所面临的挑战尤其相关，例如源代码和目标代码 API 行为不匹配、源代码逻辑被移除，以及替换 API 调用后的行为不匹配。

据我们所知，我们的研究是首个系统分析大语言模型（LLM）生成的有缺陷代码特征并根据观察到的缺陷模式构建缺陷分类的研究。此外，我们也是首个通过针对LLM用户的调查研究评估我们提出的分类在LLM生成的有缺陷代码中的适用性的研究。与先前基于从LeetCode等平台收集的编程任务进行分析的研究不同，我们的分析包括由三种不同LLM使用从CoderEval基准数据集收集的真实编程任务生成的有缺陷代码。我们断言，我们的发现有助于更全面地理解LLM生成代码中的缺陷。

7 Threats to Validity

Construct validity Our methodology may pose a potential threat as the process of collecting and labeling buggy samples could influence our results. While our methodology is similar to that of existing works that categorized bugs (Tambon et al. 2024; Humbatova et al. 2020; Nikanjam et al. 2021), we addressed this threat by meticulously describing our approach, to allow for external validations. Another limitation could come from the dataset we used: Python functions and the three examined LLMs. Python is a widely used programming language in various domains, and the examined LLMs have been used in previous studies for code generation tasks (Chen et al. 2022; Zan et al. 2022; Nguyen and Nadi 2022; Lemieux et al. 2023; Chen et al. 2023). In the absence of a pre-existing taxonomy for categorizing bugs, we recognized the risk of introducing bias into our classification. To mitigate this risk, we followed an open coding procedure. Each rater independently evaluated buggy samples, and conflicts were discussed until a consensus was reached. Additionally, to validate our categories, we surveyed practitioners who employed LLMs for code generation. Finally, removing bugs classified as runnable level above *plib_runnable* could introduce a loss of information regarding the bug patterns identified. Nonetheless, any bugs in the code generated from the prompt of those categories would not necessarily reflect a weakness of LLM in generating code but just a lack of proper information. As such, we preferred removing them and potentially losing some information rather than adding noises to our study.

Internal Validity One source of internal threats to validity is the potential bias in manual inspection and labeling of buggy samples. To alleviate this threat, the first two authors (two senior Ph.D. candidates who have experience in SE research and programming) labeled buggy code samples independently. After a series of meetings with the third author (a senior researcher with 10 years of research experience in SE and AI), a consensus was reached on the labeling process and the criteria for categorizing different bug patterns. Another threat arises from the sampling method used to select buggy snippets from CoderEval. To address potential sampling bias, we followed similar statistical approaches (Chen et al. 2020; Zhang et al. 2019; Morovati et al. 2024) utilizing stratified random sampling with 95% confidence levels and 5% confidence intervals. Furthermore, we validated through two surveys, following existing studies (Nikanjam et al. 2021; Tambon et al. 2024; Humbatova et al. 2020), to verify that our obtained categories are representative of bug patterns faced by practitioners. In both surveys, we made use of the platform Prolific to hire paid crowd-workers. We manually reviewed all submissions to ensure their quality. For the second survey, we complemented our panel with an email to authors of studies or repositories related to Code LLMs. The response rate for email solicitations was 8.9%, in line with recent survey studies in SE that received response rates of 8.4% to 13.3% (Nardone et al. 2023; Nikanjam et al. 2021; Cater-Steel et al. 2005).

External Validity One external threat to the validity is the LLMs used to generate the code in the dataset. In this study, three LLMs were used, CodeGen, PanGu-Coder, and Codex: the first two are open-source and have been used in previous studies that harness the power of LLMs for code generation tasks (Chen et al. 2022, 2023; Zan et al. 2022; Bogatinovski and Kao 2023), and the third one, Codex, is the model behind Copilot. Another threat is that the tasks for which the LLMs generated code may not be representative of real tasks which could limit the relevance of the bug patterns found. The choice of the CoderEval dataset mitigates this issue, as it is based on real tasks mined from GitHub. Nonetheless, future works should consider expanding our study to cover a more diverse set of LLMs and functions (problem and prompt) to generate code. Another threat to external validity stems from our focus on *Python* code. Since *Python* is one of the most popular languages for

有效性的7种威胁

构念效度 我们的方法可能存在潜在威胁，因为收集和标记错误样本的过程可能会影响我们的结果。虽然我们的方法与现有的错误分类研究方法相似（Tambon 等, 2024; Humbatova 等, 2020; Nikanjam 等, 2021），但我们通过详细描述我们的研究方法来应对这一威胁，以便进行外部验证。另一个限制可能来自我们使用的数据集：Python函数及三种所研究的LLM。Python是一种在各种领域广泛使用的编程语言，而所研究的LLM已经在以往的代码生成任务研究中使用过（Chen 等, 2022; Zan 等, 2022; Nguyen 和 Nadi, 2022; Lemieux 等, 2023; Chen 等, 2023）。在没有现成的错误分类体系的情况下，我们意识到在分类过程中可能引入偏差的风险。为降低这种风险，我们采用了开放式编码程序。每位评审者独立评估错误样本，并在冲突时进行处理。

内部有效性 内部有效性威胁的一个来源是手工检查和标注有缺陷样本可能存在的偏差。为了减轻这一威胁，前两位作者（两位具有软件工程研究和编程经验的高级博士生）独立标注了有缺陷的代码样本。在与第三位作者（具有10年软件工程和人工智能研究经验的资深研究人员）进行一系列会议后，就标注过程和不同缺陷模式分类的标准达成了一致。另一个威胁来自于从CoderEval中选择有缺陷代码片段所使用的抽样方法。为了应对潜在的抽样偏差，我们采用了类似的统计方法（Chen et al. 2020；Zhang et al. 2019；Morovati et al. 2024），使用分层随机抽样，置信水平为95%，置信区间为5%。此外，我们还通过两项调查进行了验证，遵循现有研究（Nikanjam et al. 2021；Tambon et al. 2024；Humbatova et al. 2020）。

外部效度外部效度的一个威胁是用于生成数据集代码的LLM。在本研究中，使用了三种LLM：CodeGen、盘古Coder（PanGu-Coder）和Codex；前两者是开源的，并且已在以往利用LLM进行代码生成任务的研究中使用（Chen等，2022, 2023；Zan等，2022；Bogatinovski和Kao，2023），第三种Codex则是Copilot背后的模型。另一个威胁是LLM生成代码的任务可能不能代表真实任务，这可能限制发现的漏洞模式的相关性。选择CoderEval数据集可以减轻这一问题，因为它是基于从GitHub挖掘的真实任务。然而，未来的研究应考虑扩大我们的研究范围，以涵盖更多样化的LLM和函数（问题和提示）来生成代码。另一个对外部效度的威胁来自我们对Python代码的关注。由于Python是最流行的语言之一

development and is widely used in benchmarks for assessing LLM, we believe that focusing on this programming language for analyzing the bugs could have the most impact. Moreover, several taxonomies are programming language-specific (Le Goues et al. 2015; Pan et al. 2009; Humbatova et al. 2020) to limit their scope. While we believe that some bug patterns would transfer to other programming languages (e.g., Syntax Error), our surveys showed that some bug patterns encountered by respondents are either language-specific (e.g., null-pointer) or task-specific (e.g., optimization issues). While we included in our taxonomy these patterns as reported by the first survey's participants, future works should consider expanding this study to include other programming languages or benchmarks focused on a particular task. The last threat is regarding the rapid development of LLMs and the long-term relevance of the proposed taxonomy. While future studies could expand this taxonomy based on the evolving landscape of LLM advancements, we aimed to draw attention to the distinctive nature of bugs in code generated by LLMs compared to human bugs, even in common types like syntax errors. Moreover, we believe the bug patterns remain broad and will not be overshadowed by the advancements in LLMs over time. Categories such as *Prompt-biased* code, *Hallucinated Objects*, or *Wrong Input Type* are linked to the limitations arising from the generative nature of LLMs. While they may see a reduction in frequency with improved model performance, they are likely to persist.

Conclusion Validity The conclusions drawn in this study have certain limitations, notably the potential for wrong/missing bug patterns. We conducted manual inspections of 666 buggy snippets from two pools of over 1,900 samples from CoderEval. Similarly, buggy snippets were generated by 3 different LLMs and 2 types of prompts to ensure the reliability of our findings. In both instances, the sample sizes were deemed substantial enough to be representative and avoid leading to misguided conclusions. Additionally, to minimize potential errors, bug labeling was independently conducted by two raters and subsequently discussed. Moreover, validation surveys were used to compare our results to users' experience and feedback. Lastly, we have provided a replication package (The replication package 2023) to facilitate the reproducibility of our findings, enabling other researchers to expand upon our study.

8 Conclusion and Future Work

In this paper, we empirically investigated the bug patterns observed in code generated by LLMs. Our analysis provides insights into the nature, frequency, and distribution of these bug patterns. We organized our examination of the patterns in a taxonomy for buggy code generated by LLMs. These analyses lead to interesting observations on different bug patterns found in LLM-generated code, such as *Prompt-biased Code*, *Wrong Attribute* and *Hallucinated Object*, which should not be common in human buggy code. In addition, certain bug patterns originate from the stochastic generative nature of LLMs such as *Hallucinated Object* or *Wrong Attribute* which are also not common in human-written buggy code. Moreover, we show that the type of prompts used have an impact on the bug patterns present in the generated codes, with human-labeled prompts generally decreasing the frequency of such bug patterns compared to prompts based on the raw docstring.

We evaluated the relevance of our findings by conducting two online surveys targeting software practitioners and researchers involved in LLMs. The first survey allowed us to complete our taxonomy by adding some bug patterns not observed but reported by participants such as Security bugs. This can serve as a basis for future expansion of the taxonomy. In the second survey, the respondents declared that they mostly encountered the bug patterns we

开发并且在评估大型语言模型 (LLM) 的基准测试中被广泛使用，我们认为专注于这种编程语言来分析漏洞可能会产生最大的影响。此外，一些分类体系是针对特定编程语言的 (Le Goues 等, 2015；Pan 等, 2009；Humbatova 等, 2020)，以限制其范围。虽然我们认为一些漏洞模式可以迁移到其他编程语言（例如语法错误），但我们的调查显示，受访者遇到的一些漏洞模式要么是特定语言的（例如空指针），要么是特定任务的（例如优化问题）。虽然我们在分类体系中包括了这些由第一次调查参与者报告的模式，但未来的研究应考虑扩展该研究，以涵盖其他编程语言或针对特定任务的基准测试。最后一个威胁是关于 LLM 的快速发展以及所提出分类体系的长期相关性。虽然未来的研究可以基于不断发展的 L

结论有效性 本研究得出的结论存在一定局限性，尤其是存在错误或缺失的缺陷模式的可能性。我们对来自 CoderEval 超过 1,900 个样本的两个样本池中的 666 个有缺陷代码片段进行了手工检查。同样，这些有缺陷的代码片段是由 3 个不同的大型语言模型（LLM）和 2 种类型的提示生成的，以确保我们研究结果的可靠性。在这两种情况下，样本量都被认为足够大，具有代表性，并且能够避免导致误导性结论。此外，为了将潜在错误降到最低，缺陷标注由两名评审独立进行，并随后进行讨论。此外，我们还使用验证调查将我们的结果与用户的经验和反馈进行了比较。最后，我们提供了一个可复现的数据包（The replication package 2023），以方便研究结果的可重复性，使其他研究人员能够在我们的研究基础上进一步扩展。

8 结论与未来工作

在本文中，我们通过实证研究了在大型语言模型（LLMs）生成的代码中观察到的错误模式。我们的分析提供了关于这些错误模式的性质、频率和分布的见解。我们将对这些模式的检查整理成了一个LLMs生成的有错误代码的分类体系。这些分析带来了关于LLMs生成代码中不同错误模式的有趣观察，例如提示偏差代码、错误属性和虚构对象，这些在人工生成的有错误代码中不应常见。此外，某些错误模式源于LLMs的随机生成特性，如虚构对象或错误属性，这在人工编写的有错误代码中也不常见。此外，我们还表明，使用的提示类型会影响生成代码中存在的错误模式，相比基于原始文档字符串的提示，人为标注的提示通常会降低此类错误模式的频率。

我们通过针对从事大型语言模型（LLM）的软件从业者和研究人员进行两项在线调查来评估我们研究结果的相关性。第一项调查使我们能够通过添加一些参与者报告但未观察到的缺陷模式（如安全漏洞）来完善我们的分类法。这可以作为未来扩展分类法的基础。在第二项调查中，受访者表示他们大多数情况下遇到的是我们所列的缺陷模式。

observed in our samples. Moreover, we noted a strong correlation between the bug patterns frequency as labeled in our samples using the human-labeled docstring and practitioners' reported frequency. Practitioners further assessed that, while the majority of the bug patterns can be relatively easy to diagnose and fix, several categories such as *Misinterpretation* or *Non-Prompted Consideration* are difficult to diagnose and fix. Beyond that, the patterns of bugs in LLMs generated code tend to be different from the bug patterns of human developers even in categories such as *Syntax Error*. Finally, we observed that, while both academic and industrial participants have overall a similar assessment of the frequency, difficulty of diagnosing/fixing and complexity of bug patterns, there exist some discrepancies between the groups on some bug patterns which might be explained by each practitioner's group experience and usage of the tools.

Our results serve as an initial taxonomy of bug patterns observed in LLM-generated code. While further investigation is required to compare their appearance in human-written code and other LLMs, the findings in this study combined with the validation surveys represent the first steps in characterizing bugs in LLM-generated code. This opens up avenues for future studies on enhancing the quality of code generated by LLM-based programming assistant tools. Our findings also can be leveraged to improve different SE tools that rely on characteristics of human buggy code, such as mutant operators in MT or repair rules in APR tools, as well as potential development in Computer Science (CS) education studies.

Appendix A: Reddit Post

We posted our survey questionnaire on two relevant Reddit channels: *LocalLLaMA* and *MachineLearning*. Figure 7 shows the posted survey on Reddit.

Fig. 7 The posted survey on Reddit

在我们的样本中观察到的情况。此外，我们注意到使用人工标注的文档字符串对样本中错误模式频率标注与从业者报告的频率之间存在强相关性。从业者进一步评估认为，虽然大多数错误模式相对容易诊断和修复，但某些类别如误解或未提示考虑则较难诊断和修复。除此之外，即使在语法错误等类别中，LLM生成代码中的错误模式往往也不同于人类开发者的错误模式。最后，我们观察到，尽管学术界和工业界的参与者在错误模式的频率、诊断/修复难度以及复杂性方面总体评估相似，但在某些错误模式上两组之间仍存在一些差异，这可能可以通过每位从业者的群体经验和工具使用情况来解释。

我们的研究结果作为对在大语言模型生成的代码中观察到的错误模式的初步分类。虽然还需要进一步研究以比较它们在人类编写的代码和其他大语言模型中的出现情况，但本研究的发现结合验证调查，代表了对大语言模型生成代码中的错误进行特征化的第一步。这为未来研究如何提升基于大语言模型的编程辅助工具生成代码的质量开辟了途径。我们的发现还可以用于改进依赖于人类错误代码特征的不同软件工程工具，例如MT中的突变操作符或APR工具中的修复规则，以及计算机科学教育研究中的潜在发展。

附录 A：Reddit 帖子

我们在两个相关的 Reddit 频道上发布了我们的调查问卷：LocalLLaMA 和 MachineLearning。图 7 显示了在 Reddit 上发布的调查问卷。

图7 在Reddit上发布的调查

Appendix B: Survey

To prepare both surveys, we used Google forms (2022), a well-known online tool for creating and sharing surveys and quizzes. The survey has two parts inspired by previous empirical studies on bug patterns (Nikanjam et al. 2021; Humbatova et al. 2020; Tambon et al. 2024). In the first part, we asked demographic questions to participants i.e. what is your current job title, what LLMs they have used, and for what programming languages they have used LLMs to generate code, other than Python (since it is the programming language of the given code snippets), as presented in Fig. 8. Then, for the first survey, we aimed to have feedback from practitioners about bug patterns in LLM-generated code they encountered without revealing our taxonomy to them. So, in one question, we ask for at least two such bug patterns participants observed in LLM-generated code, and to give a quick description of such bugs, as illustrated in Fig. 9.

For the second survey, we asked for more specific questions about bugs in LLMs generated code, more precisely, we asked:

– How often do you encounter mistakes in LLMs generated code you use? Those mistakes can be anything, from code that will not compile (variable not defined, function that does

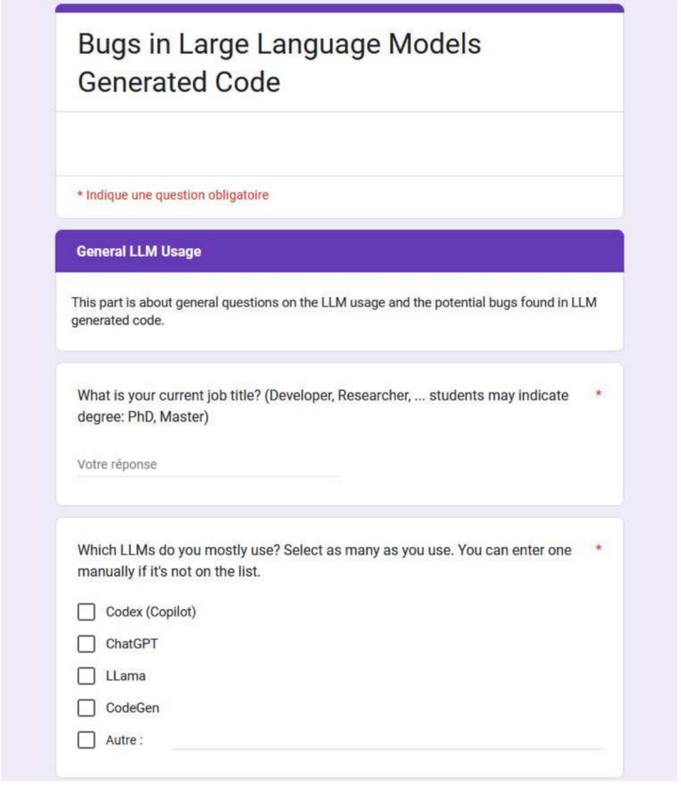


Fig. 8 The first part, for both surveys, received by the participant

附录 B：调查

为了准备这两份问卷，我们使用了 Google 表单（2022），这是一种著名的在线工具，用于创建和分享问卷和测验。问卷分为两部分，灵感来源于以往关于错误模式的实证研究（Nikanjam 等, 2021；Humbatova 等, 2020；Tambon 等, 2024）。在第一部分，我们向参与者询问人口统计问题，即您当前的职位是什么，您使用过哪些大型语言模型（LLM），以及除了 Python（因为给定的代码片段使用的是该编程语言）之外，您使用 LLM 生成代码的编程语言，如图 8 所示。随后，对于第一份问卷，我们旨在获取从业者关于他们在 LLM 生成代码中遇到的错误模式的反馈，而不向他们透露我们的分类法。因此，在一个问题中，我们要求参与者至少列出两种他们在 LLM 生成代码中观察到的此类错误模式，并对这些错误进行简要描述，如图 9 所示。

在第二次调查中，我们提出了关于大型语言模型生成代码中的错误的更具体问题，更准确地说，我们问了：

– 你在使用大型语言模型生成的代码时多久会遇到错误？这些错误可以是任何类型的，从无法编译的代码（变量未定义、函数未实现）

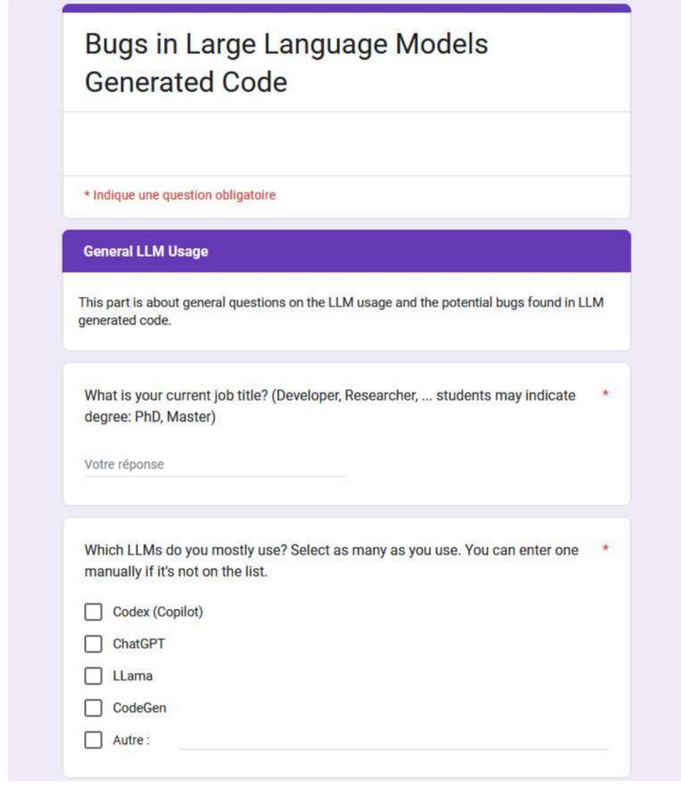


图8 两项调查中，参与者收到的第一部分

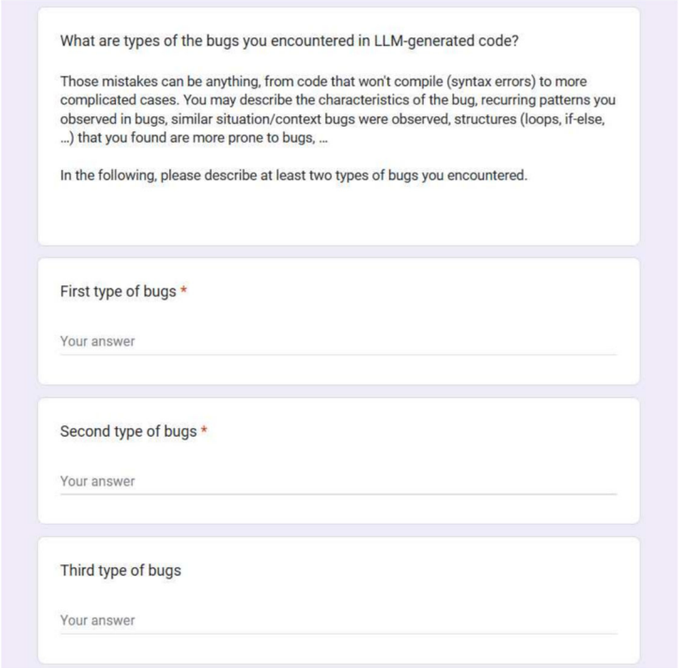


Fig. 9 The question for types of the bugs the participants encountered in LLM-generated code in the first survey

not exist) to more silly choices from the LLMs (multiples If condition checking the same thing or casting back and forth a variable for no reason).
– When you encounter mistakes in LLMs generated code, how complex is fixing those issues?
– To fix those issues, how do you proceed?

For the first three questions, we used a 5-level Likert scale (Oppenheim 2000) (Never to Always or Trivial to Very Hard). For, the third one, the respondents could choose between "Manually", "Re-prompting the LLM", "Combination of both" or "Other" (and they were given the possibility to write down how).

In the second part of the survey, we wanted to consolidate our bug patterns and gather the participants' feedback on several aspects of each pattern of bug in our study. An example of questions in the second part is shown in Fig. 10. The structure of the questions is similar for each pattern of bugs in the taxonomy.

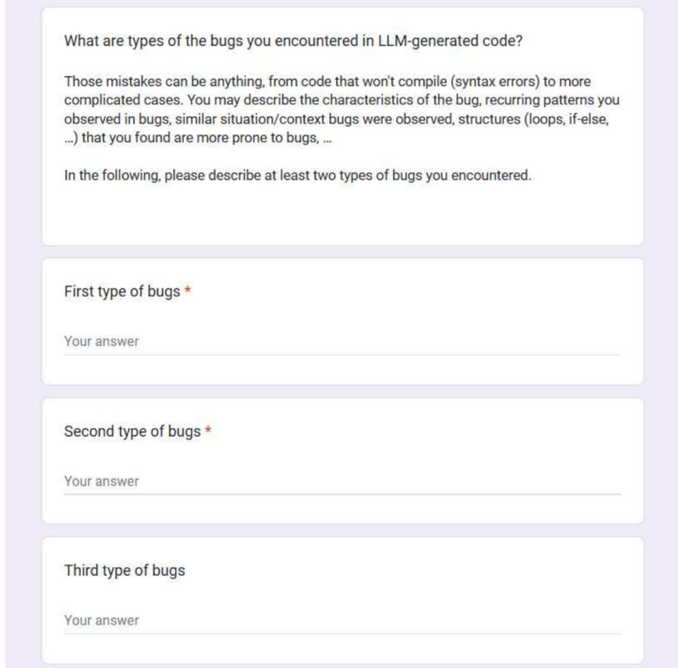
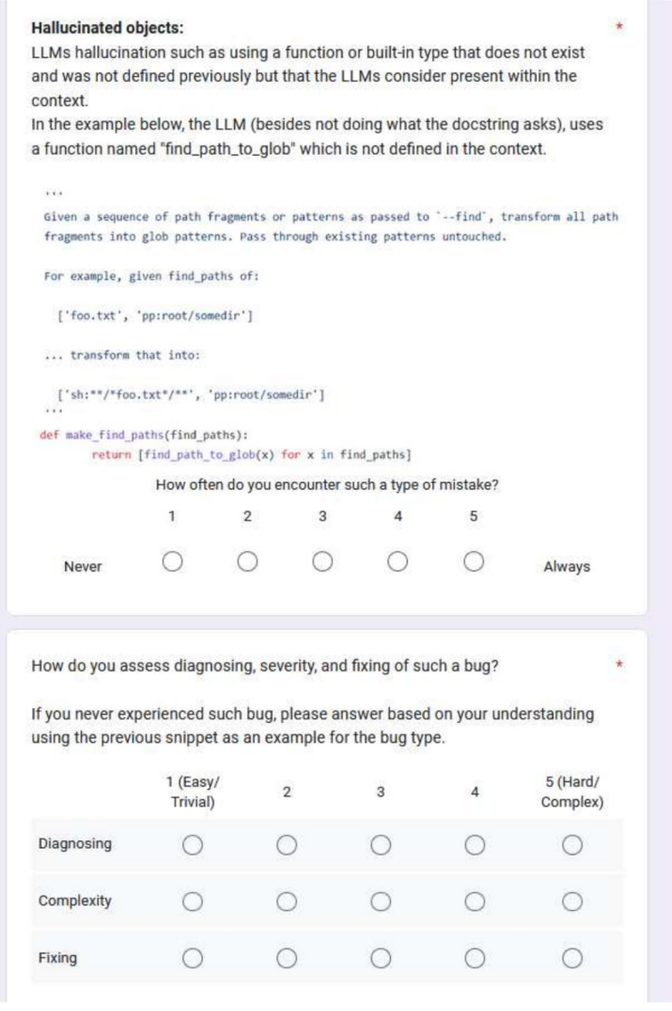


图9 参与者在第一次调查中遇到的LLM生成代码中漏洞类型的问题

不存在) 到来自大型语言模型的更愚蠢的选择（多个 If 条件检查同一件事，或无缘无故地来回转换变量）。 – 当你遇到大型语言模型生成的代码中的错误时，修复这些问题有多复杂？ – 为了修复这些问题，你会如何进行？

对于前三个问题，我们使用了五级李克特量表（Oppenheim 2000）（从从不到总是或从微不足道到非常困难）。对于第三个问题，受访者可以选择“手动”、“重新提示大型语言模型（LLM）”、“两者结合”或“其他”（并且他们可以写下具体做法）。

在调查的第二部分，我们希望整合我们的缺陷模式，并收集参与者对我们研究中每种缺陷模式的几个方面的反馈。第二部分问题的示例如图10所示。每种缺陷模式的问题结构在分类法中是相似的。



Hallucinated objects: *

LLMs hallucination such as using a function or built-in type that does not exist and was not defined previously but that the LLMs consider present within the context.
In the example below, the LLM (besides not doing what the docstring asks), uses a function named "find_path_to_glob" which is not defined in the context.

```
'''
Given a sequence of path fragments or patterns as passed to `--find`, transform all path
fragments into glob patterns. Pass through existing patterns untouched.

For example, given find_paths of:

  ['foo.txt', 'pp:root/somedir']

... transform that into:

  ['sh:**/*foo.txt*/**', 'pp:root/somedir']
'''
def make_find_paths(find_paths):
      return [find_path_to_glob(x) for x in find_paths]
```

How often do you encounter such a type of mistake?

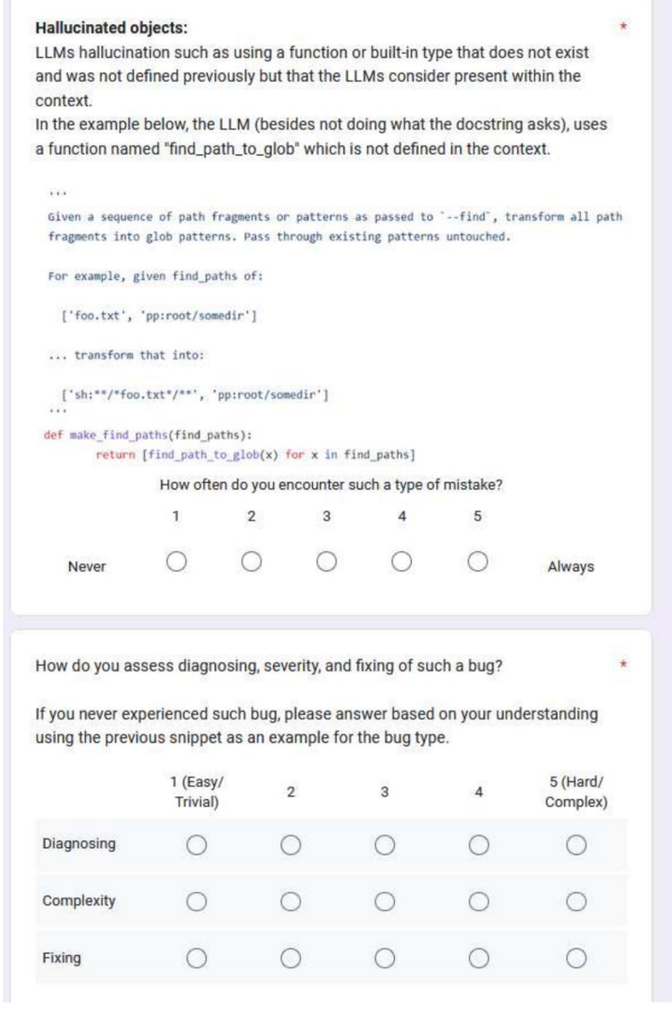
	1	2	3	4	5	
Never	○	○	○	○	○	Always

How do you assess diagnosing, severity, and fixing of such a bug? *

If you never experienced such bug, please answer based on your understanding using the previous snippet as an example for the bug type.

	1 (Easy/ Trivial)	2	3	4	5 (Hard/ Complex)
Diagnosing	○	○	○	○	○
Complexity	○	○	○	○	○
Fixing	○	○	○	○	○

Fig. 10 Example of questions for a bug pattern in the second survey. The description of the task is collected from the CoderEval dataset (Yu et al. 2023)



Hallucinated objects: *

LLMs hallucination such as using a function or built-in type that does not exist and was not defined previously but that the LLMs consider present within the context.
In the example below, the LLM (besides not doing what the docstring asks), uses a function named "find_path_to_glob" which is not defined in the context.

```
'''
Given a sequence of path fragments or patterns as passed to `--find`, transform all path
fragments into glob patterns. Pass through existing patterns untouched.

For example, given find_paths of:

  ['foo.txt', 'pp:root/somedir']

... transform that into:

  ['sh:**/*foo.txt*/**', 'pp:root/somedir']
'''
def make_find_paths(find_paths):
      return [find_path_to_glob(x) for x in find_paths]
```

How often do you encounter such a type of mistake?

	1	2	3	4	5	
Never	○	○	○	○	○	Always

How do you assess diagnosing, severity, and fixing of such a bug? *

If you never experienced such bug, please answer based on your understanding using the previous snippet as an example for the bug type.

	1 (Easy/ Trivial)	2	3	4	5 (Hard/ Complex)
Diagnosing	○	○	○	○	○
Complexity	○	○	○	○	○
Fixing	○	○	○	○	○

图10 第二次调查中某个错误模式问题的示例。任务描述收集自 CoderEval 数据集（Yu 等，2023）

Acknowledgements The authors would like to thank the participants who contributed to the survey questionnaires.

Data Availability The dataset generated during the current study is available in the replication package, which is accessible via (The replication package 2023).

Declarations

Conflict of interest The authors declare that they have no Conflict of interest.

References

Anthropic (2013) Model card and evaluations for claude models. https://www.anthropic.com/index/introducing-claude
Asare O, Nagappan M, Asokan N (2023) Is github's copilot as bad as humans at introducing vulnerabilities in code? Empirical Softw Eng 28(6). https://doi.org/10.1007/s10664-023-10380-1
Bird C, Ford D, Zimmermann T, Forsgren N, Kalliamvakou E, Lowdermilk T, Gazit I (2022) Taking flight with copilot: early insights and opportunities of ai-powered pair-programming tools. Queue 20(6):35–57
Bogatinovski J, Kao O (2023) Auto-logging: AI-centred logging instrumentation. In: 2023 IEEE/ACM 45th international conference on software engineering: new ideas and emerging results (ICSE-NIER). IEEE, pp 95–100
Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A et al (2020) Language models are few-shot learners. Adv Neural Inf Process Syst 33:1877–1901
Cater-Steel A, Toleman M, Rout T (2005) Addressing the challenges of replications of surveys in software engineering research. In: 2005 International symposium on empirical software engineering, 2005. IEEE, pp 10–pp
Catolino G, Palomba F, Zaidman A, Ferrucci F (2019) Not all bugs are the same: understanding, characterizing, and classifying bug types. J Syst Softw 152:165–181
Chen Z, Cao Y, Liu Y, Wang H, Xie T, Liu X (2020) A comprehensive study on challenges in deploying deep learning based software. In: Proceedings of the 28th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering pp 750–762
Chen X, Lin M, Schärli N, Zhou D (2023) Teaching large language models to self-debug. arXiv:2304.05128
Chen M, Tworek J, Jun H, Yuan Q, de Oliveira Pinto HP, Kaplan J, Edwards H, Burda Y, Joseph Y, Brockman G, Ray A, Puri R, Krueger G, Petrov M, Khlaaf H, Sastry G, Mishkin P, Chan B, Gray S, Ryder N, Pavlov M, Power A, Kaiser L, Bavarian M, Winter C, Tillet P, Such FP, Cummings D, Plappert M, Chantzis F, Barnes E, Herbert-Voss A, Guss WH, Nichol A, Paino A, Tezak N, Tang J, Babuschkin I, Balaji S, Jain S, Saunders W, Hesse C, Carr AN, Leike J, Achiam J, Misra V, Morikawa E, Radford A, Knight M, Brundage M, Murati M, Mayer K, Welinder P, McGrew B, Amodei D, McCandlish S, Sutskever I, Zaremba W (2021) Evaluating large language models trained on code
Chen M, Tworek J, Jun H, Yuan Q, Pinto HPdO, Kaplan J, Edwards H, Burda Y, Joseph N, Brockman G et al (2021) Evaluating large language models trained on code. arXiv:2107.03374
Chen B, Zhang F, Nguyen A, Zan D, Lin Z, Lou J-G, Chen W (2022) Codet: code generation with generated tests. arXiv:2207.10397
Chowdhery A, Narang S, Devlin J, Bosma M, Mishra G, Roberts A, Barham P, Chung HW, Sutton C, Gehrmann S et al (2023) Palm: scaling language modeling with pathways. J Mach Learn Res 24(240):1–113
Christopoulou F, Lampouras G, Gritta M, Zhang G, Guo Y, Li Z, Zhang Q, Xiao M, Shen M, Li L et al (2022) Pangu-coder: program synthesis with function-level language modeling. arXiv:2207.11280
Cohen J (2013) Statistical power analysis for the behavioral sciences. Routledge
Cramér H (1999) Mathematical methods of statistics. Princeton university press, vol 26
Du X, Liu M, Wang K, Wang H, Liu J, Chen Y, Feng J, Sha C, Peng X, Lou Y (2023) Classeval: a manually-crafted benchmark for evaluating llms on class-level code generation
Fan Z, Gao X, Mirchev M, Roychoudhury A, Tan SH (2023) Automated repair of programs from large language models. In: 2023 IEEE/ACM 45th international conference on software engineering (ICSE). IEEE, pp 1469–1481
Fischer F, Böttinger K, Xiao H, Stransky C, Acar Y, Backes M, Fahl S (2017) Stack overflow considered harmful? the impact of copy& paste on android application security. In: IEEE symposium on security and privacy (SP) pp 121–136

致谢 作者们感谢参与问卷调查的参与者。

数据可用性 本研究生成的数据集可在复制包中获取，该复制包可通过（复制包 2023）访问。

声明

利益冲突 作者声明他们没有利益冲突。

参考文献

Anthropic（2013）Claude 模型的模型卡和评估。https://www.anthropic.com/index/introducing-claude Asare O, Nagappan M, Asokan N（2023）GitHub 的 Copilot 在引入代码漏洞方面是否像人类一样糟糕？Empirical Softw Eng 28(6). https://doi.org/10.1007/s10664-023-10380-1 Bird C, Ford D, Zimmermann T, Forsgren N, Kalliamvakou E, Lowdermilk T, Gazit I（2022）使用 Copilot 起飞：AI 驱动的结对编程工具的初步见解和机会。Queue 20(6):35–57 Bogatinovski J, Kao O（2023）自动日志记录：以 AI 为中心的日志记录工具。在：2023 IEEE/ACM 第 45 届国际软件工程会议：新思想与新兴成果（ICSE-NIER）。IEEE，第 95–100 页 Brown T, Mann B, Ryder N, Subbiah M, Kaplan J D, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A 等（2020）语言模型是少量样本学习者。Adv Neural Inf Process Syst 33:1877–1901 Cater-Steel A, Toleman M, Rout T（2005）应对复制调查的挑战

Fisher RA (1922) On the interpretation of χ 2 from contingency tables, and the calculation of p. J R Stat Soc 85(1):87–94. http://www.jstor.org/stable/2340521
Fraser G, Arcuri A (2014) A large-scale evaluation of automated unit test generation using evosuite. ACM Trans Softw Eng Methodol 24(2). https://doi.org/10.1145/2685612
Fu Y, Liang P, Tahir A, Li Z, Shahin M, Yu J (2023) Security weaknesses of copilot generated code in github. arXiv:2310.02059
gitindex (2023). https://githut.info/
Google forms (2022) https://www.google.ca/forms/about/
Guilherme V, Vincenzi A (2023) An initial investigation of chatgpt unit test generation capability. In: Proceedings of the 8th brazilian symposium on systematic and automated software testing, ser. SAST '23. New York, NY, USA: Association for Computing Machinery pp 15-24. https://doi.org/10.1145/3624032.3624035
Gyimesi P, Vancsics B, Stocco A, Mazinanian D, Beszédes Á, Ferenc R, Mesbah A (2021) Bugsjs: a benchmark and taxonomy of javascript bugs. Softw Test Verif Reliab 31(4):e1751
Honarvar S, van der Wilk M, Donaldson A (2023) Turbulence: systematically and automatically testing instruction-tuned large language models for code. arXiv:2312.14856
Huang D, Zhang JM, Qing Y, Cui H (2024) Effibench: benchmarking the efficiency of automatically generated code
Humbatova N, Jahangirova G, Bavota G, Riccio V, Stocco A, Tonella P (2020) Taxonomy of real faults in deep learning systems. In: Proceedings of the ACM/IEEE 42nd international conference on software engineering pp 1110–1121
Imai S (2022) Is github copilot a substitute for human pair-programming? an empirical study. In: Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings pp 319–321
Jesse Z, Ahmed T, Devanbu PT, Morgan E (2023) Large language models and simple, stupid bugs. In: 2023 IEEE/ACM 20th international conference on mining software repositories (MSR). Los Alamitos, CA, USA: IEEE Computer Society, pp. 563–575. https://doi.ieeecomputersociety.org/10.1109/MSR59073.2023.00082
Jiang J, Wang F, Shen J, Kim S, Kim S (2024) A survey on large language models for code generation. arXiv:2406.00515
Ji Z, Ma P, Li Z, Wang S (2023) Benchmarking and explaining large language model-based code generation: a causality-centric approach
Jin M, Shahriar S, Tufano M, Shi X, Lu S, Sundaresan N, Svyatkovskiy A (2023) Inferfix: end-to-end program repair with llms," In: Proceedings of the 31st ACM joint european software engineering conference and symposium on the foundations of software engineering, ser. ESEC/FSE 2023. New York, NY, USA: Association for Computing Machinery, pp 1646-1656. https://doi.org/10.1145/3611643.3613892
Just R, Jalali D, Inozemtseva L, Ernst MD, Holmes R, Fraser G (2014) Are mutants a valid substitute for real faults in software testing?. In: Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering, ser. FSE 2014. New York, NY, USA: Association for Computing Machinery pp 654-665. https://doi.org/10.1145/2635868.2635929
Kazemitabaar M, Hou X, Henley A, Ericson BJ, Weintrop D, Grossman T (2023) How novices use llm-based code generators to solve cs1 coding tasks in a self-paced learning environment. arXiv:2309.14049
Kou B, Chen S, Wang S, Ma L, Zhang T (2023) Is model attention aligned with human attention? an empirical study on large language models for code generation
Le Goues C, Holtschulte N, Smith EK, Brun Y, Devanbu P, Forrest S, Weimer W (2015) The manybugs and introclass benchmarks for automated repair of c programs. IEEE Trans Softw Eng 41(12):1236–1256
Leetcode contest (2023). https://leetcode.com/contest/
Lemieux C, Inala JP, Lahiri SK, Sen S (2023) Codamosa: escaping coverage plateaus in test generation with pre-trained large language models. In: Accepted by 45th international conference on software engineering (ICSE)
Li J, Li G, Li Y, Jin Z (2023) Structured chain-of-thought prompting for code generation. arXiv:2305.06599
Liu MX, Sarkar A, Negreanu C, Zorn B, Williams J, Toronto N, Gordon AD (2023) "what it wants me to say": bridging the abstraction gap between end-user programmers and code-generating large language models. In: Proceedings of the 2023 CHI conference on human factors in computing systems pp 1–31
Liu Y, Le-Cong T, Widyasari R, Tantithamthavorn C, Li L, Le X-BD, Lo D (2023) Refining chatgpt-generated code: characterizing and mitigating code quality issues
Liu Y, Tantithamthavorn C, Liu Y, Li L (2023) On the reliability and explainability of automated code generation approaches. arXiv:2302.09587
Liu J, Xia CS, Wang Y, Zhang L (2023) Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. arXiv:2305.01210

Fisher RA (1922) 关于列联表中 χ 2 的解释及 p 值的计算. J R Stat Soc 85(1):87–94. http://www.jstor.org/stable/2340521Fraser G, Arcuri A (2014) 使用 evosuite 进行自动化单元测试生成的大规模评估. ACM Trans Softw Eng Methodol 24(2). https://doi.org/10.1145/2685612Fu Y, Liang P, Tahir A, Li Z, Shahin M, Yu J (2023) Git Hub 上 Copilot 生成代码的安全弱点. arXiv:2310.02059gitindex (2023). https://githut.info/Google forms (2022) https://www.google.ca/forms/about/Guilherme V, Vincenzi A (2023) ChatGPT 单元测试生成能力的初步研究. 载于: 第八届巴西系统化与自动化软件测试研讨会论文集, 系列 SAST ' 23. 纽约, NY, 美国: 计算机协会, 页 15-24. https://doi.org/10.1145/3624032.3624035Gyimesi P, Vancsics B, Stocco A, Mazinanian D, Beszédes Á, Ferenc R, Mesbah A (2021) Bugsjs: JavaScript 错误的基准测试与分类学.

Liu C, Zhang SD, Jabbarvand R (2024) Codemind: a framework to challenge large language models for code reasoning
Li Z, Wang C, Liu Z, Wang H, Chen D, Wang S, Gao C (2023) Cctest: testing and repairing code completion systems. In: 2023 IEEE/ACM 45th international conference on software engineering (ICSE) pp 1238–1250
Li J, Zhang Y, Karas Z, McMillan C, Leach K, Huang Y (2024) Do machines and humans focus on similar code? exploring explainability of large language models in code summarization. arXiv:2402.14182
Lu S, Guo D, Ren S, Huang J, Svyatkovskiy A, Blanco A, Clement C, Drain D, Jiang D, Tang D et al (2021) Codexglue: a machine learning benchmark dataset for code understanding and generation. arXiv:2102.04664
Mastropaolo A, Pascarella L, Guglielmi E, Ciniselli M, Scalabrino S, Oliveto R, Bavota G (2023) On the robustness of code generation techniques: an empirical study on github copilot. arXiv:2302.00438
Monperrus M (2018) Automatic software repair: a bibliography. ACM Comput Surv 51(1):1. https://doi.org/10.1145/3105906
Moon S, Song Y, Chae H, Kang D, Kwon T, iunn Ong KT, won Hwang S, Yeo J (2023) Coffee: boost your code llms by fixing bugs with feedback
Moradi Dakhel A, Majdinasab V, Nikanjam A, Khomh F, Desmarais MC, Jiang ZMJ (2023) Github copilot ai pair programmer: asset or liability. J Syst Softw 203:111734. https://www.sciencedirect.com/science/article/pii/S0164121223001292
Morovati MM, Nikanjam A, Tambon F, Khomh F, Jiang ZM (2024) Bug characterization in machine learning-based systems. Empir Softw Eng 29(1):14
Muennighoff N, Liu Q, Zebaze A, Zheng Q, Hui B, Zhuo TY, Singh S, Tang X, von Werra L, Longpre S (2023) Octopack: instruction tuning code large language models. arXiv:2308.07124
Nardone V, Muse B, Abidi M, Khomh F, Di Penta M (2023) Video game bad smells: what they are and how developers perceive them. ACM Trans Softw Eng Methodol 32(4). https://doi.org/10.1145/3563214
Nguyen N, Nadi S (2022) An empirical evaluation of github copilot's code suggestions. In: Proceedings of the 19th International Conference on Mining Software Repositories pp 1–5
Nijkamp E, Pang B, Hayashi H, Tu L, Wang H, Zhou Y, Savarese S, Xiong C (2022) Codegen: an open large language model for code with multi-turn program synthesis. arXiv:2203.13474
Nikanjam A, Morovati MM, Khomh F, BenBraiek H (2021) Faults in deep reinforcement learning programs: a taxonomy and a detection approach. Autom Softw Eng 29
Oppenheim AN (2000) Questionnaire design, interviewing and attitude measurement. Bloomsbury Publishing
Pan K, Kim S, Whitehead EJ (2009) Toward an understanding of bug fix patterns. Empir Softw Eng 14:286–315
Pan R, Ibrahimzada AR, Krishna R, Sankar D, Wassi LP, Merler M, Sobolev B, Pavuluri R, Sinha S, Jabbarvand R (2023) Lost in translation: a study of bugs introduced by large language models while translating code. arXiv:2308.03109v3
Sakib FA, Khan SH, Karim A (2023) Extending the frontier of chatgpt: code generation and debugging. arXiv:2307.08260
Schäfer M, Nadi S, Eghbali A, Tip F (2023) Adaptive test generation using a large language model. arXiv:2302.06527
Schulman J, Zoph B, Kim JH, Menick J, Weng J, Uribe JF, Fedus L, Metz L, Pokorny M, et al (2022) Chatgpt: optimizing language models for dialogue
Scoccia GL (2023) Exploring early adopters' perceptions of chatgpt as a code generation tool. In: 2023 38th IEEE/ACM international conference on automated software engineering workshops (ASEW) pp 88–93
Seaman CB (1999) Qualitative methods in empirical studies of software engineering. IEEE Trans Softw Eng 25(4):557–572
Spadini D, Aniche M, Bacchelli A (2018) Pydriller: python framework for mining software repositories. In: Proceedings of the 2018 26th ACM Joint meeting on european software engineering conference and symposium on the foundations of software engineering pp 908–911
Tambon F, Nikanjam A, An L, Khomh F, Antoniol G (2024) Silent bugs in deep learning frameworks: an empirical study of keras and tensorflow. Empir Softw Eng 29(1):10
Tan SH, Yi J, Mechtaev S, Roychoudhury A, et al (2017) Codeflaws: a programming competition benchmark for evaluating automated program repair tools. In: 2017 IEEE/ACM 39th international conference on software engineering companion (ICSE-C). IEEE, pp 180–182
Tan L, Liu C, Li Z, Wang X, Zhou Y, Zhai C (2014) Bug characteristics in open source software. Empir Softw Eng 19:1665–1705
Tang Y, Liu Z, Zhou Z, Luo X (2023) Chatgpt vs sbst: a comparative assessment of unit test suite generation
The replication package (2023). https://github.com/FlowSs/BugsInLLMs
tiobeindex (2023). https://www.tiobe.com/tiobe-index/

刘C, 张SD, Jabbarvand R (2024) Codemind：一个挑战大型语言模型代码推理的框架 李Z, 王C, 刘Z, 王H, 陈D, 王S, 高C (2023) Cctest：测试和修复代码补全系统。在：2023年IEEE/ACM第45届软件工程国际会议（ICSE）第1238–1250页 李J, 张Y, Karas Z, McMillan C, Leach K, 黄Y (2024) 机器和人类关注相似的代码吗？探索大型语言模型在代码总结中的可解释性。arXiv:2402.14182 陆S, 郭D, 任S, 黄J, Svyatkovskiy A, Blanco A, Clement C, Drain D, 江D, 唐D 等 (2021) Codexglue：用于代码理解和生成的机器学习基准数据集。arXiv:2102.04664
Mastropaolo A, Pascarella L, Guglielmi E, Ciniselli M, Scalabrino S, Oliveto R, Bavota G (2023) 关于代码生成技术的鲁棒性：在GitHub Copilot上的一项实证研究。arXiv:2302.00438
Monperrus M (2018) 自动软件修复：文献综述。ACM计算机调查

Touvron H, Martin L, Stone K, Albert P, Almahairi A, Babaei Y, Bashlykov N, Batra S, Bhargava P, Bhosale S et al (2023) Llama 2: open foundation and fine-tuned chat models. arXiv:2307.09288
Vaithilingam P, Zhang T, Glassman EL (2022) Expectation vs. experience: evaluating the usability of code generation tools powered by large language models. In: CHI conference on human factors in computing systems extended abstracts pp 1–7
Verdi M, Sami A, Akhondali J, Khomh F, Uddin G, Motlagh AK (2022) An empirical study of c++ vulnerabilities in crowd-sourced code examples. IEEE Trans Softw Eng 48(5):1497–1514. https://doi.org/10.1109/TSE.2020.3023664
Wei J, Wang X, Schuurmans D, Bosma M, ichter b, Xia F, Chi E, Le QV, Zhou D (2022) Chain-of-thought prompting elicits reasoning in large language models. In: Koyejo S, Mohamed S, Agarwal A, Belgrave D, Cho K, Oh A (eds) Advances in neural information processing systems, vol 35. Curran Associates, Inc., pp 24 824–24 837. https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf
Wong D, Kothig A, Lam P (2022) Exploring the verifiability of code generated by github copilot. arXiv:2209.01766
Yu H, Shen B, Ran D, Zhang J, Zhang Q, Ma Y, Liang G, Li Y, Wang Q, Xie T (2024) Codereval: a benchmark of pragmatic code generation with generative pre-trained models. In: Proceedings of the 46th IEEE/ACM international conference on software engineering pp 1–12
Yu H, Shen B, Ran D, Zhang J, Zhang Q, Ma Y, Liang G, Li Y, Xie T, Wang Q (2023) Codereval: a benchmark of pragmatic code generation with generative pre-trained models. arXiv:2302.00288v1
Zan D, Chen B, Zhang F, Lu D, Wu B, Guan B, Wang Y, Lou J-G (2022) When neural model meets nl2code: a survey. arXiv:2212.09420
Zeng W, Ren X, Su T, Wang H, Liao Y, Wang Z, Jiang X, Yang Z, Wang K, Zhang X et al (2021) Pangu-α: large-scale autoregressive pretrained chinese language models with auto-parallel computation. arXiv:2104.12369
Zhang T, Gao C, Ma L, Lyu M, Kim M (2019) An empirical study of common challenges in developing deep learning applications. In: 2019 IEEE 30th international symposium on software reliability engineering (ISSRE) pp 104–115
Zhang T, Upadhyaya G, Reinhardt A, Rajan H, Kim M (2018) Are code examples on an online q& a forum reliable?: a study of api misuse on stack overflow," In: 2018 IEEE/ACM 40th international conference on software engineering (ICSE) pp 886–896
Zid C, Zampetti F, Antoniol G, Di Penta M (2024) A study on the pythonic functional constructs' understandability. In: Proceedings of the IEEE/ACM 46th international conference on software engineering, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3597503.3639211

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Touvron H, Martin L, Stone K, Albert P, Almahairi A, Babaei Y, Bashlykov N, Batra S, Bhargava P, Bhosale S 等 (2023) Llama 2：开放基础模型与微调聊天模型。arXiv:2307.09288 Vaithilingam P, Zhang T, Glassman EL (2022) 期望与体验：评估大型语言模型驱动的代码生成工具的可用性。在：CHI 人机交互计算系统大会扩展摘要，页 1–7 Verdi M, Sami A, Akhondali J, Khomh F, Uddin G, Motlagh AK (2022) 众包代码示例中的 c++ 漏洞的实证研究。IEEE 软件工程汇刊 48(5):1497–1514. https://doi.org/10.1109/TSE.2020.3023664 Wei J, Wang X, Schuurmans D, Bosma M, Ichter B, Xia F, Chi E, Le QV, Zhou D (2022) 连锁思维提示在大型语言模型中引发推理。在：Koyejo S, Mohamed S, Agarwal A, Belgrave D, Cho K, Oh A (编辑) 神经信息处理系统进展，第 35 卷。Curran Associates, Inc.，页 24824–24837. https://proceedings.neuri

出版者注：在已出版的地图和机构隶属中，施普林格·自然对司法管辖权的声明保持中立。

施普林格·自然或其许可方（例如协会或其他合作伙伴）根据与作者或其他权利持有人的出版协议享有本文的独家权利；作者对本文已接受手稿版本的自我存档仅受该出版协议条款及适用法律的约束。

Authors and Affiliations

Florian Tambon[1] · Arghavan Moradi-Dakhel[1] · Amin Nikanjam[1] · Foutse Khomh[1] · Michel C. Desmarais[1] · Giuliano Antoniol[1]

✉ Florian Tambon
florian-2.tambon@polymtl.ca

✉ Arghavan Moradi-Dakhel
arghavan.moradi-dakhel@polymtl.ca

Amin Nikanjam
amin.nikanjam@polymtl.ca

Foutse Khomh
foutse.khomh@polymtl.ca

Michel C. Desmarais
michel.desmarais@polymtl.ca

Giuliano Antoniol
giuliano.antoniol@polymtl.ca

[1] Polytechnique Montréal, Montréal, Canada

作者及隶属机构

Florian Tambon[1] ·Arghavan Moradi-Dakhel[1] ·Amin Nikanjam[1] · Foutse Khomh[1] ·Michel C. Desmarais[1] ·Giuliano Antoniol[1]

✉ Florian Tambon
florian-2.tambon@polymtl.ca✉
Arghavan Moradi-Dakhel
arghavan.
moradi-dakhel@polymtl.ca Amin
Nikanjam
amin.nikanjam@polymtl.ca Foutse
Khomh foutse.khomh@polymtl.ca
Michel C. Desmarais
michel.desmarais@polymtl.ca
Giuliano Antoniol
giuliano.antoniol@polymtl.ca[1] 蒙特利
尔理工学院，加拿大蒙特利尔